

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»
Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

А. В. Лебедевич, С. Н. Нестеренков

ГЕНЕТИЧЕСКИЕ И ЭВОЛЮЦИОННЫЕ ВЫЧИСЛЕНИЯ: НЕЙРОННЫЕ СЕТИ И ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

Учебно-методическое пособие
по дисциплине «Генетические и эволюционные вычисления»
для студентов II степени высшего образования

Минск БГУИР 2026

УДК _____

ББК _____

Л_____

Р е ц е н з е н т ы:

Лебедевич, А. В.

Генетические и эволюционные вычисления: нейронные сети и генетические алгоритмы : учеб.-метод. пособие / А. В. Лебедевич, С. Н. Нестеренков. – Минск : БГУИР, 2026. – 97 с. : ил.

ISBN 978-985-_____.

Пособие содержит теоретический материал и лабораторный практикум по дисциплине «Генетические и эволюционные вычисления». Рассмотрены основы машинного обучения и искусственных нейронных сетей (персептрон, линейная сеть, многослойный персептрон и алгоритм обратного распространения ошибки, самоорганизующиеся карты Кохонена) и генетических алгоритмов (простой генетический алгоритм, теория схем, операторы селекции, скрещивания, мутации и сокращения популяции, вещественное и перестановочное кодирование, задачи комбинаторной оптимизации). Практикум включает семь лабораторных работ с вариантами заданий, указаниями к выполнению, примерами выполнения и контрольными вопросами.

Предназначено для студентов II ступени высшего образования. Может быть использовано для самостоятельной работы студентов очной и дистанционной форм обучения, а также всеми, кто изучает методы вычислительного интеллекта и оптимизации.

ISBN 978-985-_____

© Лебедевич А. В., Нестеренков С. Н., 2026

© УО «Белорусский государственный университет информатики и радиоэлектроники», 2026

СОДЕРЖАНИЕ

Введение.....	5
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	7
1 Основы машинного обучения и искусственных нейронных сетей.....	7
1.1 Задачи машинного обучения	7
1.2 Функция потерь, метрики и обобщение	7
1.3 Модель искусственного нейрона	8
1.4 Архитектуры нейронных сетей	9
1.5 Вычислительные инструменты Python	10
2 Персептрон и линейные нейронные сети	11
2.1 Персептрон Розенблатта	12
2.2 Линейная разделимость и проблема исключающего ИЛИ	12
2.3 Персептрон с несколькими нейронами	13
2.4 Линейная нейронная сеть ADALINE	13
2.5 Прогнозирование временного ряда линейной сетью	14
2.6 Реализация на Python	15
3 Многослойный персептрон и обратное распространение ошибки.....	15
3.1 Архитектура двухслойной сети	16
3.2 Алгоритм обратного распространения ошибки	17
3.3 Градиентные методы обучения	18
3.4 Предобработка, инициализация и критерии останова	19
3.5 Обобщение: недообучение и переобучение	20
3.6 Реализация на Python	20
4 Самоорганизующиеся карты Кохонена.....	20
4.1 Конкурентное обучение	21
4.2 Архитектура и алгоритм самоорганизующейся карты.....	21
4.3 Оценка качества карты	23
4.4 Реализация на Python	24
5 Основы эволюционных вычислений и генетических алгоритмов.....	24
5.1 Эволюционные вычисления	25
5.2 Основные понятия	25
5.3 Общая структура генетического алгоритма.....	26
5.4 Простой генетический алгоритм	26
5.5 Представление вещественных решений в двоичной форме	28
5.6 Код Грея	28
5.7 Теория схем и фундаментальная теорема ГА	29
6 Модификации генетических операторов	30
6.1 Создание начальной популяции	31
6.2 Селекция	31
6.3 Скрещивание двоичных хромосом	33
6.4 Мутация двоичных хромосом	34

6.5	Вещественное кодирование	34
6.6	Сокращение промежуточной популяции	36
6.7	Настройка параметров и методика эксперимента	36
6.8	Векторная реализация операторов на NumPy	37
7	Задачи комбинаторной оптимизации	37
7.1	Особенности применения генетических алгоритмов	38
7.2	Задача о рюкзаке и учёт ограничений	38
7.3	Задача о минимальном покрытии	40
7.4	Задача коммивояжёра	42
7.5	Локальный поиск 2-орт и меметические алгоритмы	44
8	Современные методы безградиентной оптимизации	45
8.1	Выбор метода оптимизации	46
8.2	Эволюционные стратегии и СМА-ES	46
8.3	Байесовская оптимизация и подбор гиперпараметров	47
8.4	Многокритериальная оптимизация	48
8.5	Эволюция программ с помощью языковых моделей	49
8.6	Реализация на Python	50
	ЛАБОРАТОРНЫЙ ПРАКТИКУМ	52
	Общие указания к выполнению лабораторных работ	52
	Лабораторная работа №1. Классификация с помощью персептрона	53
	Лабораторная работа №2. Прогнозирование значений процесса с помощью линейной нейронной сети	55
	Лабораторная работа №3. Аппроксимация функции с помощью двух-слойной нейронной сети	58
	Лабораторная работа №4. Кластеризация с помощью нейронной сети Кохонена	61
	Лабораторная работа №5. Поиск экстремума функции одной переменной с помощью генетического алгоритма с бинарным представлением особей	64
	Лабораторная работа №6. Поиск экстремума функции двух переменных с помощью генетических алгоритмов с бинарным и вещественным представлением особей	68
	Лабораторная работа №7. Генетический алгоритм с перестановочным представлением особей. Задача коммивояжёра	74
	Лабораторная работа №8. Современная безградиентная оптимизация: СМА-ES, байесовская оптимизация и NSGA-II	77
	Лабораторная работа №9. Эволюция эвристик с помощью большой языковой модели	82
	Список использованных источников	87
	Приложение А. Справочник функций Python для лабораторных работ ...	90
	Приложение Б. Листинги программ	91

ВВЕДЕНИЕ

Многие практические задачи сводятся к поиску наилучшего решения в огромном пространстве вариантов: подобрать веса модели так, чтобы она верно предсказывала, найти экстремум многоэкстремальной функции, составить кратчайший маршрут обхода городов. Классические методы оптимизации требуют гладкости, выпуклости или перебора, который растёт экспоненциально. Вычислительный интеллект предлагает другой путь: заимствовать механизмы у живой природы. Нервная система учится на примерах, подстраивая силу связей между нейронами, а биологическая эволюция находит приспособленные организмы через наследование, изменчивость и отбор. Первый механизм лежит в основе *искусственных нейронных сетей*, второй — в основе *эволюционных вычислений*, самым известным направлением которых являются *генетические алгоритмы* (ГА).

Обе парадигмы решают по сути задачу оптимизации, но разными способами: нейронная сеть движется по градиенту функции ошибки, а ГА работает с популяцией решений и использует только значения целевой функции, поэтому применим к негладким, дискретным и многоэкстремальным задачам. Эти подходы дополняют друг друга: генетическим алгоритмом можно подбирать веса и архитектуру сети (нейроэволюция), а нейронную сеть использовать как быструю модель приспособленности внутри ГА [1].

Пособие состоит из двух частей. *Теоретическая часть* содержит необходимый материал по машинному обучению и нейронным сетям (разделы 1–4), по генетическим алгоритмам (разделы 5–7) и по современным безградиентным методам — СМА-ES, байесовской оптимизации, NSGA-II и эволюции программ с помощью больших языковых моделей (раздел 8). Нейросетевая часть опирается на пособие В. А. Головки и В. В. Краснопрошина [2] и классические модели, на которых исторически построены задания (MATLAB Neural Network Toolbox [3]), генетическая — на курс В. Ю. Скобцова, Н. В. Лапицкой и С. Н. Нестеренкова [4] и монографии Д. Холланда, Д. Голдберга и З. Михалевича [5, 6, 7]. Подразделы «Реализация на Python» показывают, как записать алгоритм на NumPy и с какими функциями библиотек scikit-learn, PyTorch, MiniSom, SciPy, DEAP, cma, Optuna и ruoo сравнить результат.

Лабораторный практикум включает девять работ (таблица 1). Каждая содержит цель, постановку задачи, варианты, требования к программе, порядок выполнения, пример выполнения для варианта 3 с графиками и таблицами реальных вычислительных экспериментов и контрольные вопросы.

Особое внимание уделено *экспериментальной культуре*. ГА и обучение сети со случайной инициализацией — стохастические процедуры, поэтому результат подтверждается серией независимых запусков (доля успехов, медиана, худший случай) и сравнением с эталоном: аналитическим экстремумом, пределом точности кодирования или известным оптимальным туром.

Таблица 1 – Лабораторные работы, разделы теории и инструменты Python

ЛР	Тема	Теория	Python
1	Классификация перцептроном	1, 2	NumPy; sklearn Perceptron
2	Прогнозирование линейной сетью	2	np.linalg.lstsq, sliding_window_view
3	Аппроксимация двухслойной сетью	3	NumPy; MLPRegressor, PyTorch, SciPy
4	Кластеризация картой Кохонена	4	NumPy; MiniSom, KMeans
5	Простой ГА: функция одной переменной	5, 6	NumPy; minimize_scalar
6	Бинарный и вещественный ГА: функция двух переменных	5, 6	NumPy; differential_evolution
7	Перестановочный ГА: задача коммивояжёра	6, 7	NumPy; DEAP, tsplib95
8	СМА-ES, подбор гиперпараметров, NSGA-II	8	NumPy; cma, Optuna, pymoo
9	Эволюция эвристик с языковой моделью	8	NumPy; Anthropic SDK

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1 ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ И ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЕЙ

Раздел вводит понятия, общие для ЛР №1–4.

1.1 Задачи машинного обучения

Машинное обучение — построение алгоритмов, которые извлекают закономерность из данных, а не получают её в виде явно запрограммированных правил. Пусть имеется *обучающая выборка* $\{(\mathbf{x}_k, \mathbf{t}_k)\}_{k=1}^N$, где $\mathbf{x}_k \in \mathbb{R}^n$ — вектор признаков (вход), $\mathbf{t}_k$ — известный ответ (цель, target). Требуется построить модель $y = F(\mathbf{x}; \theta)$ с параметрами θ , которая даёт правильные ответы не только на обучающих, но и на новых примерах. Способность модели работать на новых данных называется *обобщением* [2, 8].

В лабораторном практикуме встречаются четыре типа задач:

- *классификация* — ответ принадлежит конечному множеству классов (ЛР №1): распознать, к какой из групп относится точка плоскости;
- *регрессия (аппроксимация)* — ответ вещественный (ЛР №3): восстановить функцию $y = f(t)$ по её значениям в узлах сетки;
- *прогнозирование временного ряда* — частный случай регрессии, где входом служат прошлые значения сигнала, а целью — следующее значение (ЛР №2);
- *кластеризация* — разбиение объектов на группы похожих без известных ответов (ЛР №4).

Первые три задачи решаются *обучением с учителем* (supervised learning): для каждого входа известен правильный выход, и параметры подстраиваются так, чтобы уменьшить расхождение между ответом модели и целью. Кластеризация относится к *обучению без учителя* (unsupervised learning): модель извлекает структуру только из взаимного расположения входов [9]. Генетические алгоритмы (разделы 5–7) формально не являются методом обучения, но решают ту же вычислительную задачу — поиск параметров, доставляющих экстремум целевой функции.

1.2 Функция потерь, метрики и обобщение

Качество модели на выборке измеряется *функцией потерь*. Для регрессии стандартной является среднеквадратичная ошибка (mean squared error, MSE)

$$E(\theta) = \frac{1}{N} \sum_{k=1}^N (y_k - t_k)^2, \quad (1.1)$$

в Python — `np.mean((y - t)**2)` или функция `mean_squared_error` из `sklearn.metrics`. Для интерпретации результата используют коэффициент

детерминации $R^2 = 1 - E/\sigma_t^2$ (r2_score), где σ_t^2 — дисперсия целевых значений: $R^2 = 1$ означает точное совпадение, $R^2 = 0$ — модель не лучше константы. Для классификации применяют долю верных ответов (ассигасу) и число ошибок на эпоху.

Обучение — это минимизация $E(\theta)$, но цель — малая ошибка на *новых* данных. Поэтому выборку делят на три части: *обучающую* (по ней настраиваются параметры), *валидационную* (по ней выбирают архитектуру и момент останова) и *тестовую* (для окончательной независимой оценки), обычно в пропорции 70/15/15 %; в Python случайное разбиение выполняет `sklearn.model_selection.train_test_split`.

Соотношение ошибок на обучении и на новых данных определяется *ёмкостью* модели — числом и гибкостью её параметров:

- *недообучение* (underfitting) — модель слишком проста, ошибка велика и на обучающих, и на новых данных, форма зависимости не передаётся;
- *переобучение* (overfitting) — модель слишком гибка и подстраивается под случайный шум обучающей выборки: ошибка на обучении мала, на новых данных велика.

Ожидаемая ошибка на новых данных раскладывается на квадрат смещения (систематическая ошибка простой модели), дисперсию (чувствительность сложной модели к конкретной выборке) и неустранимый шум данных. С ростом ёмкости смещение убывает, а дисперсия растёт, поэтому зависимость ошибки на новых данных от сложности модели имеет U-образную форму (подраздел 3.5). Способы борьбы с переобучением — уменьшение числа параметров, регуляризация (штраф за большие веса), увеличение выборки и *ранний останов*: обучение прекращают, когда ошибка на валидационной выборке перестаёт уменьшаться.

1.3 Модель искусственного нейрона

Искусственный нейрон (рисунок 1.1) — упрощённая модель нервной клетки, предложенная У. Мак-Каллоком и У. Питтсом в 1943 г. Нейрон получает входы $x_1, \dots, x_n$, умножает их на *синаптические веса* $w_1, \dots, w_n$, суммирует вместе со *смещением* (bias) b и пропускает результат через *функцию активации* f :

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b, \quad y = f(z). \quad (1.2)$$

Величина z называется *сетевым входом* (net input). Смещение удобно считать весом дополнительного входа, постоянно равного единице.

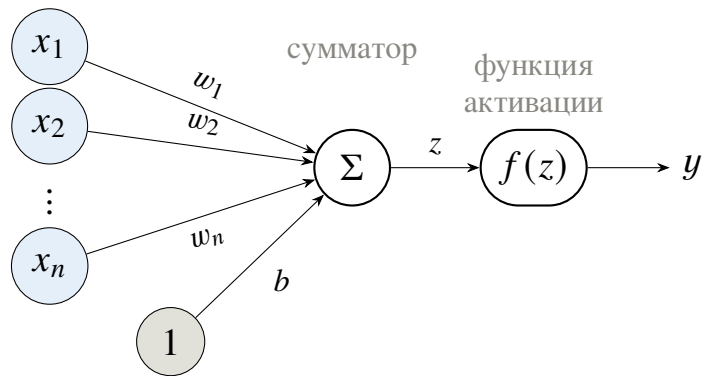


Рисунок 1.1 – Модель искусственного нейрона

Вид функции активации определяет свойства нейрона (рисунок 1.2):

- пороговая (hardlim): $f(z) = 1$ при $z \geq 0$ и $f(z) = 0$ при $z < 0$, в NumPy `(z >= 0).astype(int)` — перцептрон Розенблатта (раздел 2);
- линейная (purelin): $f(z) = z$ — линейная сеть ADALINE и выходной слой сети для регрессии;
- логистическая сигмоида (logsig): $f(z) = 1/(1 + e^{-z})$, значения в $(0; 1)$, в SciPy — `scipy.special.expit`;
- гиперболический тангенс (tansig): $f(z) = \tanh z = 2/(1 + e^{-2z}) - 1$, значения в $(-1; 1)$, `np.tanh` — скрытый слой многослойного перцептрона (раздел 3).

Сигмоидальные функции гладкие, монотонные и ограниченные, а их производные выражаются через значение самой функции: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ для `logsig` и $\tanh'(z) = 1 - \tanh^2 z$. Это делает вычисление градиентов при обучении дешёвым. В глубоких сетях сейчас чаще применяют кусочно-линейную функцию ReLU $f(z) = \max(0, z)$, но для небольших сетей практикума классические сигмоиды остаются стандартом.

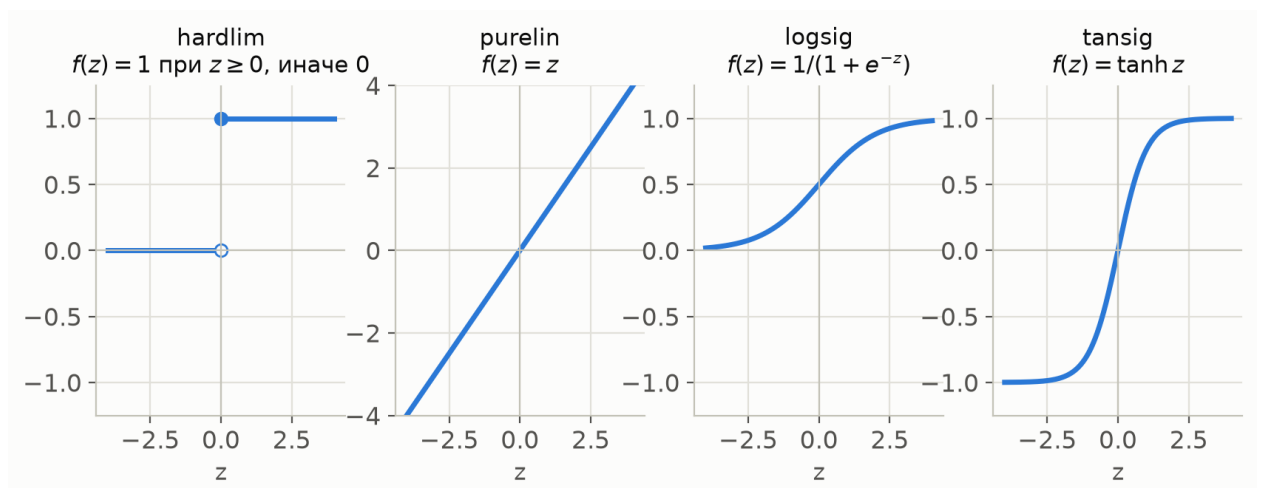


Рисунок 1.2 – Функции активации нейрона

1.4 Архитектуры нейронных сетей

Нейроны объединяются в *слои*: все нейроны слоя получают один и тот же входной вектор и работают параллельно. Слой из S нейронов с n входами

описывается матрицей весов W размера $S \times n$ и вектором смещений $\mathbf{b}$ длины S : $\mathbf{y} = f(W\mathbf{x} + \mathbf{b})$. По способу соединения слоёв различают:

- *сети прямого распространения* (feedforward): сигнал проходит от входа к выходу без обратных связей; к ним относятся персептрон, линейная сеть и многослойный персептрон (ЛР №1–3);
- *рекуррентные сети*: выходы подаются обратно на вход с задержкой, что позволяет обрабатывать последовательности;
- *сети с конкурентным слоем*: нейроны слоя соревнуются за право ответить на вход, как в картах Кохонена (ЛР №4).

В таблице 1.1 сведены модели, изучаемые в практикуме.

Таблица 1.1 – Нейросетевые модели лабораторного практикума

Модель	Обучение	Задача	Библиотечный аналог для сравнения
Персептрон	правило персептрона, с учителем	классификация линейно разделимых данных	<code>sklearn.linear_model.Perceptron</code>
Линейная сеть	МНК, правило Уидроу — Хоффа	прогнозирование, линейная регрессия	<code>np.linalg.lstsq</code> , <code>LinearRegression</code> , <code>SGDRegressor</code>
Многослойный персептрон	обратное распространение ошибки	аппроксимация, классификация	<code>MLPRegressor</code> , <code>torch.nn</code>
Карта Кохонена	конкурентное, без учителя	кластеризация, визуализация	<code>MiniSom</code> , <code>KMeans</code>

Обучение сети выполняется *эпохами*: за одну эпоху сети предъявляются все примеры обучающей выборки. При *последовательном* (стохастическом, онлайн) обучении веса меняются после каждого примера, при *пакетном* — один раз за эпоху по суммарному градиенту; промежуточный вариант — обучение *мини-пакетами*, стандартное для PyTorch (`DataLoader(batch_size=...)`).

1.5 Вычислительные инструменты Python

Все алгоритмы пособия реализуются на Python с библиотекой NumPy, а готовые библиотеки используются для проверки и сравнения собственной реализации [10, 11, 12]. Роли основных библиотек приведены в таблице 1.2.

Три соглашения делают код быстрым и воспроизводимым. *Векторизация*: выборка или популяция хранится одним массивом (строка — объект, столбец — признак или ген), и операции выполняются над массивом целиком без циклов Python. *Транслирование* (broadcasting): массивы разной формы согласуются автоматически, например `X[:, None]` – `X[None]` даёт все попарные разности. *Воспроизводимость*: все случайные числа берутся из

Таблица 1.2 – Библиотеки Python, используемые в практикуме

Библиотека	Назначение
NumPy	массивы, векторные операции, линейная алгебра (<code>np.linalg</code>), генератор случайных чисел
SciPy	эталонные решения: <code>optimize.minimize_scalar</code> , <code>differential_evolution</code> , <code>least_squares</code>
scikit-learn	готовые модели и метрики: <code>Perceptron</code> , <code>MLPRegressor</code> , <code>KMeans</code> , <code>r2_score</code>
PyTorch	нейронные сети с автоматическим дифференцированием (<code>torch.nn</code> , <code>torch.optim</code>)
MiniSom	самоорганизующиеся карты Кохонена
DEAP, PyGAD	библиотеки эволюционных вычислений (только для сравнения)
Matplotlib, Plotly, ipywidgets	графики, анимация, ползунки и кнопки в Jupyter

одного генератора `rng = np.random.default_rng(seed)`, и значение `seed` сохраняется вместе с результатом.

2 ПЕРСЕПТРОН И ЛИНЕЙНЫЕ НЕЙРОННЫЕ СЕТИ

Раздел используется в ЛР №1 и №2.

2.1 Персептрон Розенблатта

Персептрон, предложенный Ф. Розенблаттом в 1958 г. [13], — нейрон (1.2) с пороговой функцией активации. Он относит вход $\mathbf{x}$ к классу 1, если $\mathbf{w}^T \mathbf{x} + b \geq 0$, и к классу 0 в противном случае. Геометрически уравнение

$$\mathbf{w}^T \mathbf{x} + b = 0 \quad (2.1)$$

задаёт гиперплоскость в пространстве входов (на плоскости — прямую $w_1 x_1 + w_2 x_2 + b = 0$). Вектор весов $\mathbf{w}$ перпендикулярен этой границе и направлен в сторону класса 1, а смещение b сдвигает границу относительно начала координат: расстояние от начала координат до прямой равно $|b|/\|\mathbf{w}\|$.

Правило обучения персептрона. Веса инициализируются нулями или малыми случайными числами. Для очередного примера $(\mathbf{x}, t)$ вычисляется выход y и ошибка $e = t - y \in \{-1, 0, 1\}$, после чего веса корректируются:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta e \mathbf{x}, \quad b \leftarrow b + \eta e, \quad (2.2)$$

где $\eta > 0$ — скорость обучения; обычно $\eta = 1$ (при нулевых начальных весах η лишь масштабирует веса и не меняет границу). Если пример классифицирован верно ($e = 0$), веса не меняются; если точка класса 1 отнесена к классу 0 ($e = 1$), вектор $\mathbf{w}$ поворачивается к ней, в обратном случае — от неё. Эпохи повторяются, пока за эпоху не будет допущено ни одной ошибки.

Теорема сходимости (Розенблатт, Новиков, 1962): если классы *линейно разделимы*, т.е. существует гиперплоскость, строго разделяющая их с зазором $\gamma > 0$, а все входы лежат в шаре радиуса R , то правило (2.2) находит разделяющую гиперплоскость не более чем за $(R/\gamma)^2$ коррекций независимо от порядка предъявления примеров. Найденная граница не единственна и зависит от начальных весов и порядка примеров; персептрон ищет *какую-нибудь* разделяющую прямую, а не «лучшую».

2.2 Линейная разделимость и проблема исключающего ИЛИ

Логические функции И и ИЛИ от двух переменных линейно разделимы, а исключающее ИЛИ (XOR) нет: точки $(0, 1)$ и $(1, 0)$ класса 1 лежат на одной диагонали квадрата, точки $(0, 0)$ и $(1, 1)$ класса 0 — на другой, и никакая прямая их не разделяет (рисунок 2.1). Это легко доказать: из условий $b < 0$, $w_1 + b \geq 0$, $w_2 + b \geq 0$ следует $w_1 + w_2 + b \geq -b > 0$, что противоречит требованию $w_1 + w_2 + b < 0$ для точки $(1, 1)$.

Для неразделимых данных теорема сходимости не действует: правило персептрона бесконечно исправляет одну ошибку ценой другой, и число

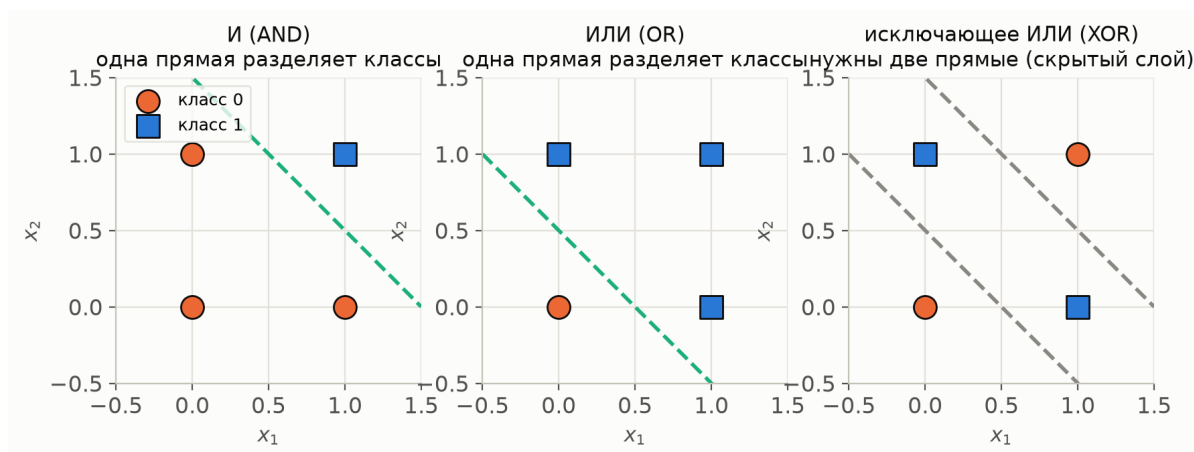


Рисунок 2.1 – Линейно разделяемые функции И, ИЛИ и неразделимая функция XOR

ошибок за эпоху не убывает до нуля. Анализ М. Минского и С. Пейперта (1969) [14] показал ограниченность однослойного персептрона и надолго снизил интерес к нейронным сетям. Решение даёт *скрытый слой*: XOR выражается через разделяемые функции, $x_1 \oplus x_2 = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$, т.е. два нейрона скрытого слоя строят две прямые (рисунок 2.1, справа), а выходной нейрон комбинирует их ответы. Обучать такие сети позволил алгоритм обратного распространения ошибки (раздел 3).

2.3 Персептрон с несколькими нейронами

Слой из m персептронов с общими входами строит m гиперплоскостей, и каждый нейрон отвечает за один двоичный разряд кода класса. Два нейрона делят плоскость двумя прямыми на четыре области и различают до $2^2 = 4$ классов, закодированных парами $(y_1, y_2) \in \{00, 01, 10, 11\}$; m нейронов различают до 2^m классов. Правило (2.2) применяется к каждому нейрону независимо со своей целью — соответствующим разрядом кода. Условие применимости: каждый разряд кода должен быть линейно разделяемой функцией входа. Поэтому кодирование классов выбирают так, чтобы соседние на плоскости группы различались одним разрядом.

2.4 Линейная нейронная сеть ADALINE

Линейная сеть (ADALINE, Б. Уидроу и М. Хофф, 1960 [15]) отличается от персептрона линейной функцией активации: $y = \mathbf{w}^T \mathbf{x} + b$. Выход непрерывен, поэтому ошибка $e = t - y$ также непрерывна, и можно минимизировать среднеквадратичную ошибку (1.1), которая для линейной модели является квадратичной функцией весов с единственным минимумом.

Итерационное обучение выполняется правилом Уидроу — Хоффа (LMS, least mean squares), которое представляет собой стохастический градиентный спуск по мгновенной квадратичной ошибке:

$$\mathbf{w} \leftarrow \mathbf{w} + 2\alpha e \mathbf{x}, \quad b \leftarrow b + 2\alpha e. \quad (2.3)$$

Процесс устойчив при $0 < \alpha < 1/\lambda_{\max}$, где $\lambda_{\max}$ — наибольшее собственное число корреляционной матрицы входов $\mathbf{R} = \mathbf{M}[\mathbf{x}\mathbf{x}^T]$ (в NumPy — `np.linalg.eigvalsh(P @ P.T / N).max()`).

Аналитическое решение. Объединим входные векторы в столбцы матрицы P размера $n \times N$ (добавив строку единиц для смещения), а цели — в строку T длины N . Минимум суммы квадратов ошибок $\|WP - T\|^2$ даётся нормальными уравнениями метода наименьших квадратов (МНК):

$$W = TP^T(PP^T)^{-1} = TP^+, \quad (2.4)$$

где P^+ — псевдообратная матрица Мура — Пенроуза. Если PP^T вырождена (например, строки P линейно зависимы), P^+ вычисляется через сингулярное разложение и даёт решение с наименьшей нормой весов. В NumPy решение даёт `W = T @ np.linalg.pinv(P)` или численно более устойчивая функция `np.linalg.lstsq`, решающая задачу МНК без явного обращения матрицы PP^T .

2.5 Прогнозирование временного ряда линейной сетью

Пусть сигнал $x(t)$ дискретизирован с шагом Δt : $x_i = x(t_i)$, $t_i = t_0 + (i - 1)\Delta t$, $i = 1, \dots, N$. Требуется предсказать следующее значение x_i по k предыдущим $x_{i-1}, \dots, x_{i-k}$. Для этого на вход линейной сети подаётся выход *линии задержки* (tapped delay line), и сеть реализует авторегрессионную модель порядка k :

$$y_i = w_1 x_{i-1} + w_2 x_{i-2} + \dots + w_k x_{i-k} + b. \quad (2.5)$$

Матрица входов строится методом скользящего окна: i -й столбец содержит k значений сигнала, предшествующих x_i , а целевой вектор — сам сигнал:

$$P = \begin{pmatrix} 0 & x_1 & x_2 & \cdots & x_{N-1} \\ 0 & 0 & x_1 & \cdots & x_{N-2} \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & \cdots & x_{N-k} \end{pmatrix}, \quad T = (x_1, x_2, \dots, x_N). \quad (2.6)$$

Нули в левой части соответствуют нулевым начальным условиям линии задержки (до начала наблюдения сигнал считается равным нулю). Веса находятся по формуле (2.4).

Для гармонического сигнала $x(t) = \sin(\omega t)$ существует точное рекуррентное соотношение, вытекающее из формулы суммы синусов:

$$x_i = 2 \cos(\omega \Delta t) x_{i-1} - x_{i-2}. \quad (2.7)$$

Следовательно, линейная сеть с $k = 2$ уже способна прогнозировать синусоиду без ошибки, а при $k > 2$ МНК находит те же два ненулевых

веса. Ошибка возникает только в начале ряда, где линия задержки ещё заполнена нулями: при $t_0 = 0$ отсчёт $x_2 = \sin(\omega\Delta t)$ не может быть предсказан по предыстории $(x_1, 0, \dots, 0) = (0, \dots, 0)$, и среднеквадратичная ошибка примерно равна $\sin^2(\omega\Delta t)/N$. Она не зависит от k и уменьшается с шагом дискретизации. Для реальных сигналов с шумом ситуация иная: увеличение k позволяет усреднять шум по большому числу отсчётов, и ошибка убывает до насыщения (см. пример выполнения ЛР №2).

Выбор шага дискретизации ограничен *теоремой Котельникова* (Найквиста — Шеннона): сигнал с наибольшей частотой $f_{\max} = \omega/(2\pi)$ однозначно восстанавливается по отсчётам, если $\Delta t < 1/(2f_{\max}) = \pi/\omega$, т. е. на период приходится больше двух отсчётов. На практике для прогнозирования берут 10–20 отсчётов на период: при слишком крупном шаге соседние отсчёты слабо связаны, модель становится плохо обусловленной и чувствительной к шуму.

2.6 Реализация на Python

Обучение персептрона — два вложенных цикла: по эпохам и по примерам (правило (2.2), $\eta = 1$). Для многоклассового случая w становится матрицей, а e — вектором ошибок нейронов.

```
w, b = np.zeros(2), 0.0
for epoch in range(100):
    errors = 0
    for x, t in zip(X, T):
        e = t - int(w @ x + b >= 0)
        w, b = w + e * x, b + e
        errors += e != 0
    if errors == 0:
        break
```

X: (N, 2), T: (N,) из {0, 1}
ошибка: -1, 0 или 1
эпоха без ошибок: классы разделены

Для сравнения служит `sklearn.linear_model.Perceptron(eta0=1.0)`: он тоже находит разделяющую прямую, но из-за перемешивания примеров — другую, что иллюстрирует неединственность решения.

Матрица входов линии задержки (2.6) строится без циклов функцией `sliding_window_view`, веса находятся одним вызовом МНК:

```
from numpy.lib.stride_tricks import sliding_window_view
xp = np.concatenate([np.zeros(k), x])
P = sliding_window_view(xp[:-1], k)[: , :-1].T
Pb = np.vstack([P, np.ones(len(x))])
W = np.linalg.lstsq(Pb.T, x, rcond=None)[0]
y = W @ Pb
```

нулевые начальные условия
k x N: x[i-1], ..., x[i-k]
строка единиц для смещения
то же, что x @ np.linalg.pinv(Pb)
прогноз сети

Готовые аналоги — `sklearn.linear_model.LinearRegression` (МНК) и `SGDRegressor` (итерационное обучение, близкое к правилу LMS).

3 МНОГОСЛОЙНЫЙ ПЕРСЕПТРОН И ОБРАТНОЕ РАСПРОСТРАНЕНИЕ ОШИБКИ

Раздел используется в ЛР №3.

3.1 Архитектура двухслойной сети

Многослойный перцептрон (multilayer perceptron, MLP) — сеть прямого распространения, в которой между входом и выходным слоем расположены один или несколько *скрытых* слоёв нелинейных нейронов. Для аппроксимации функции одной переменной $y = f(t)$ достаточно сети 1– S –1: один вход, скрытый слой из S нейронов с функцией активации $\tanh$ и один линейный выходной нейрон (рисунок 3.1):

$$\mathbf{a}_1 = \tanh(\mathbf{w}_1 t + \mathbf{b}_1), \quad y = \mathbf{w}_2^\top \mathbf{a}_1 + b_2 = \sum_{i=1}^S w_{2,i} \tanh(w_{1,i} t + b_{1,i}) + b_2. \quad (3.1)$$

Сеть имеет $3S + 1$ настраиваемых параметров. Каждый скрытый нейрон — это гладкая «ступенька» $\tanh(w_{1,i} t + b_{1,i})$ с центром в точке $t = -b_{1,i}/w_{1,i}$ и крутизной $w_{1,i}$, а выходной нейрон складывает ступеньки с весами $w_{2,i}$ (рисунок 3.2). Из суммы нескольких ступенек можно собрать кривую любой формы; без нелинейности композиция линейных слоёв остаётся линейной функцией, и сеть при любом S даёт прямую.

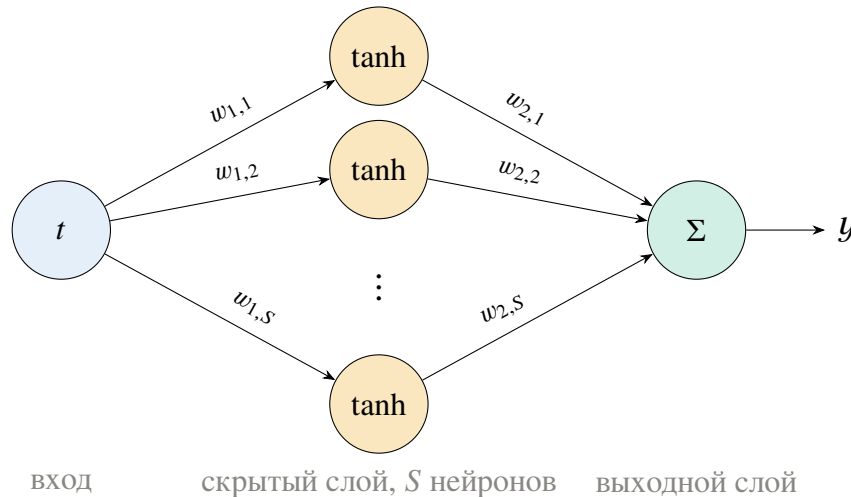


Рисунок 3.1 – Двухслойная сеть 1– S –1 (смещения не показаны)

Теорема об универсальной аппроксимации (Г. Цыбенко, 1989 [16]; К. Хорник, 1991) утверждает: для любой непрерывной функции f на компакте $K \subset \mathbb{R}^n$ и любого $\varepsilon > 0$ существует сеть с *одним* скрытым слоем из конечного числа сигмоидальных нейронов и линейным выходом, для которой $\max_{\mathbf{x} \in K} |f(\mathbf{x}) - y(\mathbf{x})| < \varepsilon$. Близкий по духу результат — теорема Колмогорова — Арнольда о представлении непрерывной функции многих переменных суперпозицией функций одной переменной и сложения. Следствие для практики: двух слоёв *достаточно*, однако теорема не

указывает нужное число нейронов, не гарантирует, что градиентный спуск найдёт подходящие веса, и ничего не говорит о поведении сети между обучающими точками.

Выход сети как сумма сдвинутых и масштабированных гиперболических тангенсов

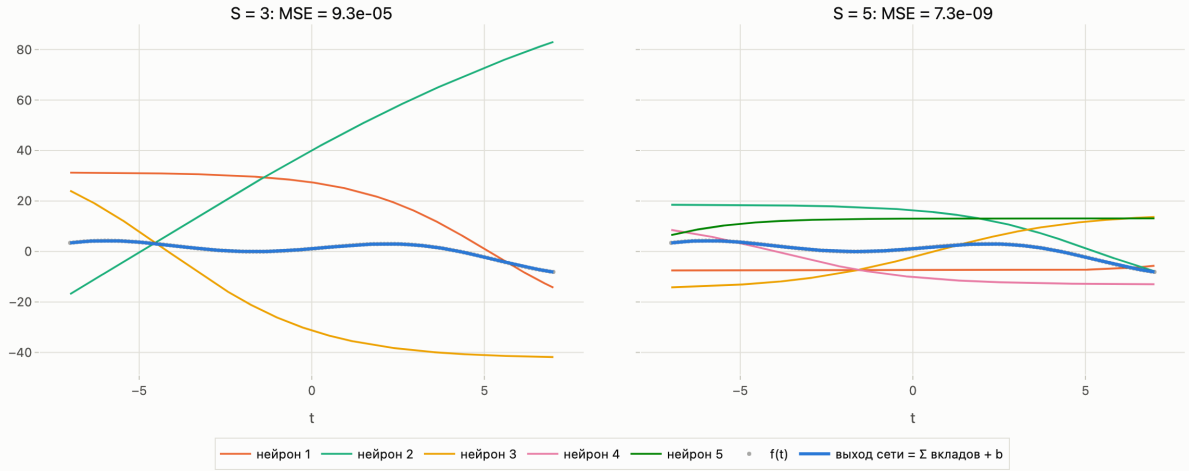


Рисунок 3.2 – Выход сети как сумма вкладов скрытых нейронов: функция $f(t) = (t + 1,3) \sin(0,5t + 1)$ приближена тремя и пятью нейронами (ЛР №3)

3.2 Алгоритм обратного распространения ошибки

Обучение минимизирует MSE (1.1) по всем весам и смещениям θ . *Обратное распространение ошибки* (backpropagation, Д. Румельхарт, Дж. Хинтон, Р. Уильямс, 1986) — способ вычислить градиент $\nabla_{\theta} E$ за один проход по сети от выхода к входу по правилу дифференцирования сложной функции. Для сети (3.1) и выборки из N точек:

- 1) *прямой проход*: для каждого t_k вычисляются $\mathbf{a}_{1,k}$ и выход y_k ;
- 2) *ошибка выхода*: $\delta_{2,k} = \frac{2}{N}(y_k - d_k)$, где $d_k = f(t_k)$ — целевое значение;
- 3) *обратный проход*: ошибка передаётся на скрытый слой через веса $\mathbf{w}_2$ и умножается на производную активации, $\delta_{1,k} = \delta_{2,k} \mathbf{w}_2 \odot (1 - \mathbf{a}_{1,k}^2)$ ($\odot$ — поэлементное произведение);
- 4) *градиенты*: производная по весу равна сумме произведений сигнала ошибки на вход этого веса,

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{w}_2} &= \sum_k \delta_{2,k} \mathbf{a}_{1,k}, & \frac{\partial E}{\partial b_2} &= \sum_k \delta_{2,k}, \\ \frac{\partial E}{\partial \mathbf{w}_1} &= \sum_k \delta_{1,k} t_k, & \frac{\partial E}{\partial \mathbf{b}_1} &= \sum_k \delta_{1,k}. \end{aligned} \quad (3.2)$$

Корректность реализации проверяют сравнением с численной производной $(E(\theta + h) - E(\theta - h))/2h$ при $h \sim 10^{-6}$: относительное расхождение должно быть порядка 10^{-7} и меньше.

3.3 Градиентные методы обучения

Градиентный спуск на каждой эпохе сдвигает параметры против градиента:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla E(\theta^{(k)}), \quad (3.3)$$

где η — коэффициент обучения (learning rate, lr). Вблизи минимума функцию ошибки можно приблизить квадратичной формой с матрицей Гессе H . Тогда спуск устойчив только при

$$\eta < \frac{2}{\lambda_{\max}}, \quad (3.4)$$

где $\lambda_{\max}$ — наибольшее собственное число H , а скорость сходимости определяется числом обусловленности $\kappa = \lambda_{\max}/\lambda_{\min}$: чем больше κ , тем сильнее поверхность ошибки вытянута в узкий «овраг». Шаг приходится выбирать по самому крутому направлению, и вдоль пологого дна спуск идёт очень медленно (рисунок 3.3). Для сетей практикума κ достигает 10^{11} , поэтому градиентный спуск с постоянным шагом требует тысяч эпох.

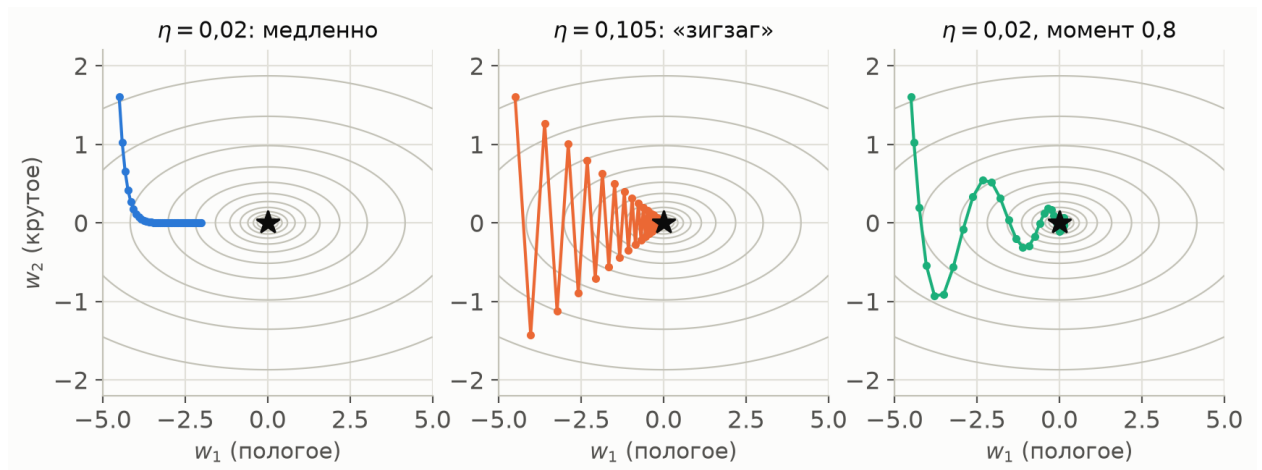


Рисунок 3.3 – Градиентный спуск на вытянутой квадратичной поверхности ($\lambda_{\max} = 18$, $2/\lambda_{\max} \approx 0,11$): 40 шагов из одной начальной точки

Ускоренные алгоритмы (в скобках — реализации в Python):

- спуск с *моментом* (`torch.optim.SGD(momentum=0.9)`): $\Delta\theta^{(k)} = \mu \Delta\theta^{(k-1)} - \eta \nabla E$, $\mu \approx 0,9$. Инерция гасит колебания поперёк оврага и накапливает скорость вдоль дна;
- *адаптивный шаг*: если ошибка уменьшилась, $\eta \leftarrow 1,05 \eta$; если выросла более чем в 1,04 раза, шаг отменяется и $\eta \leftarrow 0,7 \eta$ (правило `traingdx` MATLAB); близкий эффект дают планировщик `torch.optim.lr_scheduler.ReduceLROnPlateau` и `MLPRegressor(learning_rate='adaptive')`;
- *Adam* (`torch.optim.Adam`, `MLPRegressor(solver='adam')`) — свой адаптивный шаг для каждого параметра по скользящим средним градиента и его квадрата; стандарт для больших сетей;

- метод *Левенберга — Марквардта* [17], использующий якобиан J вектора ошибок сети по параметрам (`scipy.optimize.least_squares(method='lm')`):

$$\Delta\theta = -(J^T J + \mu I)^{-1} J^T \mathbf{e}. \quad (3.5)$$

Матрица $J^T J$ приближает гессиан, поэтому метод учитывает кривизну. При малом μ шаг близок к шагу Гаусса — Ньютона, при большом — к короткому градиентному шагу; после удачного шага μ уменьшается в 10 раз, после неудачного — увеличивается. Для сетей из десятков параметров метод сходится за единицы–десятки эпох;

- *L-BFGS* (`MLPRegressor(solver='lbfgs')`, `torch.optim.LBFGS`) — квазиньютоновский метод, для малых сетей и выборок обычно самый быстрый.

3.4 Предобработка, инициализация и критерии останова

Входы и цели нормируют в диапазон $[-1; 1]$ (`MinMaxScaler` из `sklearn.preprocessing` с параметром `feature_range=(-1, 1)`):

$$\tilde{t} = 2 \frac{t - t_{\min}}{t_{\max} - t_{\min}} - 1, \quad (3.6)$$

чтобы аргументы $\tanh$ не попадали в область насыщения, где производная близка к нулю и обучение останавливается. Выход сети пересчитывается обратно в исходный масштаб. Веса скрытого слоя инициализируются методом Нгуена — Уидроу [18]: активные области сигмоид равномерно распределяются по отрезку входа, для одного входа $|w_{1,i}| = 0,7 S$, а центры ступенек $-b_{1,i}/w_{1,i}$ расставлены с равным шагом. В PyTorch слой `nn.Linear` по умолчанию берёт веса из $U[-1/\sqrt{n}; 1/\sqrt{n}]$ (n — число входов), для $\tanh$ применяют инициализацию Глоро (`nn.init.xavier_uniform_`).

Обучение прекращается при выполнении любого из условий:

- достигнуто максимальное число эпох `epochs` (в `MLPRegressor` — `max_iter`);
- ошибка не превышает целевую: $E \leq \text{goal}$;
- норма градиента меньше порога (сеть «застряла» на плато);
- ошибка на валидационной выборке не уменьшалась заданное число эпох — ранний останов (`MLPRegressor(early_stopping=True, validation_fraction=0.15, n_iter_no_change=6)`).

Параметр `show` задаёт период вывода протокола обучения и перерисовки графиков и на результат не влияет.

3.5 Обобщение: недообучение и переобучение

Число нейронов скрытого слоя S определяет ёмкость сети. При малом S сеть не может передать все изгибы функции (недообучение): кривая слишком гладкая, экстремумы пропущены, ошибка велика и одинакова на обучающих и контрольных точках. При избыточном S на зашумлённых данных сеть проходит почти через каждую обучающую точку, повторяя шум, и между точками возникают осцилляции и выбросы (рисунок 3.4). Признак переобучения — ошибка на обучении ниже уровня шума при большой ошибке на контрольных данных. В ЛР №3 при шуме $\sigma = 0,7$ ошибка на обучающих точках с ростом S монотонно убывает, а ошибка относительно истинной функции минимальна при $S = 3$ (0,06) и возрастает до 4,06 при $S = 30$; ранний останов снижает её до 0,37.

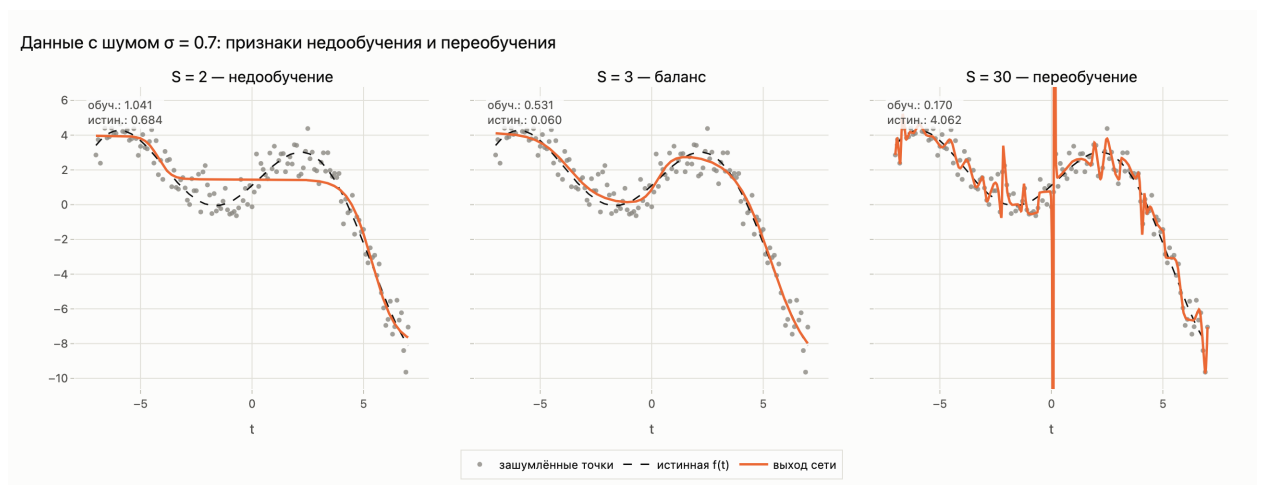


Рисунок 3.4 – Недообучение ($S = 2$), удачная сложность ($S = 3$) и переобучение ($S = 30$) на данных с шумом $\sigma = 0,7$ (ЛР №3)

Переобучение следует отличать от *недоученности* больших сетей при малом бюджете эпох: если ошибка на контрольных точках равна ошибке на обучающих, рябь кривой исчезает при продолжении обучения.

3.6 Реализация на Python

Одна эпоха пакетного градиентного спуска для сети (3.1) на нормированных данных t, d (массивы длины N) записывается векторно:

```
a1 = np.tanh(t[:, None] * w1 + b1)      # N x S: выходы скрытого слоя
y = a1 @ w2 + b2                        # выход сети
d2 = 2 * (y - d) / len(t)               # сигнал ошибки выходного слоя
d1 = np.outer(d2, w2) * (1 - a1 ** 2)  # ошибка, переданная на скрытый слой
w2 -= lr * a1.T @ d2; b2 -= lr * d2.sum()
w1 -= lr * t @ d1; b1 -= lr * d1.sum(axis=0)
```

Градиенты проверяют численно (`scipy.optimize.check_grad`). Готовая сеть для сравнения — `MLPRegressor(hidden_layer_sizes=(10,), activation='tanh', solver='lbfgs')` или `nn.Sequential(nn.Linear(1, S), nn.Tanh(), nn.Linear(S, 1))` в PyTorch, где градиенты вычисляются автоматически (`loss.backward()`) [12].

4 САМООРГАНИЗУЮЩИЕСЯ КАРТЫ КОХОНЕНА

Раздел используется в ЛР №4.

4.1 Конкурентное обучение

В задаче кластеризации нет целевых выходов, поэтому сеть должна извлечь структуру из самих входных векторов. Простейший механизм — *конкурентное обучение*. Слой содержит M нейронов с векторами весов $W_1, \dots, W_M$ той же размерности, что и вход. На вход $\mathbf{p}$ откликается один нейрон — *победитель* (best matching unit, BMU), вектор весов которого ближе всего к входу:

$$c = \arg \min_i \|W_i - \mathbf{p}\|. \quad (4.1)$$

По правилу «победитель получает всё» (winner-take-all, WTA) корректируются только его веса: $W_c \leftarrow W_c + \eta(\mathbf{p} - W_c)$. Нейрон-победитель сдвигается к входу на долю η расстояния, и нейрон, выигрывающий на некоторой группе точек, в итоге оказывается в её центре. По сути это стохастический вариант алгоритма k -средних (k -means), минимизирующий *квантизационную ошибку* — среднее расстояние от точек до ближайшего нейрона. Недостаток WTA — «мёртвые» нейроны: нейрон, изначально оказавшийся далеко от данных, никогда не побеждает и не обучается.

4.2 Архитектура и алгоритм самоорганизующейся карты

Самоорганизующаяся карта (self-organizing map, SOM), предложенная Т. Кохоненом [19], дополняет конкурентное обучение *топологией*. Нейроны расположены в узлах решётки, и у каждого нейрона i , кроме вектора весов W_i в пространстве данных, есть фиксированное положение $\mathbf{r}_i$ на решётке. Биологический прототип — топографические карты коры головного мозга, где соседние участки реагируют на близкие стимулы (соматотопическая, ретинотопическая, тонотопическая организация).

При обучении сдвигаются веса не только победителя, но и его соседей по решётке:

$$W_i(t+1) = W_i(t) + \eta(t) h_{ci}(t) (\mathbf{p} - W_i(t)), \quad (4.2)$$

где $h_{ci}(t)$ — *функция окрестности*, убывающая с расстоянием между нейронами c и i на решётке. Наиболее распространена гауссова окрестность

$$h_{ci}(t) = \exp\left(-\frac{\|\mathbf{r}_c - \mathbf{r}_i\|^2}{2\sigma^2(t)}\right). \quad (4.3)$$

Скорость обучения $\eta(t)$ и радиус окрестности $\sigma(t)$ уменьшаются в ходе обучения, например экспоненциально или линейно: σ от половины диаметра решётки до долей шага, η от 0,5 до 0,01. В начале, пока окрестность широкая, вся карта движется согласованно и *упорядочивается*: соседние на

решётке нейроны занимают соседние области пространства данных. В конце окрестность сужается до одного нейрона, и веса точно *подстраиваются* под плотность данных. Благодаря соседству в обучении участвуют и нейроны, далёкие от данных, поэтому мёртвых нейронов становится значительно меньше.

Топология решётки: прямоугольная (gridtop), у внутреннего нейрона 4 ближайших соседа; гексагональная (hextop), нечётные ряды сдвинуты на половину шага, у внутреннего нейрона 6 соседей на одинаковом расстоянии. *Расстояние на решётке* измеряют евклидовой метрикой между координатами узлов или числом связей по кратчайшему пути (linkdist, рисунок 4.1). Гексагональная решётка изотропнее прямоугольной: окрестность ближе к кругу, и карта изгибается равномернее.

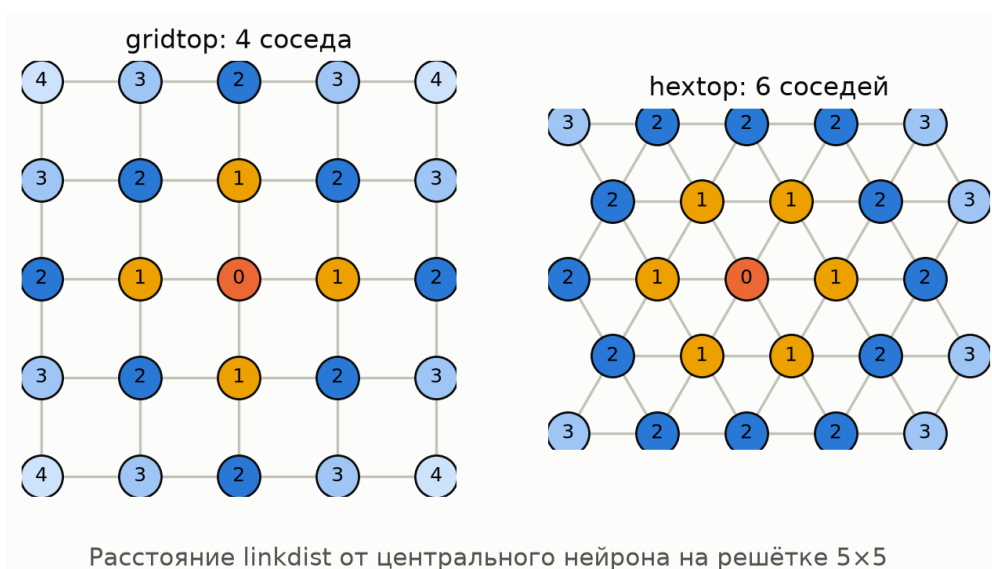


Рисунок 4.1 – Прямоугольная и гексагональная решётки 5×5: числа — расстояние linkdist от центрального нейрона

Алгоритм обучения SOM:

- 1) задать решётку, инициализировать веса случайно в области данных (или по главным компонентам);
- 2) для каждой эпохи и каждого входа $\mathbf{p}$ в случайном порядке: найти победителя (4.1), вычислить h_{ci} для всех нейронов, обновить веса по формуле (4.2);
- 3) после эпохи уменьшить η и σ ; повторять до заданного числа эпох;
- 4) отнести каждую точку к кластеру своего нейрона-победителя.

На практике применяют и более простое правило с двумя фазами (так работает, например, `learnsom` в MATLAB): в *фазе упорядочивания* скорость падает с 0,9 до 0,02, а радиус окрестности — от максимального расстояния на решётке до фиксированного значения (по умолчанию 1), в *фазе подстройки* оба параметра постоянны; соседи в пределах радиуса получают шаг вдвое меньше победителя. Если окрестность не сужается до одного нейрона, на маленькой карте все нейроны остаются соседями до конца

обучения, продолжают тянуть друг друга — центроиды кластеров стягиваются к центру данных (рисунок 4.2). Поэтому для точного нахождения центров кластеров радиус к концу обучения уменьшают до нуля или используют гауссову окрестность с убывающим σ .

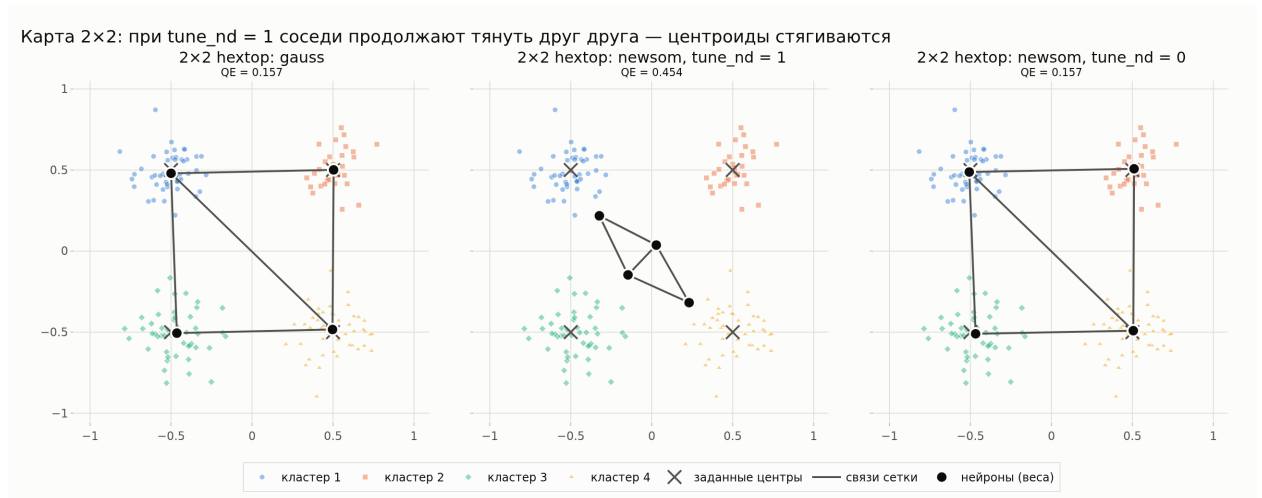


Рисунок 4.2 – Карта 2×2 при разных правилах обучения: несужающаяся окрестность стягивает центроиды (ЛР №4)

4.3 Оценка качества карты

Поскольку истинные метки кластеров в реальных задачах неизвестны, качество карты оценивают внутренними показателями:

- *квантизационная ошибка* $QE = \frac{1}{N} \sum_k \| \mathbf{p}_k - W_{c(\mathbf{p}_k)} \|$ — точность представления данных нейронами; убывает с ростом числа нейронов;
- *топографическая ошибка* TE — доля точек, для которых первый и второй ближайшие нейроны не являются соседями на решётке; показывает, насколько карта сохраняет топологию;
- *карта попаданий* (hits) — число точек, для которых нейрон является победителем; нейроны без попаданий лежат между кластерами или вне данных;
- *U-матрица* — для каждого нейрона среднее расстояние между его весами и весами соседей; большие значения образуют «хребты» на границах кластеров.

Если истинное разбиение известно, совпадение с ним измеряют индексом Рэнда (ARI, 1 — полное совпадение).

Когда число нейронов равно числу кластеров, SOM эквивалентна k -средним. *Избыточная* карта растягивается по данным: нейроны распределяются внутри облаков пропорционально их плотности, а несколько нейронов остаются между облаками, удерживаемые связями решётки; на U-матрице они образуют границы кластеров.

4.4 Реализация на Python

Шаг обучения карты (4.2) для одного входа p : W — веса $M \times 2$, R — координаты узлов решётки $M \times 2$ (для hexтор нечётные ряды сдвинуты на 0,5 по горизонтали и сжаты по вертикали в $\sqrt{3}/2$ раз):

```
c = np.argmin(((W - p) ** 2).sum(axis=1))          # нейрон-победитель
h = np.exp(-((R - R[c]) ** 2).sum(axis=1) / (2 * sigma ** 2)) # окрестность на решётке
W += eta * h[:, None] * (p - W)                  # сдвиг победителя и соседей
```

Для сравнения служит библиотека MiniSom [20]; она же вычисляет показатели качества:

```
from minisom import MiniSom
som = MiniSom(5, 5, 2, sigma=2.0, learning_rate=0.5, topology='hexagonal',
             neighborhood_function='gaussian', random_seed=3)
som.random_weights_init(X)
som.train(X, 200 * len(X), random_order=True)      # 200 эпох
qe, te = som.quantization_error(X), som.topographic_error(X)
U, hits = som.distance_map(), som.activation_response(X) # U-матрица, попадания
```

Карту с числом нейронов, равным числу кластеров, полезно сравнить с `sklearn.cluster.KMeans`, а совпадение разбиений измерить функцией `sklearn.metrics.adjusted_rand_score`.

5 ОСНОВЫ ЭВОЛЮЦИОННЫХ ВЫЧИСЛЕНИЙ И ГЕНЕТИЧЕСКИХ АЛГОРИТМОВ

Раздел используется в ЛР №5–7.

5.1 Эволюционные вычисления

Эволюционные вычисления — общее название алгоритмов поиска, оптимизации и обучения, основанных на формализованных принципах естественного отбора. Эволюционный поиск последовательно преобразует одно конечное множество промежуточных решений (*популяцию*) в другое: лучшие решения чаще участвуют в создании новых, новые решения наследуют черты родителей со случайными изменениями, а худшие постепенно вытесняются. Выделяют четыре основные парадигмы [21, 22]:

- *генетические алгоритмы* (ГА, Дж. Холланд, 1975 [5]) — решения кодируются строками (классически двоичными), основной оператор — скрещивание, мутация играет вспомогательную роль;
- *эволюционные стратегии* (И. Рехенберг, Х.-П. Швэфель) — вещественные векторы, основной оператор — гауссова мутация с самоадаптацией шага;
- *эволюционное программирование* (Л. Фогель) — эволюция конечных автоматов, затем вещественных векторов, только мутация;
- *генетическое программирование* (Дж. Коза) — эволюция программ, представленных деревьями выражений.

Парадигмы различаются представлением решений и набором операторов; современные алгоритмы свободно комбинируют их идеи. К эволюционным вычислениям примыкают дифференциальная эволюция и алгоритмы роевого интеллекта (муравьиные, роя частиц).

Генетический алгоритм — случайно направленный поисковый алгоритм, основанный на механизмах естественного отбора и генетического наследования. От классических методов оптимизации ГА отличаются тем, что:

- работают не с самими параметрами задачи, а с их кодами;
- ведут поиск из популяции точек, а не из одной точки;
- используют только значения целевой функции, не требуя производных, непрерывности и выпуклости;
- применяют вероятностные, а не детерминированные правила перехода.

Благодаря этому ГА применимы к многоэкстремальным, негладким и дискретным задачам, но не гарантируют нахождения глобального оптимума за конечное время и требуют многократных вычислений целевой функции.

5.2 Основные понятия

Терминология ГА заимствована из генетики (таблица 5.1).

Таблица 5.1 – Основные понятия генетических алгоритмов

Понятие	Значение в ГА
Особь, хромосома	закодированное допустимое решение задачи, например строка 0110100
Ген	элемент хромосомы: бит, число или номер города
Аллель	значение гена (для двоичного кода — 0 или 1)
Локус	позиция гена в хромосоме
Генотип / фенотип	код решения / само решение (значение x , маршрут)
Популяция	множество из N особей, обрабатываемое на одной итерации
Поклоение	популяция на очередной итерации t
Приспособленность (fitness)	значение функции, которое ГА стремится максимизировать
Родители, потомки	особи, участвующие в скрещивании, и особи, полученные в его результате

Функция приспособленности (фитнес-функция) $F(x)$ оценивает качество решения. Её выводят из целевой функции задачи $f(x)$. Если требуется найти максимум положительной функции, можно взять $F = f$. Для поиска минимума и для функций, принимающих отрицательные значения, применяют преобразования: $F(x) = C_{\max} - f(x)$, $F(x) = f(x) - f_{\min} + \varepsilon$ (сдвиг на минимум по текущей популяции, $\varepsilon > 0$ исключает нулевую вероятность отбора) или $F(x) = 1/(1 + f(x))$. Для методов селекции, использующих только порядок особей (ранжирование, турнир), достаточно сравнивать значения f со знаком, соответствующим направлению поиска.

Приспособленность популяции характеризуют лучшим и средним значениями F по особям; их графики по поколениям — основной инструмент наблюдения за ходом эволюции.

5.3 Общая структура генетического алгоритма

Для определения ГА нужно задать: способ кодирования решений, функцию приспособленности, способ создания начальной популяции, операторы селекции, скрещивания и мутации, стратегию формирования нового поколения, параметры (размер популяции N , вероятности P_c и P_m) и критерий останова. Общая схема показана на рисунке 5.1.

Критериями останова служат: достижение заданного числа поколений или вычислений функции, отсутствие улучшения лучшей особи в течение заданного числа поколений, достижение известного оптимального значения с заданной точностью, вырождение популяции (все особи одинаковы).

5.4 Простой генетический алгоритм

Простой (классический) ГА, описанный Д. Голдбергом [6], использует:
– двоичное кодирование решений строками фиксированной длины L ;

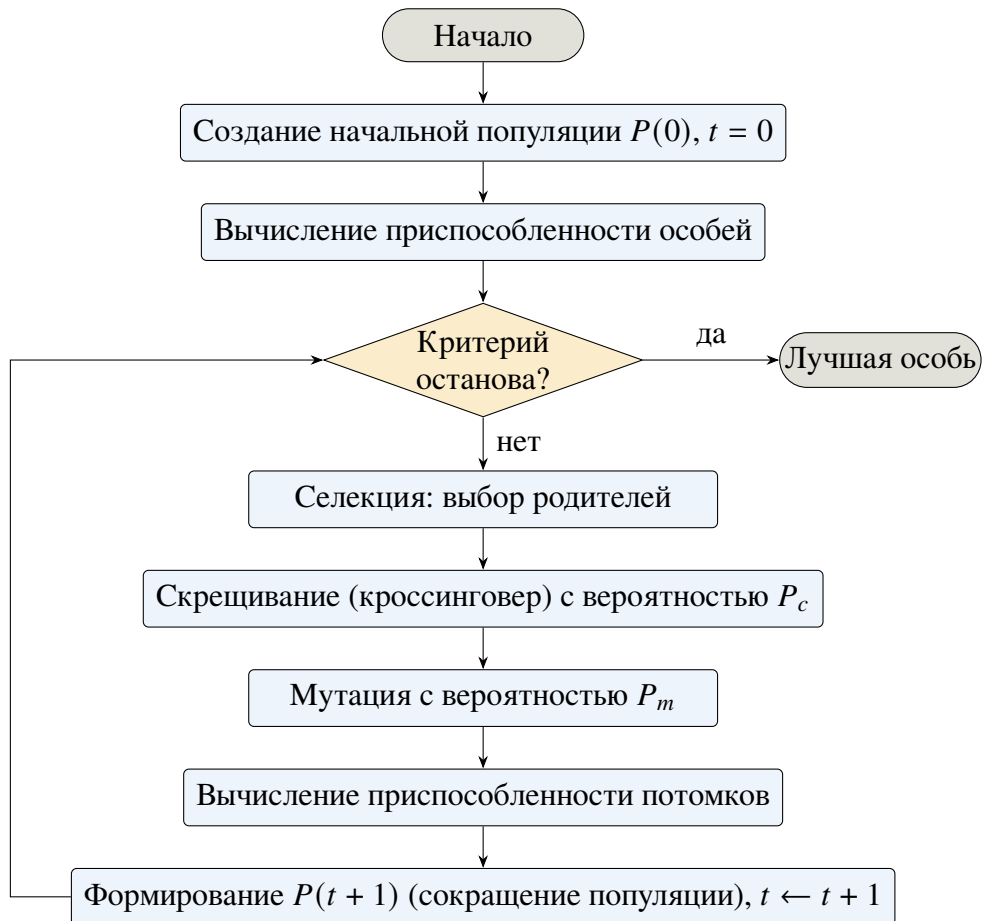


Рисунок 5.1 – Общая схема генетического алгоритма

- пропорциональную селекцию (метод «рулетки»): особь i выбирается в родительский пул с вероятностью

$$p_i = \frac{F_i}{\sum_{j=1}^N F_j}; \quad (5.1)$$

- одноточечный кроссинговер: пары родителей с вероятностью P_c (обычно 0,6–0,95) обмениваются частями хромосом после случайной точки разреза $k \in \{1, \dots, L - 1\}$;
- побитовую мутацию: каждый бит инвертируется с малой вероятностью P_m (обычно 0,001–0,01 или $1/L$);
- полную замену поколения: потомки заменяют родителей.

Пример. Найдём максимум $f(x) = x(30 - x)$ на целых числах $x \in [0; 31]$, закодированных пятью битами; истинный максимум $f(15) = 225$. Пусть начальная популяция из $N = 4$ особей получена случайно (таблица 5.2). Сумма приспособленностей равна 750, поэтому вероятности отбора $p_i = f_i/750$, а ожидаемое число копий Np_i .

Отрезки рулетки: $[0; 0,295)$ — особь 1, $[0,295; 0,487)$ — особь 2, $[0,487; 0,721)$ — особь 3, $[0,721; 1]$ — особь 4. Случайные числа 0,12; 0,27; 0,55; 0,90 выбирают в родительский пул особи 1, 1, 3, 4: лучшая особь получила две копии, худшая (особь 2) — ни одной, что соответствует ожидаемым

Таблица 5.2 – Одно поколение простого ГА для $f(x) = x(30 - x)$

№	Начальная популяция и селекция					После скрещивания		
	Строка	x	$f(x)$	p_i	Np_i	Пара, точка	Потомок	$f(x)$
1	01101	13	221	0,295	1,18	1–4, $k = 3$	011 11 = 15	225
2	11000	24	144	0,192	0,77	1–4, $k = 3$	100 01 = 17	221
3	01000	8	176	0,235	0,94	1–3, $k = 4$	0110 0 = 12	216
4	10011	19	209	0,279	1,11	1–3, $k = 4$	0100 1 = 9	189
Сумма / среднее / макс.			750 / 187,5 / 221			851 / 212,8 / 225		

числам Np_i . Пары (1, 4) и (1, 3) скрещиваются в точках $k = 3$ и $k = 4$. Потомок 01111 ($x = 15$) оказался оптимальным: он объединил начало 011 особи 1 с окончанием 11 особи 4. Мутация с $P_m = 0,01$ на 20 битах в среднем меняет 0,2 бита за поколение; в примере мутаций нет. Средняя приспособленность выросла с 187,5 до 212,8 за одно поколение.

5.5 Представление вещественных решений в двоичной форме

Для оптимизации функции непрерывной переменной $x \in [a; b]$ отрезок разбивается на $2^L - 1$ равных частей, и строке s из L бит сопоставляется узел сетки:

$$x = a + \text{dec}(s) \frac{b - a}{2^L - 1}, \quad \text{dec}(s) = \sum_{j=0}^{L-1} s_j 2^j. \quad (5.2)$$

Шаг сетки $\Delta = (b - a)/(2^L - 1)$ определяет предельную точность: никакой ГА не найдёт решение точнее $\Delta/2$. Чтобы получить точность ε , длину хромосомы выбирают из условия

$$2^L - 1 \geq \frac{b - a}{\varepsilon}, \quad L = \left\lceil \log_2 \left(\frac{b - a}{\varepsilon} + 1 \right) \right\rceil. \quad (5.3)$$

Например, для $[0; 7]$ и $\varepsilon = 10^{-5}$ получаем $\log_2 700\,001 \approx 19,4$, т.е. $L = 20$ и $\Delta = 6,7 \cdot 10^{-6}$. Слишком короткая хромосома делает требуемую точность недостижимой, а каждый лишний бит вдвое увеличивает пространство поиска без выигрыша в качестве.

При n переменных хромосома — конкатенация n строк, каждая из которых декодируется по формуле (5.2) на своём отрезке. Следует помнить, что узлы сетки не обязательно содержат точку оптимума: например, при симметричной области $[-100; 100]$ и чётном числе интервалов нет узла, равного нулю, так как $(2^L - 1)/2$ — не целое число. Предел точности ГА тогда определяется значением функции в ближайшем узле.

5.6 Код Грея

У обычного двоичного кода соседние целые числа могут отличаться многими битами: $7 = 0111$ и $8 = 1000$ различаются всеми четырьмя. Для перехода между ними мутацией нужно одновременно изменить все

биты — возникает «хэмминговский обрыв» (Hamming cliff), мешающий доводке решения. В коде Грея соседние числа отличаются ровно одним битом (таблица 5.3).

Таблица 5.3 – Двоичный код и код Грея

Число	0	1	2	3	4	5	6	7
Двоичный код	000	001	010	011	100	101	110	111
Код Грея	000	001	011	010	110	111	101	100

Преобразования (разряды нумеруются от старшего b_1 к младшему b_L , $\oplus$ — сложение по модулю 2):

$$g_1 = b_1, \quad g_i = b_{i-1} \oplus b_i \quad (i \geq 2); \quad b_1 = g_1, \quad b_i = b_{i-1} \oplus g_i. \quad (5.4)$$

В NumPy прямое преобразование целого числа записывается как $g = b \wedge (b \gg 1)$, а обратное для массива битов — как накопленное исключающее ИЛИ `np.bitwise_xor.accumulate(g, axis=1)`, что равносильно матричной форме $\mathbf{b} = \mathbf{A}\mathbf{g} \pmod{2}$ с нижней треугольной матрицей $\mathbf{A}$ из единиц. Код Грея сохраняет близость: малое изменение числа требует малого изменения кода, поэтому мутация чаще порождает близкие решения. Эффект заметен на практике: в примере ЛР №5 все неудачные запуски двоичного ГА останавливаются у хэммингового обрыва, а с кодом Грея таких остановок нет.

5.7 Теория схем и фундаментальная теорема ГА

Почему ГА работает? Классическое объяснение дано Холландом через понятие *схемы* — шаблона в алфавите $\{0, 1, *\}$, где $*$ означает «любое значение». Схема $H = 1**0*$ описывает все строки длины 5 с единицей в первой позиции и нулём в четвёртой; ей соответствует $2^3 = 8$ строк. Характеристики схемы:

- *порядок* $o(H)$ — число фиксированных позиций ($o(1**0*) = 2$);
- *определяющая длина* $\delta(H)$ — расстояние между первой и последней фиксированными позициями ($\delta(1**0*) = 4 - 1 = 3$).

Каждая строка длины L принадлежит 2^L схемам, поэтому, оценивая N строк, ГА неявно получает информацию о гораздо большем числе схем (*неявный параллелизм*; по оценке Холланда эффективно обрабатывается порядка N^3 схем).

Пусть $t(H, t)$ — число строк популяции, принадлежащих схеме H в поколении t , $F(H)$ — средняя приспособленность этих строк, $\bar{F}$ — средняя приспособленность популяции. Оценим действие операторов простого ГА.

Репродукция. При пропорциональной селекции строка выбирается с вероятностью $F_i/\sum F_j$, поэтому после N выборов

$$m(H, t + 1) = m(H, t) \frac{F(H)}{\bar{F}}. \quad (5.5)$$

Схемы с приспособленностью выше средней получают больше копий, ниже средней — меньше. Если схема остаётся выше средней на $c\bar{F}$, её число растёт в геометрической прогрессии: $m(H, t) = m(H, 0)(1 + c)^t$.

Кроссинговер разрушает схему, если точка разреза попадает между её фиксированными позициями. Вероятность этого не превышает $\delta(H)/(L - 1)$, поэтому схема выживает с вероятностью $p_s \geq 1 - P_c \delta(H)/(L - 1)$.

Мутация сохраняет схему, если не изменена ни одна из $o(H)$ фиксированных позиций: $(1 - P_m)^{o(H)} \approx 1 - o(H)P_m$ при $P_m \ll 1$.

Объединяя эти оценки, получаем *фундаментальную теорему ГА* (теорему схем):

$$m(H, t + 1) \geq m(H, t) \frac{F(H)}{\bar{F}} \left[1 - P_c \frac{\delta(H)}{L - 1} - o(H) P_m \right]. \quad (5.6)$$

Короткие схемы низкого порядка с приспособленностью выше средней получают экспоненциально растущее число представителей в следующих поколениях. Такие схемы называют *строительными блоками*. *Гипотеза строительных блоков* утверждает, что ГА находит хорошие решения, комбинируя короткие высокоприспособленные блоки в более длинные — именно это произошло в примере таблицы 5.2, где потомок 01111 собран из блоков 011** и ***11.

Теорема схем даёт нижнюю оценку на одно поколение и не учитывает создание новых представителей схемы кроссинговером, поэтому не является строгим доказательством сходимости. Существуют *обманчивые* (deceptive) функции, для которых короткие схемы ведут в сторону от оптимума. Тем не менее теорема объясняет практические правила: гены, совместно влияющие на качество, следует располагать в хромосоме рядом; вероятность мутации должна быть мала, иначе разрушаются даже хорошие схемы; давление отбора определяется отношением $F(H)/\bar{F}$ и зависит от масштаба функции приспособленности.

6 МОДИФИКАЦИИ ГЕНЕТИЧЕСКИХ ОПЕРАТОРОВ

Раздел используется в ЛР №5–7.

6.1 Создание начальной популяции

Начальная популяция должна быть разнообразной, чтобы ГА с самого начала «видел» всю область поиска. Применяют три способа:

- *случайная генерация* («дробовик»): каждый ген выбирается равновероятно из допустимых значений; стандартный способ при отсутствии априорной информации;
- *фокусировка*: особи генерируются в окрестности известных хороших решений, полученных эвристикой или предыдущими запусками; ускоряет поиск, но повышает риск преждевременной сходимости;
- *комбинированный способ*: часть популяции создаётся эвристически, остальные особи — случайно.

Для перестановочных задач (коммивояжёр) начальные особи — случайные перестановки или туры, построенные жадным методом ближайшего соседа.

6.2 Селекция

Оператор селекции формирует из текущей популяции *родительский пул*, используя приспособленность как единственный источник информации о качестве решений. Методы селекции различаются *давлением отбора* — тем, насколько сильно лучшие особи преобладают в пуле. Слабое давление замедляет сходимость, сильное ведёт к быстрой потере разнообразия и *преждевременной сходимости* к локальному экстремуму.

Пропорциональный отбор (рулетка, формула (5.1)) каждой особи сопоставляет сектор колеса, пропорциональный её приспособленности. Недостатки метода: он требует положительных значений F ; зависит от масштаба (прибавление к F большой константы делает выбор почти равновероятным); в начале поиска одна «суперособь» может захватить популяцию, а в конце, когда значения F близки, давление отбора исчезает. Для устранения этих эффектов применяют *масштабирование* приспособленности:

- сдвиг: $F' = F - F_{\min}$ (для максимизации) или $F' = F_{\max} - F$ (для минимизации);
- линейное: $F' = aF + b$, коэффициенты выбирают так, чтобы среднее не изменилось, а лучшая особь получала в среднем $C \approx 1,5\text{--}2$ копии;
- сигма-отсечение: $F' = \max(0, F - (\bar{F} - c\sigma_F))$, $c \in [1; 3]$, где σ_F — стандартное отклонение приспособленности в популяции;
- степенное: $F' = F^k$, $k > 1$;
- больцмановское: $F' = \exp(F/T)$ с убывающей «температурой» T , как при имитации отжига.

Реализацию рулетки улучшает *стохастическая универсальная выборка* (SUS, Дж. Бейкер): колесо вращают один раз, а N указателей расставлены с шагом $1/N$. Число копий каждой особи тогда отличается от ожидаемого Np_i не более чем на единицу.

Ранжирование. Особи сортируются по приспособленности, и вероятность выбора зависит только от ранга i ($i = 1$ — лучшая), а не от самого значения F . При линейном ранжировании

$$p_i = \frac{1}{N} \left(a - (a - b) \frac{i - 1}{N - 1} \right), \quad 1 \leq a \leq 2, \quad b = 2 - a, \quad (6.1)$$

где a — ожидаемое число копий лучшей особи. Метод не чувствителен к масштабу и знаку F и сохраняет давление отбора, когда значения приспособленности близки.

Отбор усечением (truncation): в пул допускаются только $T \cdot 100\%$ лучших особей ($10\% \leq T \leq 50\%$), среди которых родители выбираются равновероятно. Применяется в основном для больших популяций.

Турнирный отбор: из популяции случайно выбираются k особей, и лучшая из них становится родителем; процедура повторяется N раз. Размер турнира k ($2 \leq k \leq N$) прямо регулирует давление отбора: вероятность выбора особи ранга i равна $((N - i + 1)^k - (N - i)^k) / N^k$. Турнир не зависит от масштаба F , легко распараллеливается и потому является самым распространённым методом в современных ГА. Сравнение методов приведено на рисунке 6.1.

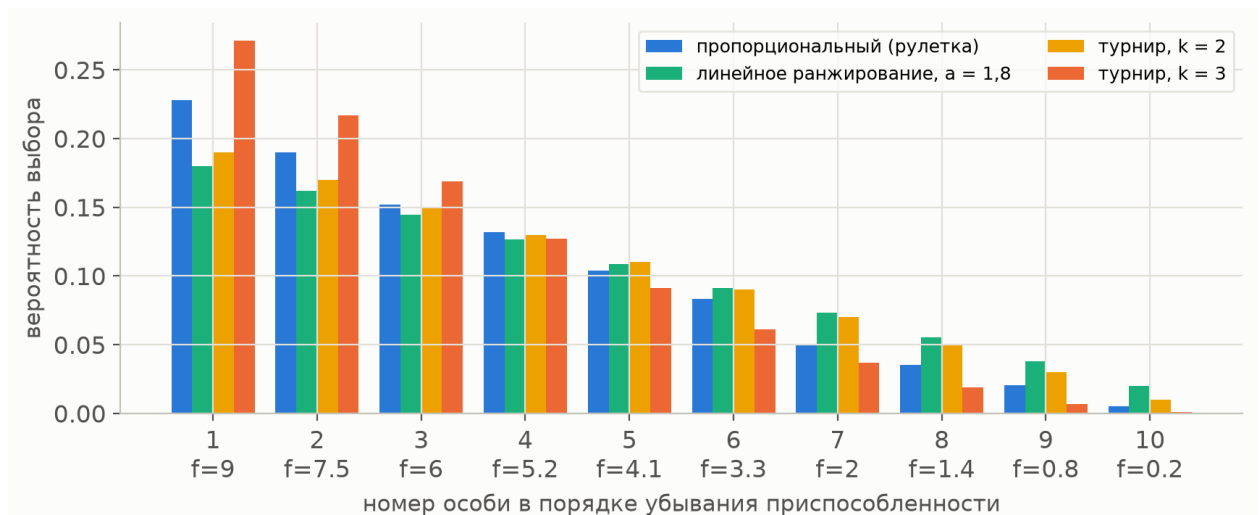


Рисунок 6.1 – Вероятности выбора особей при разных методах селекции (популяция из 10 особей)

Локальный отбор выполняется среди соседей особи в пространственно структурированной популяции (линейной, двумерной с 4- или 8-связностью). Малые окрестности замедляют распространение хороших решений и помогают сохранить разнообразие.

После формирования пула выбираются *пары для скрещивания*:

- *панмиксия* — оба родителя выбираются случайно; универсальный метод;
- *селективный выбор* — родителями могут стать только особи с приспособленностью не ниже средней; ускоряет сходимость, но опасен для многоэкстремальных задач;
- *инбридинг* — второй родитель выбирается как ближайший к первому (генотипически, по расстоянию Хэмминга, или фенотипически, по расстоянию в пространстве решений); способствует локальному поиску внутри «ниши»;
- *аутбридинг* — пары образуют наиболее далёкие особи; поддерживает разнообразие.

Комбинация инбридинга и аутбридинга применяется при поиске нескольких экстремумов.

6.3 Скрещивание двоичных хромосом

Оператор скрещивания (кроссинговер, рекомбинация) создаёт потомков, комбинируя гены двух родителей. Он применяется к паре с вероятностью P_c ; в противном случае потомки — копии родителей. Основные двоичные операторы показаны на рисунке 6.2:

- *одноточечный*: случайно выбирается точка $k \in \{1, \dots, L - 1\}$, и родители обмениваются частями хромосом после неё;
- *многоточечный*: выбираются m точек, и родители обмениваются фрагментами через один; двухточечный кроссинговер меньше разрушает схемы, фиксированные позиции которых находятся на концах хромосомы;
- *однородный*: генерируется случайная двоичная маска, и каждый ген первого потомка берётся из первого родителя там, где в маске 1, и из второго там, где 0 (второй потомок — наоборот). Каждая позиция является потенциальной точкой обмена, поэтому однородный кроссинговер сильнее всего перемешивает гены и не зависит от их взаимного расположения.

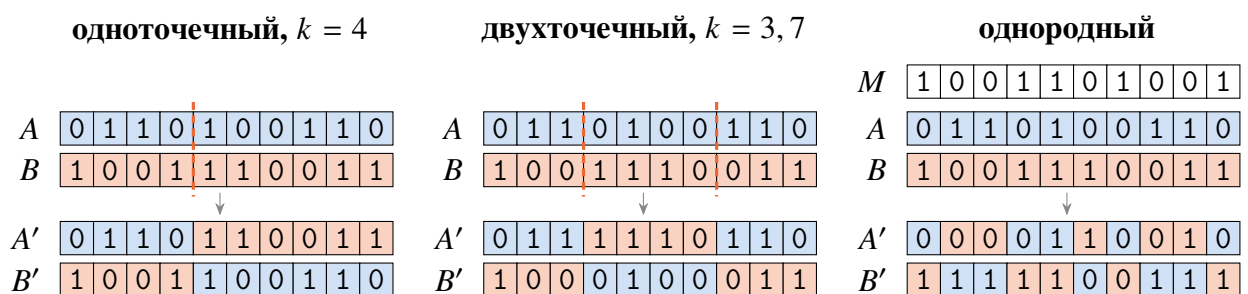


Рисунок 6.2 – Двоичные операторы скрещивания: A, B — родители, A', B' — потомки, M — маска однородного кроссинговера

6.4 Мутация двоичных хромосом

Мутация вносит в популяцию новые значения генов, которых могло не быть ни у одного родителя, и предотвращает вырождение популяции. Для двоичного кодирования применяются (рисунок 6.3):

- *побитовая мутация* — каждый бит инвертируется с вероятностью P_m (типично $1/L$, т. е. в среднем один бит на хромосому);
- *инверсия* — выбираются две позиции, и отрезок хромосомы между ними записывается в обратном порядке; порядок генов меняется, а их состав (число нулей и единиц) сохраняется;
- *обмен (перестановка)* — два случайных гена меняются местами.

Инверсия и обмен не создают новых аллелей: если во всей популяции в некоторой позиции стоит 0, эти операторы могут перенести туда единицу из другой позиции хромосомы, но не гарантируют этого. Поэтому для числовых задач с двоичным кодированием их используют вместе с побитовой мутацией.

	побитовая, позиция 4	инверсия, позиции 3–7	обмен, позиции 1 и 5
до	0 1 1 0 1 0 0 1 1 0	0 1 1 0 1 0 0 1 1 0	0 1 1 0 1 0 0 1 1 0
после	0 1 1 1 1 0 0 1 1 0	0 1 0 0 1 0 1 1 1 0	1 1 1 0 0 0 0 1 1 0

Рисунок 6.3 – Операторы мутации двоичной хромосомы (изменённые гены выделены)

6.5 Вещественное кодирование

При *вещественном кодировании* особь — вектор $\mathbf{x} = (x_1, \dots, x_n)$ самих параметров задачи. Оно не ограничено сеткой двоичного кода, не требует декодирования, естественно для непрерывных задач и позволяет использовать операторы, учитывающие метрику пространства. Операторы скрещивания для вещественных векторов [4, 7]:

- *дискретная рекомбинация* — каждая компонента потомка берётся из первого или второго родителя с равной вероятностью (аналог однородного кроссинговера);
- *промежуточная рекомбинация* — компоненты потомка лежат между значениями родителей или в их окрестности:

$$c_i = p_i^1 + \alpha_i(p_i^2 - p_i^1), \quad \alpha_i \in [-d; 1 + d], \quad (6.2)$$

где α_i выбирается случайно для каждой компоненты; при $d = 0$ потомок лежит внутри гиперпрямоугольника родителей, при $d > 0$ (обычно $d = 0,25$) — и вне его;

- *линейная рекомбинация* — то же, но α одно для всех компонент: потомок лежит на прямой, проходящей через родителей; частный случай $\alpha \in [0; 1]$ — *арифметический* кроссинговер $\mathbf{c} = \lambda \mathbf{p}^1 + (1 - \lambda) \mathbf{p}^2$;

- *BLX- α* (blend crossover, Л. Эшелман и Д. Шаффер [23]) — компонента потомка выбирается равномерно из интервала, расширенного на долю α расстояния между родителями:

$$c_i \sim U[\min(p_i^1, p_i^2) - \alpha d_i; \max(p_i^1, p_i^2) + \alpha d_i], \quad d_i = |p_i^1 - p_i^2|. \quad (6.3)$$

BLX- α совпадает с промежуточной рекомбинацией при $d = \alpha$. Значение $\alpha \approx 0,5$ сохраняет разброс популяции в среднем неизменным, $\alpha = 0$ сжимает популяцию, большие α приближают поиск к случайному (рисунок 6.4).

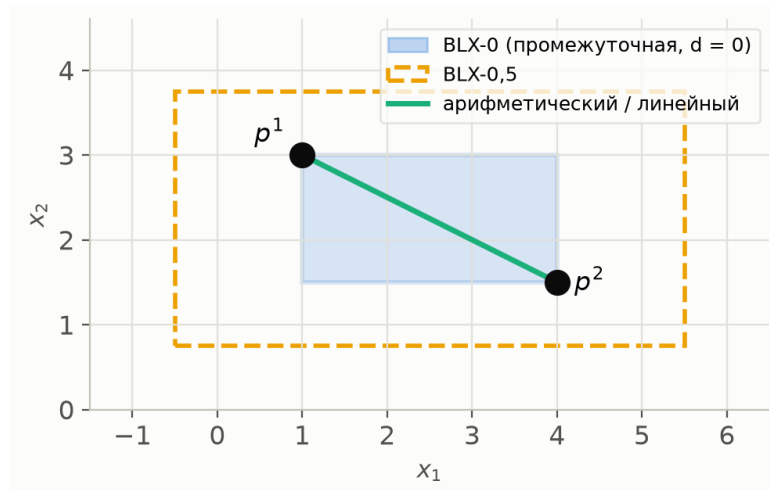


Рисунок 6.4 – Области возможных потомков при вещественном скрещивании двух родителей

Мутация вещественной особи — прибавление к гену случайного *шага мутации*:

- *равномерная*: ген заменяется случайным значением из допустимого отрезка $[a_i; b_i]$;
- *гауссова*: $x'_i = x_i + \mathcal{N}(0, \sigma^2)$; малый шаг даёт точность, большой — способность выходить из локальных экстремумов. Поэтому шаг уменьшают по ходу поиска, например линейно: $\sigma(t) = \sigma_0(b - a)(1 - t/T)$, где T — число поколений;
- *неравномерная мутация* З. Михалевича [7]: $x'_i = x_i + \Delta(t, b_i - x_i)$ или $x'_i = x_i - \Delta(t, x_i - a_i)$ с равной вероятностью, где

$$\Delta(t, y) = y \left(1 - r^{(1-t/T)^\beta} \right), \quad (6.4)$$

$r \sim U[0; 1]$, $\beta \approx 2$. В начале поиска шаг может достигать границ области, к концу он стремится к нулю, и мутация выполняет локальную доводку.

Если после мутации или *BLX- α* ген выходит за пределы допустимого отрезка, его возвращают на границу (отсечение) или отражают внутрь области.

6.6 Сокращение промежуточной популяции

После скрещивания и мутации получается промежуточная популяция из родителей и потомков, численность которой больше N . Стратегия сокращения (редукции) определяет, кто перейдёт в следующее поколение:

- *чистая замена* (поколенческая схема): N потомков полностью заменяют родителей; каждая особь живёт одно поколение, и лучшее решение может быть потеряно;
- *элитарная схема* ($\mu + \lambda$): из объединения родителей и потомков отбираются N лучших; найденное лучшее решение никогда не теряется, но разнообразие убывает быстрее. Частный случай — *элитизм*: в новое поколение без изменений переносятся 1–2 лучшие особи, остальные места занимают потомки;
- *равномерная случайная замена*: N особей выбираются из объединения равновероятно;
- *пропорциональная редукция*: особи выбираются из объединения рулеткой по приспособленности;
- *отбор вытеснением* (crowding): потомок заменяет наиболее похожую на него особь, что сохраняет разнообразие и позволяет поддерживать несколько экстремумов;
- *схема Больцмана*: потомок заменяет родителя с вероятностью, зависящей от разности их приспособленностей и убывающей температуры;
- *локальная замена*: в пространственно структурированной популяции потомок заменяет особь из окрестности своего родителя.

В *стационарном* ГА (steady-state) за итерацию создаётся один–два потомка, которые заменяют худшие особи; поколения в явном виде не выделяются.

6.7 Настройка параметров и методика эксперимента

Работа ГА определяется балансом двух тенденций: *исследования* (exploration) — поиска в новых областях, которое обеспечивают мутация, большая популяция и слабое давление отбора, и *использования* (exploitation) — уточнения найденных хороших решений, которое обеспечивают селекция, элитизм и скрещивание близких особей. Перекос в сторону исследования превращает ГА в случайный поиск, перекос в сторону использования ведёт к преждевременной сходимости. Ориентировочные значения параметров приведены в таблице 6.1.

Результат одного запуска ГА случаен, поэтому выводы делают по серии из 20–50 независимых запусков с разными начальными значениями генератора случайных чисел. Для каждой настройки фиксируют:

- долю *успешных* запусков — найдено решение с заданной точностью относительно известного оптимума;
- медиану и худшее значение найденного экстремума;

Таблица 6.1 – Типичные значения параметров генетического алгоритма

Параметр	Диапазон	Влияние
Размер популяции N	20–200	главный фактор надёжности; растёт с числом переменных и числом экстремумов
Вероятность скрещивания P_c	0,6–0,95	ускоряет сходимость; при $P_c = 0$ ГА сводится к мутационному поиску
Вероятность мутации P_m	$1/L$; 0,001–0,01 на бит	без мутации популяция вырождается, при большой — случайный поиск
Размер турнира k	2–5	давление отбора
Число элитных особей	1–2	монотонность лучшего решения
Длина хромосомы L	формула (5.3)	предельная точность и размер пространства поиска

– *скорость* — медиану числа поколений или вычислений функции до первого успеха.

При исследовании параметра меняют только его, оставляя остальные неизменными. Сравнение разных алгоритмов корректно только при одинаковом бюджете вычислений функции.

6.8 Векторная реализация операторов на NumPy

Популяцию из N двоичных хромосом длины L удобно хранить массивом `pop` формы (N, L) типа `int8`, приспособленности — вектором `F`. Тогда каждый оператор — одна-две строки без циклов по особям:

```

idx = rng.choice(N, N, p=F / F.sum())           # рулетка (F > 0)
cand = rng.integers(0, N, (N, k))               # турнир: k случайных особей,
idx = cand[np.arange(N), F[cand].argmax(axis=1)] # побеждает лучшая
a, b = pop[idx][0::2], pop[idx][1::2]          # пары родителей
cut = rng.integers(1, L, (len(a), 1))           # одноточечный кроссинговер
mask = (np.arange(L) >= cut) & (rng.random((len(a), 1)) < pc)
kids = np.vstack([np.where(mask, b, a), np.where(mask, a, b)])
kids ^= (rng.random(kids.shape) < pm).astype(np.int8) # побитовая мутация
t = lo + kids @ 2 ** np.arange(L - 1, -1, -1) * (hi - lo) / (2 ** L - 1) # декодирование (5.2)

```

Для однородного кроссинговера маска генерируется как `rng.random(a.shape) < 0.5`, для BLX- α потомок — `rng.uniform(np.minimum(a, b) - alpha * d, np.maximum(a, b) + alpha * d)`.

Готовые библиотеки используются для проверки своей реализации, а не вместо неё. В DEAP [24] операторы собраны в модуле `deap.tools` (`selTournament`, `cxTwoPoint`, `cxUniform`, `mutFlipBit`, `cxBlend`, `mutGaussian`), в PyGAD задаются параметрами `parent_selection_type`, `crossover_type`, `mutation_type`. Эталонный экстремум для оценки точности находят детерминированными методами: `scipy.optimize.minimize_scalar` на мелкой сетке для функции одной переменной и `scipy.optimize.differential_evolution` для функций двух переменных.

7 ЗАДАЧИ КОМБИНАТОРНОЙ ОПТИМИЗАЦИИ

Раздел используется в ЛР №7.

7.1 Особенности применения генетических алгоритмов

В задачах комбинаторной оптимизации решение — подмножество, перестановка или разбиение конечного множества объектов, а число допустимых решений растёт экспоненциально или факториально с размером задачи. Многие такие задачи NP-трудны, и ГА здесь — эвристика, находящая хорошие решения за приемлемое время. Главные вопросы при его построении:

- *представление*: кодирование должно покрывать все допустимые решения и по возможности только их;
- *операторы*: скрещивание и мутация должны сохранять допустимость и передавать потомкам существенные для качества свойства родителей (для маршрута — смежность городов, а не их абсолютные позиции);
- *учёт ограничений*: недопустимые особи *штрафуются* (штраф растёт с величиной нарушения), *восстанавливаются* (repair) эвристикой либо не возникают благодаря специальным операторам.

7.2 Задача о рюкзаке и учёт ограничений

Задача о рюкзаке (knapsack problem): из n предметов с весами w_i и ценностями c_i выбрать набор наибольшей суммарной ценности, суммарный вес которого не превышает вместимости рюкзака W :

$$\sum_{i=1}^n c_i x_i \rightarrow \max, \quad \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}, \quad (7.1)$$

где $x_i = 1$, если предмет i взят. К этой схеме сводятся загрузка транспорта и складских ячеек, выбор проектов или закупок при ограниченном бюджете, распределение рекламного бюджета по каналам, раскрой материала. На практике встречаются варианты с несколькими ограничениями (вес, объём и бюджет одновременно — многомерный рюкзак), с несколькими рюкзаками и с ценностью, зависящей от сочетания предметов [25].

Задача NP-трудна, но при целых весах точно решается динамическим программированием за $O(nW)$ операций, а жадный алгоритм — брать предметы в порядке убывания удельной ценности c_i/w_i , пока они помещаются, — обычно даёт решение, близкое к оптимальному. Поэтому рюкзак удобен как *учебная* задача: ответ ГА всегда можно сверить с точным.

Решение естественно кодируется двоичной хромосомой $X = (x_1, \dots, x_n)$, и простой ГА со стандартными операторами применим без изменений. Трудность в другом: кроссинговер и мутация порождают пере-

груженные рюкзаки. Способы учёта ограничений общие для всех задач с ограничениями [7, 4].

1. *Штрафная функция.* Задача с ограничением заменяется задачей без ограничения: $f(X) = \sum_i c_i x_i - \text{Pen}(X)$, где штраф растёт с перегрузкой $\Delta(X) = \max(0, \sum_i w_i x_i - W)$ логарифмически, линейно или квадратично:

$$\text{Pen}(X) = \log_2(1 + \rho \Delta(X)), \quad \text{Pen}(X) = \rho \Delta(X), \quad \text{Pen}(X) = (\rho \Delta(X))^2, \\ \rho = \max_i c_i / w_i.$$

Слишком слабый штраф делает перегруженный рюкзак выгоднее допустимого, слишком сильный отбрасывает почти всю начальную популяцию и лишает поиск разнообразия.

2. *Восстановление (repair):* недопустимая хромосома исправляется, а фитнес вычисляется по исправленной версии X' . Из рюкзака удаляют предметы, пока он не станет допустимым: случайные или, при *жадном восстановлении*, с наименьшим отношением c_i/w_i . Исправленная особь может заменять исходную в популяции (ламарковская схема) или только давать ей оценку (болдуиновская); в литературе рекомендуют заменять около 5 % особей [7].

3. *Декодер:* хромосома кодирует не само решение, а *способ его построения*, при котором ограничение выполняется автоматически. В *порядковом представлении* задан базовый список предметов L , а ген $e_i \in [1; n - i + 1]$ — номер предмета в текущем списке. Предметы извлекаются из списка по очереди и кладутся в рюкзак, если помещаются. Например, при $L = (1\ 2\ 3\ 4\ 5\ 6)$ хромосома $e = (4\ 3\ 4\ 1\ 1\ 1)$ задаёт порядок 4, 3, 6, 1, 2, 5 [4]. Любая хромосома декодируется в допустимое решение, поэтому работают обычный одноточечный кроссинговер и мутация, заменяющая ген случайным числом из допустимого диапазона. Если упорядочить L по убыванию c_i/w_i , хромосома из одних единиц даёт решение жадного алгоритма, а остальные — его вариации.

Таблица 7.1 – Пример задачи о рюкзаке, $W = 30$

Предмет i	1	2	3	4	5	6
Вес w_i	16	15	15	8	6	5
Ценность c_i	34	30	31	15	11	8
c_i/w_i	2,125	2,000	2,067	1,875	1,833	1,600

Пример. Для задачи из таблицы 7.1 хромосома $X = (1\ 1\ 0\ 1\ 0\ 0)$ имеет вес 39 и ценность 79, перегрузка $\Delta = 9$, $\rho = 2,125$. Логарифмический штраф даёт $f = 79 - 4,3 = 74,7$, линейный — $f = 79 - 19,1 = 59,9$, квадратичный — $f = 79 - 365,8 = -286,8$. Оптимум задачи — предметы 2 и 3 с ценностью 61, поэтому при логарифмическом штрафе недопустимая особь оказывается лучше оптимума. Жадное восстановление удаляет из X предмет 4, затем предмет 2 и оставляет только предмет 1 с ценностью 34; жадный алгоритм

выбирает предметы 1, 4, 5 с ценностью 60. Декодер с жадным списком $L = (1\ 3\ 2\ 4\ 5\ 6)$ переводит хромосому $(1\ 1\ 1\ 1\ 1\ 1)$ в жадное решение, а хромосому $(2\ 1\ 1\ 1\ 1\ 1)$ — в оптимальное: сначала берётся предмет 3, предмет 1 уже не помещается, затем берётся предмет 2.

7.3 Задача о минимальном покрытии

Дано конечное множество S и семейство его подмножеств $F_1, \dots, F_n$. Требуется выбрать наименьшее число подмножеств, объединение которых совпадает с S [4]. В общем случае подмножество F_j имеет стоимость c_j , и задача записывается через двоичную матрицу $A = (a_{ij})$, $a_{ij} = 1$, если F_j содержит элемент i (столбец j покрывает строку i):

$$\sum_{j=1}^n c_j x_j \rightarrow \min, \quad \sum_{j=1}^n a_{ij} x_j \geq 1, \quad i = 1, \dots, m, \quad x_j \in \{0, 1\}. \quad (7.2)$$

К задаче о покрытии сводятся размещение пунктов обслуживания (станций скорой помощи, пожарных частей, пунктов выдачи заказов), когда каждый район должен обслуживаться хотя бы одним пунктом; составление смен экипажей, когда каждый рейс должен быть закрыт хотя бы одной сменой; выбор минимального набора тестов, обнаруживающих все неисправности цифровой схемы.

Хромосома — двоичный вектор выбранных подмножеств, ограничение — покрытие всех элементов. *Жадный алгоритм* [26] на каждом шаге добавляет подмножество с наименьшей стоимостью в расчёте на один ещё не покрытый элемент; его решение хуже оптимального не более чем в $H(d) = 1 + \frac{1}{2} + \dots + \frac{1}{d} \approx \ln d + 1$ раз, где d — размер наибольшего подмножества. В ГА непокрытые элементы учитываются штрафом ($f(X) = \sum_j c_j x_j + P \cdot$ число непокрытых строк) или восстановлением: для каждой непокрытой строки добавляется столбец с наименьшим отношением стоимости к числу покрываемых им непокрытых строк, затем в порядке убывания стоимости удаляются *избыточные* столбцы, без которых покрытие сохраняется [27].

Пример. $S = \{1, \dots, 6\}$, $F_1 = \{1, 2, 3\}$, $F_2 = \{4, 5, 6\}$, $F_3 = \{1, 2, 4, 5\}$, $F_4 = \{3, 5\}$, все стоимости равны единице. Жадный алгоритм сначала берёт самое большое F_3 , затем F_1 для элемента 3 и F_2 для элемента 6 — три подмножества. После них F_3 избыточно: его элементы покрыты F_1 и F_2 , и удаление избыточных столбцов даёт оптимальное покрытие $\{F_1, F_2\}$. Хромосома $(0\ 0\ 1\ 1)$ не покрывает элемент 6, восстановление добавляет F_2 и получает допустимое, но не оптимальное покрытие $\{F_2, F_3, F_4\}$; к оптимуму $(1\ 1\ 0\ 0)$ ведут уже кроссинговер и мутация.

Сравнение способов учёта ограничений. Задача о рюкзаке: 100 предметов, веса $w_i \in [10; 100]$, ценности $c_i = w_i + \delta_i$, $\delta_i \in [-10; 10]$, вместимость — 25 % суммы весов ($W = 1397$). Оптимум 1634 найден динамическим про-

граммированием за доли миллисекунды, жадный алгоритм даёт 1626 (99,5 %). Задача о покрытии: 200 строк, 1000 столбцов, плотность матрицы 2 %, стоимости от 1 до 100 (как в классе 4 библиотеки OR-Library [27]). Оптимум 460 найден решателем смешанного целочисленного программирования (MILP) за 0,02 с, жадный алгоритм даёт 554 (+20 %). ГА во всех случаях: $N = 100$, 200 поколений (20 000 вычислений фитнеса), турнир из трёх особей, однородный кроссинговер с $P_c = 0,9$, побитовая мутация $1/n$, две элиты (таблица 7.2, рисунок 7.1).

Таблица 7.2 – Способы учёта ограничений в ГА: задача о рюкзаке, 20 запусков

Способ	Медиана, % оптимума	Худший, % оптимума	Оптимум найден	Лучшая особь недопустима
штраф логарифмический	—	—	0	20
штраф линейный	99,63	99,14	0	10
штраф квадратичный	98,59	98,04	0	4
восстановление случайное	99,63	99,33	0	0
восстановление жадное	99,91	99,63	3	0
декодер	98,35	97,37	0	0
декодер + жадная особь	99,94	99,82	6	0

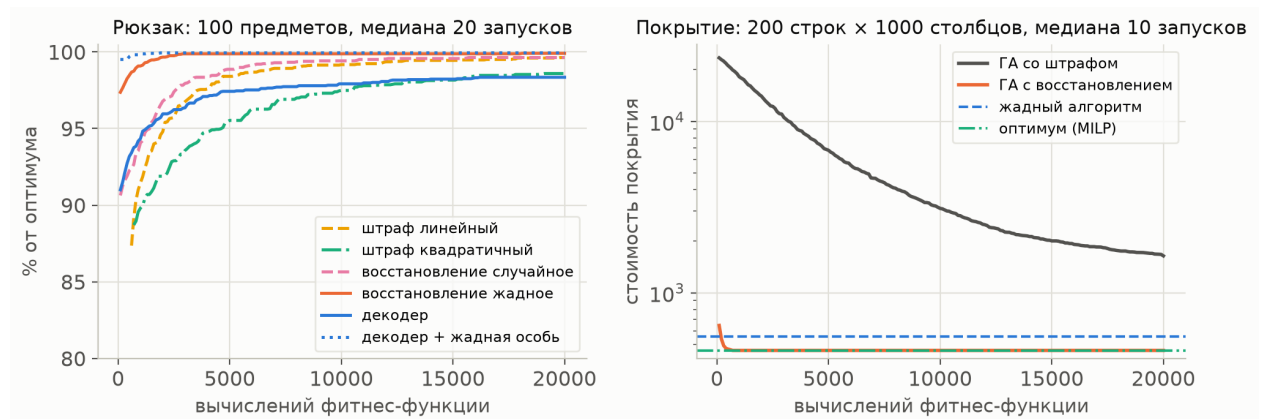


Рисунок 7.1 – Медиана лучшего допустимого решения: слева — задача о рюкзаке (логарифмический штраф допустимых решений не дал), справа — задача о покрытии (логарифмическая шкала)

С логарифмическим штрафом ГА ни разу не нашёл допустимого решения: перегруженные рюкзаки выгоднее. Линейный штраф даёт хороший результат, но в половине запусков лучшая по фитнесу особь недопустима — её приходится отбрасывать. Квадратичный штраф при малой перегрузке слабее линейного, а при большой отсекает почти всю начальную популяцию, и поиск идёт медленнее. Лучше всего работают способы, использующие знание о задаче: жадное восстановление и декодер с жадным списком. Простой декодер уступает даже жадному алгоритму, пока в начальную популяцию не добавлена жадная особь, — *затравка* популяции решением известной эвристики — стандартный практический приём. В задаче о покрытии разница ещё резче: ГА

со штрафом за 20 000 вычислений нашёл покрытие стоимостью 1641 (медиана 10 запусков), в 3,6 раза дороже оптимального, а ГА с восстановлением — 461 (оптимум 460 найден в трёх запусках из десяти).

Задачи такого размера точно решаются за доли секунды динамическим программированием и решателями MILP (`scipy.optimize.milp`, OR-Tools). Генетический алгоритм оправдан там, где точная модель не записывается линейными ограничениями: ценность зависит от сочетания предметов, решение оценивается имитационной моделью, размер задачи слишком велик для точного решателя или нужны многие разнообразные хорошие решения. Проверка ГА на задаче с известным оптимумом — обязательный первый шаг. Векторная оценка популяции на NumPy и точное решение для сравнения:

```
weight, value = P @ w, P @ c # P - популяция (N, n) из нулей и единиц
fit = value - rho * np.maximum(weight - W, 0) # линейный штраф
uncovered = (P @ A.T == 0).sum(axis=1) # покрытие: число непокрытых строк у каждой особи
from scipy.optimize import milp, LinearConstraint, Bounds
res = milp(cost, integrality=np.ones(n), bounds=Bounds(0, 1),
           constraints=LinearConstraint(A, lb=1)) # res.fun - оптимальная стоимость
```

7.4 Задача коммивояжёра

Дано n городов и матрица расстояний d_{ij} . Требуется найти замкнутый маршрут (тур), проходящий через каждый город ровно один раз и имеющий минимальную длину:

$$L(\pi) = \sum_{k=1}^{n-1} d_{\pi_k \pi_{k+1}} + d_{\pi_n \pi_1} \rightarrow \min_{\pi}, \quad (7.3)$$

где π — перестановка номеров городов. Для симметричной задачи число различных туров равно $(n-1)!/2$: для 76 городов это около $1,2 \cdot 10^{109}$, полный перебор невозможен. В тестовой библиотеке TSPLIB [28] для задач типа EUC_2D расстояние — евклидово, округлённое до ближайшего целого: $d_{ij} = \lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + 0,5 \rfloor$; для многих задач библиотеки известны оптимальные туры (таблица Л7.1).

Обычные двоичные операторы порождают строки, не являющиеся перестановками, поэтому используются три специальных представления тура [7, 4].

Представление соседства (adjacency): в позиции i записан город, следующий в туре за городом i . Тур 1–2–4–3–8–5–9–6–7 кодируется вектором (2 4 8 3 9 7 1 5 6). Не всякий вектор соответствует одному туру (возможны подциклы), поэтому применяются специальные операторы: *обмен рёбер* (потомок строится поочерёдным выбором ребра первого и второго родителей), *обмен подтуров* (чередуются фрагменты туров родителей) и *эвристическое скрещивание* (из текущего города выбирается более короткое из двух рёбер родителей). Представление явно работает с рёбрами, которые и определяют длину тура.

Порядковое представление (ordinal): задаётся упорядоченный список городов, например (1 2 ... 9); очередной город тура кодируется номером его позиции в списке, после чего город удаляется из списка. Тур 1–2–4–3–8–5–9–6–7 кодируется как (1 1 2 1 4 1 3 1 1). Любой вектор с i -м геном из $[1; n - i + 1]$ является допустимым туром, поэтому работает обычный одноточечный кроссинговер, но хвост хромосомы после разреза задаёт в потомке другие города, и хорошие фрагменты не наследуются.

Представление путей (path) — самое естественное: тур записывается как перестановка городов в порядке обхода, (1 2 4 3 8 5 9 6 7). Для него разработаны операторы, сохраняющие перестановку. Рассмотрим их на родителях

$$P_1 = (1\ 2\ 3\ |\ 4\ 5\ 6\ 7\ |\ 8\ 9), \quad P_2 = (4\ 5\ 2\ |\ 1\ 8\ 7\ 6\ |\ 9\ 3)$$

с точками разреза после третьей и седьмой позиций.

Частично соответствующий кроссинговер (PMX, Д. Голдберг и Р. Лингл [29]). Потомок C_1 получает средний сегмент P_1 : $(\cdot\ \cdot\ \cdot\ |\ 4\ 5\ 6\ 7\ |\ \cdot\ \cdot)$. Сегменты задают отображение $4 \leftrightarrow 1, 5 \leftrightarrow 8, 6 \leftrightarrow 7, 7 \leftrightarrow 6$. Остальные позиции заполняются генами P_2 ; если ген уже есть в сегменте, он заменяется по отображению (при необходимости многократно): $4 \rightarrow 1, 5 \rightarrow 8, 2$ и $9, 3$ переносятся без изменений. Получаем $C_1 = (1\ 8\ 2\ |\ 4\ 5\ 6\ 7\ |\ 9\ 3)$, аналогично $C_2 = (4\ 2\ 3\ |\ 1\ 8\ 7\ 6\ |\ 5\ 9)$. PMX сохраняет абсолютные позиции большинства городов.

Упорядоченный кроссинговер (ОХ, Л. Дэвис). Потомок получает сегмент P_1 , а свободные позиции, начиная с позиции после второй точки разреза, заполняются городами P_2 в порядке их следования в P_2 (также начиная после второй точки и по кругу), пропуская уже имеющиеся. Порядок обхода в P_2 с девятой позиции: 9, 3, 4, 5, 2, 1, 8, 7, 6; без городов сегмента 4, 5, 6, 7 остаётся 9, 3, 2, 1, 8. Итог: $C_1 = (2\ 1\ 8\ |\ 4\ 5\ 6\ 7\ |\ 9\ 3)$, аналогично $C_2 = (3\ 4\ 5\ |\ 1\ 8\ 7\ 6\ |\ 9\ 2)$. ОХ сохраняет относительный порядок городов, что для тура важнее абсолютных позиций.

Циклический кроссинговер (СХ, И. Оливер и др. [30]) не использует точек разреза. Позиции разбиваются на *циклы*: начиная с позиции 1, берём город $P_1[1] = 1$, смотрим, какой город стоит на этой позиции в P_2 (4), находим позицию города 4 в P_1 (позиция 4), и так далее, пока не вернёмся к началу. Получаем циклы позиций $\{1, 4\}$, $\{2, 5, 8, 9, 3\}$ и $\{6, 7\}$ (рисунок 7.2). Потомок C_1 берёт позиции первого цикла из P_1 , второго — из P_2 , третьего — снова из P_1 :

$$C_1 = (1\ 5\ 2\ 4\ 8\ 6\ 7\ 9\ 3), \quad C_2 = (4\ 2\ 3\ 1\ 5\ 7\ 6\ 8\ 9).$$

Каждый город потомка стоит на той же позиции, что и у одного из родителей. Если родители состоят из одного цикла, потомки совпадают с родителями.

Скрещивание с рекомбинацией рёбер (edge recombination, ER, Д. Уитли [31]) строит для каждого города список его соседей в обоих родителях и

	1	2	3	4	5	6	7	8	9	
P_1	1	2	3	4	5	6	7	8	9	цикл 1: позиции {1, 4}
P_2	4	5	2	1	8	7	6	9	3	цикл 2: позиции {2, 5, 8, 9, 3}
										цикл 3: позиции {6, 7}
C_1	1	5	2	4	8	6	7	9	3	C_1 : циклы 1, 3 из P_1 , цикл 2 из P_2
C_2	4	2	3	1	5	7	6	8	9	C_2 : циклы 1, 3 из P_2 , цикл 2 из P_1

Рисунок 7.2 – Циклический кроссинговер: цветом выделены циклы позиций формирует потомка, переходя в соседний город с наименьшим числом оставшихся соседей. Почти все рёбра потомка унаследованы от родителей, поэтому ER обычно эффективнее PMX, OX и CX, сохраняющих позиции или порядок, но не смежность городов.

Мутации перестановок: обмен (swap) — два города меняются местами (изменяются четыре ребра); *вставка* — город переносится на другую позицию (три ребра); *инверсия* — отрезок тура записывается в обратном порядке (для симметричной задачи изменяются только два ребра); *перемешивание* случайного отрезка.

7.5 Локальный поиск 2-opt и меметические алгоритмы

Операторы, сохраняющие позиции (CX, PMX), плохо передают потомкам главное свойство хорошего тура — набор коротких рёбер. Поэтому чистый ГА для задачи коммивояжёра сходится медленно. Существенно лучшие результаты даёт сочетание ГА с *локальным поиском*.

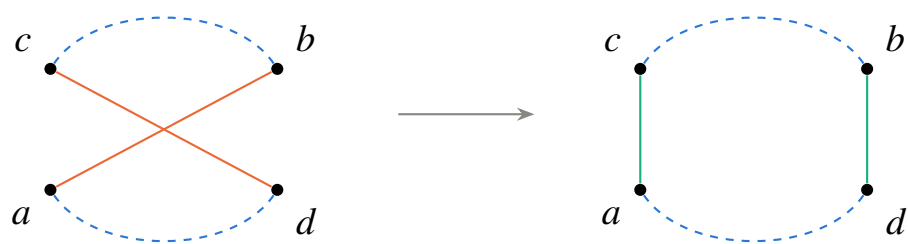
Алгоритм 2-opt (Г. Крос, 1958 [32]) удаляет из тура два ребра (a, b) и (c, d) и соединяет концы крест-накрест рёбрами (a, c) и (b, d) , что эквивалентно инверсии отрезка тура от b до c (рисунок 7.3). Замена выполняется, если она укорачивает тур:

$$\Delta = d_{ac} + d_{bd} - d_{ab} - d_{cd} < 0. \quad (7.4)$$

Перебор всех $O(n^2)$ пар рёбер повторяется, пока есть улучшающая замена. Результат — *2-оптимальный* тур без самопересечений, в среднем примерно на 5 % длиннее оптимального (более сильные эвристики — 3-opt и алгоритм Лина — Кернигана).

Меметический алгоритм (П. Москато, 1989) — ГА, в котором каждый потомок улучшается локальным поиском до ближайшего локального оптимума: ГА комбинирует фрагменты разных локальных оптимумов, а локальный поиск быстро доводит решения. На наборе eil76 меметический ГА находит оптимальный тур, а ГА с CX без локального поиска отстаёт от оптимума на 40–60 % (см. пример выполнения ЛР №7).

Реализация на Python. Тур хранится как перестановка индексов городов, популяция — массивом (N, n) ; длина всех туров популяции вычисляется одной строкой `D[pop, np.roll(pop, -1, axis=1)].sum(axis=1)`. Выигрыш 2-opt (7.4) для всех пар рёбер считается одной операцией над массивами



до: рёбра (a, b) и (c, d) пересекаются

после: рёбра (a, c) и (b, d)

Рисунок 7.3 – Ход 2-орт: рёбра (a, b) , (c, d) заменяются на (a, c) , (b, d) , и путь от b до c проходится в обратном направлении (пунктир — остальная часть тура)

индексов, и случайный тур из 76 городов доводится до 2-оптимального за несколько миллисекунд (приложение Б). PMX и OX для проверки есть в DEAP (`tools.cxPartiallyMatched`, `tools.cxOrdered`).

8 СОВРЕМЕННЫЕ МЕТОДЫ БЕЗГРАДИЕНТНОЙ ОПТИМИЗАЦИИ

Раздел используется в ЛР №8 и №9.

8.1 Выбор метода оптимизации

Классический ГА с двоичными строками сегодня редко бывает лучшим инструментом, но выросшие из него идеи лежат в основе методов, которые применяются в промышленности и в исследованиях искусственного интеллекта. Общая черта этих методов — они используют только значения целевой функции («чёрный ящик») и потому подходят там, где градиента нет: при подборе гиперпараметров, настройке симуляторов, проектировании конструкций, поиске архитектур и программ. Если градиент доступен и дешёв, градиентные методы почти всегда эффективнее. Ориентиры для выбора приведены в таблице 8.1.

Таблица 8.1 – Выбор безградиентного метода по свойствам задачи

Задача	Метод	Библиотеки Python
Непрерывная, вычисление дешёвое, до сотни переменных, много экстремумов	СМА-ES, IPOP-CMA-ES	сма, CmaEsSampler (Optuna)
Вычисление дорогое (обучение модели), десятки–сотни проб	байесовская оптимизация: TPE, гауссовы процессы	Optuna
Несколько противоречивых критериев	NSGA-II, NSGA-III, MOEA/D	py-moo, NSGAIISampler (Optuna)
Комбинаторная с ограничениями (маршруты, расписания)	целочисленное программирование, локальный поиск, меметический ГА	OR-Tools, HiGHS
Поиск формул, эвристик, программ	генетическое программирование, эволюция с LLM	PySR, FunSearch-подобные системы

8.2 Эволюционные стратегии и СМА-ES

Эволюционные стратегии (ЭС) работают с вещественными векторами и основным оператором считают мутацию $\mathbf{x}' = \mathbf{x} + \sigma \mathcal{N}(0, I)$. Главная задача ЭС — управлять распределением мутаций. Простейшая (1+1)-ЭС регулирует шаг по *правилу одной пятой* И. Рехенберга: если успешными (улучшающими) оказываются больше $1/5$ мутаций, шаг σ увеличивают, если меньше — уменьшают.

СМА-ES (covariance matrix adaptation evolution strategy, Н. Хансен [33, 34]) адаптирует не только шаг, но и форму распределения. На каждой итерации из многомерного нормального распределения с центром $\mathbf{m}$, шагом σ и ковариационной матрицей C порождаются λ точек, из них отбираются

$\mu = \lambda/2$ лучших, и по ним обновляются параметры распределения:

$$\mathbf{x}_k = \mathbf{m} + \sigma \mathbf{y}_k, \quad \mathbf{y}_k \sim \mathcal{N}(0, C), \quad \mathbf{m} \leftarrow \mathbf{m} + \sigma \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda}, \quad (8.1)$$

где $\mathbf{y}_{i:\lambda}$ — шаг i -й по качеству точки, $w_1 \geq \dots \geq w_{\mu} > 0$ — веса, $\sum w_i = 1$. Ковариационная матрица обновляется двумя слагаемыми:

$$C \leftarrow (1 - c_1 - c_{\mu}) C + c_1 \mathbf{p}_c \mathbf{p}_c^T + c_{\mu} \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^T. \quad (8.2)$$

Ранг- μ слагаемое растягивает распределение в направлениях удачных шагов текущего поколения, ранг-1 слагаемое использует *путь эволюции* $\mathbf{p}_c$ — экспоненциально сглаженную сумму последовательных шагов среднего, которая помнит устойчивое направление движения. Шаг σ регулируется вторым путём $\mathbf{p}_{\sigma}$ (*кумулятивная адаптация шага*): если последовательные шаги направлены в одну сторону, путь длиннее, чем при случайном блуждании, и шаг увеличивается, если шаги гасят друг друга — уменьшается:

$$\sigma \leftarrow \sigma \exp \left(\frac{c_{\sigma}}{d_{\sigma}} \left(\frac{\|\mathbf{p}_{\sigma}\|}{M \|\mathcal{N}(0, I)\|} - 1 \right) \right). \quad (8.3)$$

В результате матрица C постепенно приближает форму линий уровня функции в окрестности текущей точки (подобно обратному гесссиану в квазиньютоновских методах), и СМА-ES быстро движется вдоль узких изогнутых оврагов (рисунок Л8.1 в ЛР №8). Метод использует только ранги точек, поэтому не зависит от монотонных преобразований f , и инвариантен к поворотам системы координат. Все параметры, кроме начальных $\mathbf{m}$ и σ , имеют надёжные значения по умолчанию: $\lambda = 4 + \lfloor 3 \ln n \rfloor$, начальный шаг — около 0,3 ширины области.

С популяцией по умолчанию СМА-ES ведёт себя как очень эффективный *локальный* метод и на многоэкстремальных функциях застревает в ближайшем минимуме. Стандартное решение — перезапуски с увеличением популяции: *IPOP-СМА-ES* после каждого останова начинает заново из случайной точки с удвоенным λ , и большая популяция сглаживает мелкий рельеф функции.

8.3 Байесовская оптимизация и подбор гиперпараметров

Если одно вычисление целевой функции — это обучение модели, бюджет исчисляется десятками проб, и популяционные методы расходуют его слишком щедро. *Байесовская оптимизация* [35] строит по уже вычисленным точкам *суррогатную* вероятностную модель функции и выбирает следующую точку, максимизируя *функцию приобретения*, которая сочетает ожидаемое

качество и неопределённость. Для гауссова процесса с прогнозом $\mu(\mathbf{x})$ и стандартным отклонением $s(\mathbf{x})$ стандартный выбор — ожидаемое улучшение лучшего найденного значения f^* (для минимизации):

$$EI(\mathbf{x}) = M \max(0, f^* - f(\mathbf{x})) = (f^* - \mu) \Phi(z) + s \varphi(z), \quad z = \frac{f^* - \mu}{s}, \quad (8.4)$$

где Φ и φ — функция распределения и плотность стандартного нормального закона.

Древовидный парзеновский оценищик (TPE, Дж. Бергстра [36]) — алгоритм по умолчанию в библиотеке Optuna [37] — делит уже проверенные точки на «хорошие» (лучшие $\gamma \approx 10\text{--}25\%$) и остальные, оценивает их плотности $l(\mathbf{x})$ и $g(\mathbf{x})$ и выбирает точку с наибольшим отношением $l(\mathbf{x})/g(\mathbf{x})$, что эквивалентно максимизации ожидаемого улучшения. TPE естественно работает с целочисленными, категориальными и условными параметрами.

Практика подбора гиперпараметров:

- целевая функция — качество на кросс-валидации, а не на обучающей выборке;
- масштабные параметры (коэффициенты регуляризации, скорость обучения, ширина ядра) ищутся в логарифмической шкале;
- случайный поиск эффективнее перебора по сетке: обычно важны лишь немногие параметры, и сетка тратит пробы на повторение одних и тех же их значений [38];
- безнадёжные пробы останавливают досрочно по промежуточным результатам (pruning, алгоритмы Hyperband и ASHA);
- лучшее значение кросс-валидации оптимистично смещено (оно выбрано как максимум среди шумных оценок), поэтому итоговое качество измеряют на отложенной тестовой выборке или вложенной кросс-валидацией.

8.4 Многокритериальная оптимизация

В задаче с критериями $f_1, \dots, f_M$ (например, ошибка модели и её размер, стоимость и срок доставки) решение $\mathbf{x}$ *доминирует* $\mathbf{y}$ ($\mathbf{x} \prec \mathbf{y}$), если оно не хуже по всем критериям и строго лучше хотя бы по одному. Недоминируемые решения образуют *множество Парето*, их образы в пространстве критериев — *фронт Парето*. Взвешенная сумма критериев находит только выпуклую часть фронта и требует заранее знать веса, а эволюционный алгоритм приближает весь фронт за один запуск, оставляя выбор компромисса человеку.

NSGA-II (К. Деб, 2002 [39]) — самый распространённый многокритериальный ГА:

- *быстрая недоминируемая сортировка* делит популяцию на фронты $F_1, F_2, \dots$: F_1 — недоминируемые решения, F_2 — недоминируемые после удаления F_1 и т. д.;
- *расстояние скученности* решения внутри фронта — сумма нормированных расстояний между его соседями по каждому критерию (крайним точкам — бесконечность); оно поощряет равномерное покрытие фронта;
- *турнир скученности*: из двух особей побеждает та, у которой меньше номер фронта, а при равенстве — больше расстояние скученности;
- элитарное сокращение ($\mu + \lambda$): из объединения родителей и потомков берутся целые фронты, пока они помещаются, последний фронт урезается по расстоянию скученности.

Для вещественных переменных применяют *имитацию двоичного кроссинговера* (SBX): потомки $c_{1,2} = \frac{1}{2}((1 \pm \beta)p_1 + (1 \mp \beta)p_2)$, где $\beta = (2u)^{1/(\eta_c+1)}$ при $u \leq 0,5$ и $\beta = (2(1-u))^{-1/(\eta_c+1)}$ иначе, $u \sim U[0; 1]$, с последующим обменом переменными между потомками с вероятностью 0,5, и *полиномиальную мутацию* с параметром η_m . Большие η дают потомков ближе к родителям; типичные значения $\eta_c = 15$, $\eta_m = 20$.

Качество приближения фронта измеряют *гиперобъёмом* (HV) — площадью (объёмом) области, которую доминирует найденное множество относительно опорной точки (больше — лучше), и *инвертированным расстоянием поколений* (IGD) — средним расстоянием от точек истинного фронта до ближайшего найденного решения (меньше — лучше). Стандартные тестовые задачи — семейство ZDT [40]; для ZDT1 $f_1 = x_1$, $f_2 = g(1 - \sqrt{f_1/g})$, $g = 1 + 9 \sum_{i=2}^n x_i / (n - 1)$, истинный фронт $f_2 = 1 - \sqrt{f_1}$. Для четырёх и более критериев применяют NSGA-III и MOEA/D.

8.5 Эволюция программ с помощью языковых моделей

Генетическое программирование эволюционирует программы случайными заменами поддеревьев, и большинство таких изменений ломает программу или ничего не улучшает. Большие языковые модели (LLM), обученные на коде, порождают синтаксически правильные и осмысленные правки, поэтому в роли оператора мутации и скрещивания они резко повышают долю полезных потомков.

FunSearch (DeerMind, 2023 [41]) организует эволюционный цикл вокруг одной функции в программе-шаблоне:

- 1) в *базе программ* (набор «островов» для сохранения разнообразия) хранятся версии функции и их оценки;

- 2) в подсказку для LLM помещаются две–три хорошие версии, упорядоченные по возрастанию качества, и просьба написать следующую версию;
- 3) сгенерированный код выполняется *оценщиком* на наборе примеров, некорректные и медленные программы отбрасываются, остальные добавляются в базу с оценкой.

Так были найдены новые конструкции в комбинаторике (задача о cap set) и эвристики онлайн-упаковки, превосходящие классическое правило Best Fit. Развитие подхода — *AlphaEvolve* (2025 [42]), эволюционирующий целые файлы кода: он нашёл алгоритм умножения комплексных матриц 4×4 за 48 скалярных умножений и эвристику планировщика задач в центрах обработки данных Google, возвращающую в среднем 0,7 % вычислительных ресурсов.

Необходимые условия применения:

- *автоматический оценщик*: качество решения должно вычисляться программой, быстро и однозначно; без этого эволюционный поиск невозможен;
- *изоляция*: сгенерированный код считается недоверенным и выполняется в отдельном процессе с ограничением времени, памяти и разрешённых модулей, в промышленных системах — в контейнере;
- *контроль переобучения*: эвристика подбирается на обучающих примерах и проверяется на отложенных, как модель машинного обучения;
- *контроль стоимости*: число и стоимость вызовов модели фиксируются так же, как бюджет вычислений функции.

Онлайн-упаковка в контейнеры (online bin packing): предметы размеров $s_1, s_2, \dots$ приходят по одному, каждый нужно сразу и окончательно поместить в контейнер ёмкости C , минимизируя число контейнеров. Простая нижняя граница $L_1 = \lceil \sum_i s_i / C \rceil$ [25]. Классические правила: *первый подходящий* (First Fit) — самый ранний контейнер, куда предмет помещается, *наилучший подходящий* (Best Fit) — контейнер с наименьшим остатком после укладки, *наихудший подходящий* (Worst Fit) — с наибольшим остатком. Эвристику удобно записать функцией приоритета контейнеров; её и эволюционирует FunSearch.

8.6 Реализация на Python

Своя реализация CMA-ES на NumPy занимает около 50 строк (приложение Б); для сравнения используют библиотеки:

```
import cma, optuna
es = cma.CMAEvolutionStrategy(x0, 0.3 * (b - a), {"bounds": [a, b], "seed": 1})
es.optimize(f)
study = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0))
study.optimize(lambda tr: cv_score(tr.suggest_float("C", 1e-2, 1e4, log=True)), n_trials=30)

from pymoo.algorithms.moo.nsga2 import NSGA2
from pymoo.optimize import minimize
```

```
res = minimize(problem, NSGA2(pop_size=100), ("n_gen", 200), seed=1) # res.F - фронт
```

Библиотека `rufoo` [43] содержит также NSGA-III, MOEA/D, тестовые задачи и показатели качества HV и IGD. Замена сэмплера `Optuna` на `CmaEsSampler` или `NSGAIIISampler` превращает тот же код в CMA-ES или многокритериальный поиск. Модуль мини-FunSearch для ЛП №9 (окружение упаковки, проверка кода, вызов модели и эволюционный цикл) также приведён в приложении Б.

ЛАБОРАТОРНЫЙ ПРАКТИКУМ

ОБЩИЕ УКАЗАНИЯ К ВЫПОЛНЕНИЮ ЛАБОРАТОРНЫХ РАБОТ

Порядок выполнения и защиты

Перед выполнением работы изучают указанные в ней разделы теории и получают номер варианта. На защите студент демонстрирует работу программы, изменяя параметры по указанию преподавателя, и отвечает на контрольные вопросы.

Программная среда

Работы выполняются на Python 3 в Jupyter: NumPy, Matplotlib или Plotly, SciPy (подраздел 1.5, приложение А). Все алгоритмы реализуются *самостоятельно* на NumPy: это требует понимания каждой формулы и позволяет исследовать внутренние величины алгоритма. Готовые библиотеки (scikit-learn, PyTorch, MiniSom, DEAP, PyGAD) служат для сравнения, расхождение с ними нужно объяснить. MATLAB и JavaScript допускаются по желанию студента.

Требования к программе

- параметры алгоритма задаются в интерфейсе (ipywidgets) или в одном блоке в начале программы;
- процесс обучения или эволюции отображается динамически, с заданным периодом перерисовки;
- генератор случайных чисел создаётся с фиксированным seed (`np.random.default_rng(seed)`);
- для стохастических алгоритмов предусматривается серия из M независимых запусков со сводной статистикой: доля успехов, медиана и худший результат, сравнение с эталоном (подраздел 6.7).

Содержание отчёта

Отчёт по СТП БГУИР содержит: титульный лист, цель, постановку задачи и вариант, краткую теорию, описание реализации, результаты с графиками, таблицами и их анализом, выводы, опирающиеся на числа, ответы на контрольные вопросы и листинг программы в приложении.

ЛАБОРАТОРНАЯ РАБОТА №1

КЛАССИФИКАЦИЯ С ПОМОЩЬЮ ПЕРСЕПТРОНА

Цель работы

Изучить модель персептрона и правило его обучения, исследовать возможности и ограничения однослойного персептрона на задачах бинарной и многоклассовой классификации.

Теоретические сведения

Подразделы 1.3 и 2.1–2.3 теоретической части: модель нейрона с пороговой функцией активации, правило обучения (2.2), теорема сходимости, проблема XOR, кодирование классов выходами нескольких нейронов.

Постановка задачи

Исходные данные — четыре точки плоскости (столбцы матрицы P) и их классы T :

$$P = \begin{pmatrix} -0,5 & -0,5 & 0,3 & -0,1 \\ -0,5 & 0,5 & -0,5 & 1 \end{pmatrix}, \quad T = (0 \ 0 \ 1 \ 1).$$

Код обучения на NumPy приведён в подразделе 2.6; для визуализации точки классов выводятся функцией `plt.scatter(*P, c=T)`, а после каждой эпохи на график добавляется текущая разделяющая прямая $x_2 = -(w_1x_1 + b)/w_2$ (`plt.plot`).

Задания:

1. Построить персептрон, разделяющий на два класса заданные входные векторы.
2. На векторах $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$ с классами $0, 1, 1, 0$ продемонстрировать проблему исключающего ИЛИ — невозможность линейного разделения.
3. Построить персептрон из двух нейронов, разбивающий входные векторы на четыре класса: четыре группы по 20–30 точек вокруг центров в разных квадрантах (например, $(\pm 2; \pm 2)$, разброс 0,3), классы кодируются парами выходов 00, 01, 10, 11.

Порядок выполнения работы

1. Реализовать персептрон на NumPy (подраздел 2.6).
2. Выполнить задание 1 с перерисовкой разделяющей прямой после каждой эпохи; зафиксировать итоговые веса и число эпох.
3. Выполнить задание 2 с ограничением в 50 эпох, построить график числа ошибок по эпохам и объяснить результат.
4. Выполнить задание 3: обучить оба нейрона, построить две разделяющие прямые и карту решений — раскраску плоскости по коду (y_1, y_2) ; вычислить долю верно классифицированных точек.

Пример выполнения

Результаты для заданных векторов приведены на рисунке Л1.1. В задании 1 персептрон с нулевыми начальными весами и $\eta = 1$ разделил классы за 3 эпохи (в третьей эпохе ошибок нет): $\mathbf{w} = (2; 0,5)$, $b = 0$. В задании 2 после двух эпох персептрон ошибается на всех четырёх точках XOR: коррекции компенсируют друг друга, к концу каждой эпохи веса возвращаются к значениям $\mathbf{w} = (-1; 0)$, $b = 0$, и обучение не сходится. В задании 3 два нейрона разделили 100 точек на четыре класса за 2 эпохи без ошибок: каждый нейрон отвечает за один разряд кода и строит одну из двух прямых.

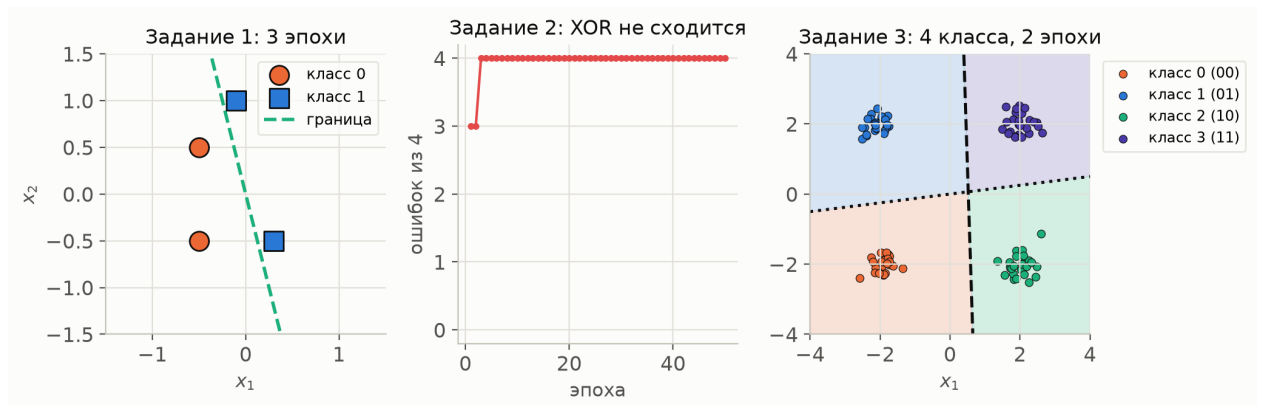


Рисунок Л1.1 – Персептрон: разделение двух классов, число ошибок на задаче XOR, карта решений персептрона из двух нейронов

Контрольные вопросы

1. Опишите модель персептрона Розенблатта. Какую функцию активации он использует?
2. Какое геометрическое место точек задаёт уравнение $\mathbf{w}^T \mathbf{x} + b = 0$? Как расположен вектор весов относительно разделяющей прямой и за что отвечает смещение?
3. Запишите правило обучения персептрона. Как изменяются веса при ошибке каждого знака?
4. Сформулируйте теорему сходимости персептрона. Единственна ли найденная граница?
5. Какие множества называются линейно разделимыми? Докажите, что функция XOR линейно неразделима. Как решить эту задачу нейронной сетью?
6. Сколько классов может различить слой из m персептронов? Какие требования предъявляются к кодированию классов?

ЛАБОРАТОРНАЯ РАБОТА №2

ПРОГНОЗИРОВАНИЕ ЗНАЧЕНИЙ ПРОЦЕССА С ПОМОЩЬЮ ЛИНЕЙНОЙ НЕЙРОННОЙ СЕТИ

Цель работы

Изучить линейную нейронную сеть и метод наименьших квадратов для настройки её весов; построить линейную сеть, прогнозирующую следующее значение дискретного сигнала по k предыдущим значениям, и исследовать влияние шага дискретизации и размера окна на точность прогноза.

Теоретические сведения

Подразделы 2.4 и 2.5 теоретической части: линейная сеть ADALINE, решение через псевдообратную матрицу (2.4), линия задержки и матрица входов (2.6), рекуррентное соотношение для гармонического сигнала (2.7), теорема Котельникова.

Постановка задачи

Сигнал $x(t) = f(t)$ задан на отрезке $t \in [T_1; T_2]$ и дискретизирован с шагом $0 < \Delta t \leq 0,05$ с. Необходимо построить линейную нейронную сеть, прогнозирующую будущее значение сигнала по $3 \leq k \leq 7$ предыдущим значениям, получить вектор прогнозов y моделированием сети и построить: график сигнала $y(t)$, на котором символом «*» отмечены точки эталонного сигнала $x(t)$, и график ошибки $e = y - x$. Варьируя Δt и k , минимизировать ошибку. Варианты заданий приведены в таблице Л2.1: для синусоидальных сигналов $t \in [0; 5]$, для косинусоидальных $t \in [-3; 3]$.

Таблица Л2.1 – Варианты заданий к лабораторной работе №2

№	$x(t)$	№	$x(t)$	№	$x(t)$	№	$x(t)$
1	$\sin(\pi t)$	5	$\sin(2\pi t)$	9	$\sin(3\pi t)$	13	$\sin(4\pi t)$
2	$\cos(\pi t)$	6	$\cos(2\pi t)$	10	$\cos(3\pi t)$	14	$\cos(4\pi t)$
3	$\sin(1,5\pi t)$	7	$\sin(2,5\pi t)$	11	$\sin(3,5\pi t)$		
4	$\cos(1,5\pi t)$	8	$\cos(2,5\pi t)$	12	$\cos(3,5\pi t)$		

Порядок выполнения работы

1. Для заданного отрезка и шага Δt сформировать вектор времени t и вектор значений сигнала x .
2. Сформировать матрицу входов P размера $k \times N$ (2.6): i -й столбец содержит k значений сигнала, предшествующих x_i , начальные значения линии задержки нулевые. Целевой вектор — x .
3. Найти веса линейной сети с k входами и одним выходом по МНК (2.4): `np.linalg.lstsq` или `np.linalg.pinv` с добавлением строки единиц для смещения (подраздел 2.6); для контроля сравнить с `sklearn.linear_model.LinearRegression`.

4. Вычислить прогноз сети $y = WP + b$.
5. Построить графики $x(t)$, $y(t)$ и ошибки $e(t)$ в отдельных окнах; вычислить MSE и максимальную ошибку.
6. Исследовать зависимость ошибки от $k \in [3; 7]$ и $\Delta t \in (0; 0,05]$; сравнить найденные веса с теоретическими значениями из соотношения (2.7).
7. Дополнительно: добавить к сигналу нормальный шум ($\sigma = 0,05$) и повторить исследование зависимости ошибки от k , оценивая ошибку относительно незашумлённого сигнала.

Пример выполнения

Вариант 3: $x(t) = \sin(1,5\pi t)$, $t \in [0; 5]$, $\Delta t = 0,05$ с ($N = 101$ отсчёт), $k = 5$. МНК дал веса $\mathbf{w} = (1,934; -0,990; -0,001; 0,010; -0,011)$ и смещение $b = 0,002$. Первые два веса близки к теоретическим коэффициентам $2 \cos(\omega \Delta t) = 1,945$ и -1 из соотношения (2.7), остальные — к нулю. Прогноз практически совпадает с сигналом (рисунок Л2.1), $MSE = 5,3 \cdot 10^{-4}$. Почти вся ошибка сосредоточена в начале ряда, пока линия задержки заполнена нулями: отсчёт $x(\Delta t) = 0,233$ не может быть предсказан по нулевой предыстории.

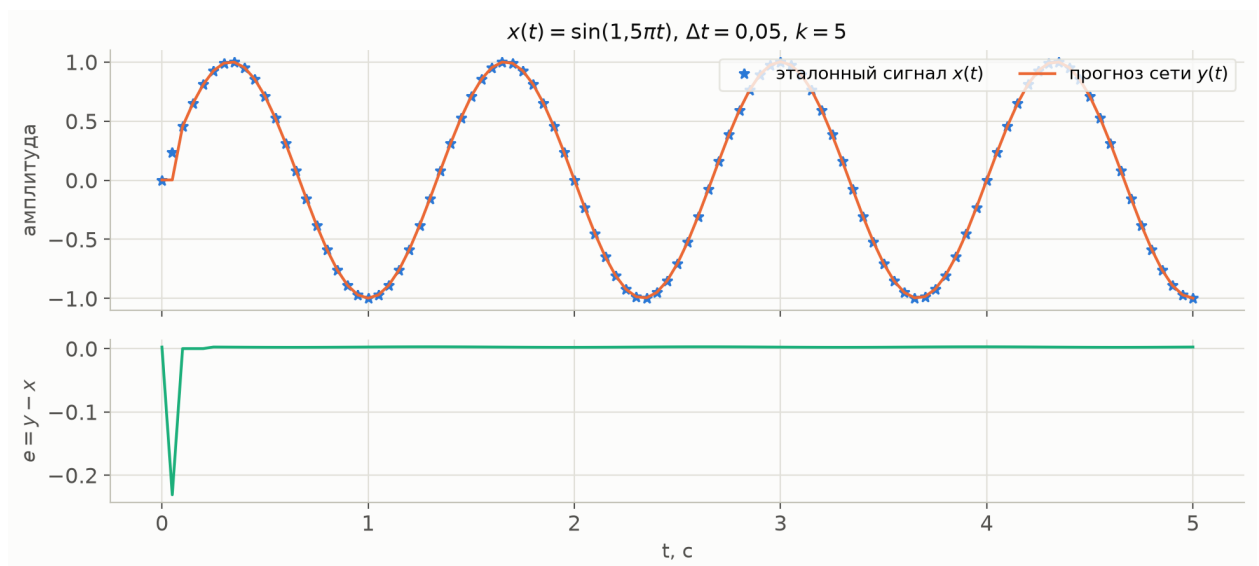


Рисунок Л2.1 – Прогноз линейной сети для варианта 3 ($\Delta t = 0,05$, $k = 5$) и ошибка прогноза

Исследование параметров (рисунок Л2.2) показывает, что для незашумлённой синусоиды ошибка не зависит от k при $k \geq 2$ и совпадает с оценкой $\sin^2(\omega \Delta t)/N$: уменьшение шага с 0,05 до 0,005 снижает MSE в тысячу раз. Для зашумлённого сигнала картина иная: при $k = 2$ коэффициенты $2 \cos(\omega \Delta t) \approx 2$ и -1 усиливают шум, и ошибка до четырёх раз превышает его дисперсию $\sigma^2 = 0,0025$. С ростом k прогноз усредняет шум по большему числу отсчётов, и ошибка уменьшается в 2,5–5 раз; при $\Delta t = 0,05$ насыщение наступает около $k = 6$ –7.

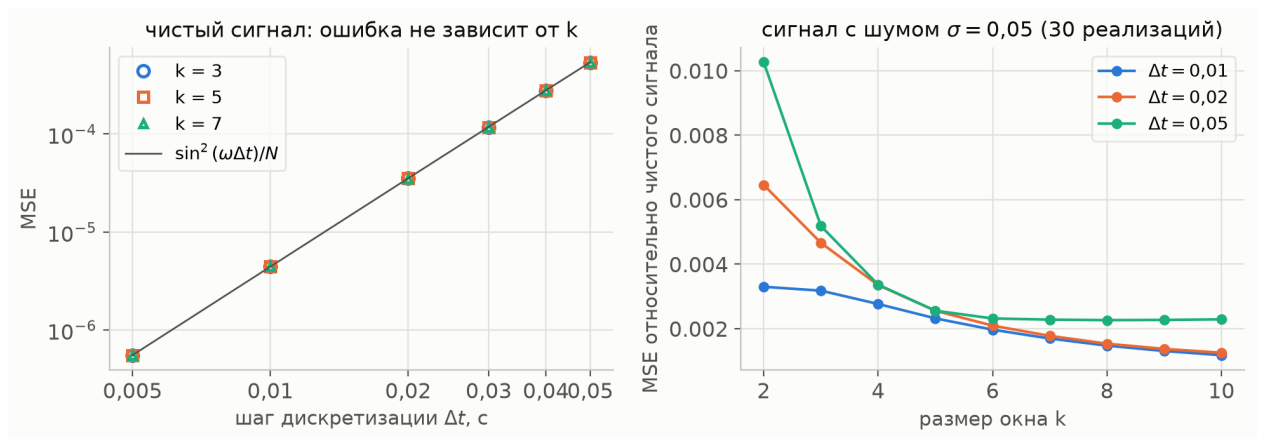


Рисунок Л2.2 – Зависимость ошибки прогноза от шага дискретизации (чистый сигнал) и от размера окна (сигнал с шумом)

Контрольные вопросы

1. Чем линейная сеть отличается от персептрона? Какую функцию ошибки она минимизирует?
2. Запишите правило Уидроу — Хоффа. Чем ограничен шаг обучения?
3. Выведите формулу весов линейной сети через псевдообратную матрицу. Почему `np.linalg.lstsq` предпочтительнее явного обращения матрицы PP^T ?
4. Как формируется матрица входов для прогнозирования временного ряда? Что такое линия задержки?
5. Почему для прогноза синусоиды достаточно двух предыдущих значений? Выведите соотношение (2.7).
6. Где на графике ошибки сосредоточена основная ошибка прогноза и почему?
7. Как шаг дискретизации влияет на точность прогноза? Сформулируйте теорему Котельникова.
8. Как влияет размер окна k на прогноз зашумлённого сигнала?
9. Какие сигналы невозможно точно прогнозировать линейной сетью?

ЛАБОРАТОРНАЯ РАБОТА №3

АППРОКСИМАЦИЯ ФУНКЦИИ С ПОМОЩЬЮ ДВУХСЛОЙНОЙ НЕЙРОННОЙ СЕТИ

Цель работы

Изучить архитектуру двухслойной нейронной сети (многослойного персептрона), принципы её обучения методом обратного распространения ошибки и исследовать точность аппроксимации нелинейной функции в зависимости от числа нейронов скрытого слоя.

Теоретические сведения

Раздел 3 теоретической части: сеть 1–S–1 (3.1), теорема об универсальной аппроксимации, алгоритм обратного распространения ошибки, градиентный спуск и условие устойчивости (3.4), ускоренные алгоритмы, критерии останова, недообучение и переобучение, реализация на Python (подраздел 3.6).

Постановка задачи

Дана функция $y = f(t)$ (таблица Л3.1). Необходимо аппроксимировать её по вектору значений y для $t = t_0 : 0,1 : t_1$ (`np.arange(t0, t1 + 0.05, 0.1)`) двухслойной сетью 1–S–1 с функцией активации $\tanh$ в скрытом слое и линейной в выходном. Сеть обучается пакетным градиентным спуском по методу обратного распространения ошибки с параметрами `show = 50` (период перерисовки), `lr = 0.05` (коэффициент обучения), `epochs = 600` (число эпох), `goal = 1e-2` (целевая ошибка); в исходной постановке для MATLAB это функции `newff` и `traingd`. Построить графики функции и выхода сети на одной координатной плоскости и проанализировать точность аппроксимации в зависимости от числа нейронов скрытого слоя.

Таблица Л3.1 – Варианты заданий к лабораторной работе №3

№	$f(t)$	t	№	$f(t)$	t
1	$(1,5t + 0,9) \sin(t + 1,1)$	$[-5; 5]$	8	$(1,5t + 0,9) \cos(t + 1,1)$	$[-7; 7]$
2	$(1,3t + 1,9) \cos(1,1t - 1,5)$	$[-6; 6]$	9	$(0,3t + 1,1) \sin(1,8t - 0,5)$	$[-6; 6]$
3	$(t + 1,3) \sin(0,5t + 1)$	$[-7; 7]$	10	$(0,5t + 1,9) \cos(0,7t - 1,3)$	$[-5; 5]$
4	$(1,1t - 1,7) \cos(t + 1,5)$	$[-9; 9]$	11	$(2,5t - 0,9) \sin(1,1t + 1,7)$	$[-6; 6]$
5	$(2,5t + 1,7) \sin(1,1t + 0,7)$	$[-10; 10]$	12	$(2,1t + 0,8) \cos(1,3t - 0,9)$	$[-7; 7]$
6	$(0,5t - 1,4) \cos(0,5t + 1,1)$	$[-9; 9]$	13	$(0,9t + 1,5) \sin(1,1t + 1)$	$[-8; 8]$
7	$(1,7t + 1,5) \sin(0,7t - 1,4)$	$[-8; 8]$	14	$(1,7t - 1,9) \cos(1,4t - 1,7)$	$[-9; 9]$

Требования к программе

Через интерфейс задаются число нейронов скрытого слоя (ползунок от 1 до 30), коэффициент обучения $\eta \in [0,01; 0,5]$ и целевая ошибка $goal \in [10^{-4}; 10^{-1}]$. Точки функции и кривая аппроксимации выводятся на одной плоскости; кривая перерисовывается каждые `show` эпох, синхронно

строится график ошибки по эпохам. Скорость анимации задаётся ползунком «Задержка (мс)», процесс управляется кнопками «Старт», «Пауза», «Сброс». В Jupyter интерфейс собирается из ipywidgets (IntSlider, FloatText, Button) и графика plotly.graph_objects.FigureWidget; допустимо и отдельное веб-приложение.

Порядок выполнения работы

1. Сформировать вектор $t = t_0 : 0,1 : t_1$ и вектор эталонных значений $y = f(t)$.
2. Создать сеть с 10 нейронами и параметрами обучения по заданию, обучить её с мультипликацией. Проанализировать форму итоговой кривой и протокол обучения.
3. Провести серию экспериментов при $S = 1, 2, 5, 10, 20, 30$, для каждого S — не менее 5–10 запусков с разной начальной инициализацией. Занести в таблицу медиану MSE, R^2 и факт достижения цели.
4. Для оценки качества между узлами обучающей сетки вычислить ошибку на контрольной сетке с шагом 0,01.
5. Дополнительно: сравнить градиентный спуск со спуском с моментом и адаптивным шагом, методом Левенберга — Марквардта (scipy.optimize.least_squares) и MLPRegressor; исследовать влияние lr и найти границу устойчивости; добавить к значениям функции шум и показать переобучение избыточной сети и эффект раннего останова.

Пример выполнения

Вариант 3: $f(t) = (t + 1,3) \sin(0,5t + 1)$, $t \in [-7; 7]$, 141 точка. Сеть с $S = 10$ за 600 эпох градиентного спуска уменьшила MSE со 145 до 0,045 ($R^2 = 0,995$); за первые 50 эпох кривая грубо повторяет форму функции, далее медленно «дотягивает» экстремумы, цель 10^{-2} не достигнута. Результаты серии — в таблице ЛЗ.2 и на рисунке ЛЗ.1.

Таблица ЛЗ.2 – Серия экспериментов (медиана по 10 инициализациям):
ГС — градиентный спуск, ГС+АШ — спуск с моментом и адаптивным шагом, ЛМ — метод Левенберга — Марквардта

S	Параметров	ГС, 600 эпох		Эпох до цели 10^{-2}		Предельная MSE (ЛМ)
		MSE	R^2	ГС+АШ	ЛМ	
1	4	4,08	0,53	—	—	1,5
2	7	2,18	0,75	—	—	0,68
5	16	0,048	0,995	145	5	$7 \cdot 10^{-9}$
10	31	0,049	0,994	117	3	$2,5 \cdot 10^{-9}$
20	61	0,067	0,992	196	1	$1 \cdot 10^{-9}$
30	91	0,096	0,989	386	1	$3 \cdot 10^{-10}$

При $S = 1, 2$ — недообучение при любом алгоритме; цель достижима начиная с $S = 3$ (по нейрону на изгиб функции), при $S \geq 5$ ошибка снижается до 10^{-8} . Рябь у сетей с $S = 20, 30$ — недоученность, а не переобучение:

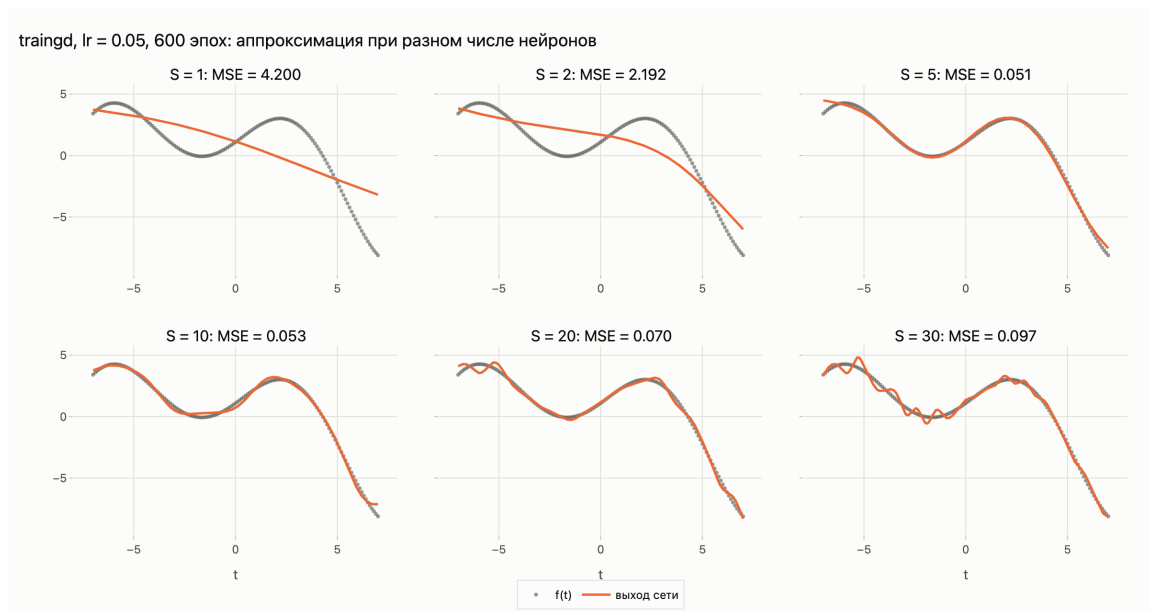


Рисунок Л3.1 – Аппроксимация при разном числе нейронов (градиентный спуск, $\eta = 0,05$, 600 эпох)

ошибка на контрольной сетке равна ошибке на обучающей. Без ограничения в 600 эпох градиентный спуск достигает цели за 2360–3960 эпох, спуск с адаптивным шагом — за сотни, метод Левенберга — Марквардта — за единицы эпох. Граница устойчивости градиентного спуска: при $\eta = 0,2$ обучение сходится, при $\eta \geq 0,25$ расходится, что согласуется с оценкой $2/\lambda_{\max} \approx 0,19$ по гессиану. На данных с шумом $\sigma = 0,7$ сеть с $S = 30$ переобучается (рисунок 3.4), лучший результат даёт $S = 3$, а ранний останов уменьшает ошибку избыточной сети в 11 раз.

Контрольные вопросы

1. Какова математическая и алгоритмическая роль нелинейной функции активации в скрытом слое?
2. Опишите смысл алгоритма обратного распространения ошибки. Как вычисляются градиенты по весам скрытого и выходного слоёв?
3. Как коэффициент обучения влияет на сходимость? Чем определяется предельное значение lr ?
4. Сформулируйте теорему об универсальной аппроксимации (Цыбенко, Колмогорова). Что она говорит о числе слоёв и чего не гарантирует?
5. Каковы визуальные признаки недообучения и переобучения на графике аппроксимации? Как отличить переобучение от недоученности?
6. Объясните назначение параметров `show`, `goal`, `epochs`, `min_grad`.
7. Зачем нормировать входы и цели (`MinMaxScaler`)? Как инициализировать веса (Нгуен — Уидроу, Глоро)?

ЛАБОРАТОРНАЯ РАБОТА №4

КЛАСТЕРИЗАЦИЯ С ПОМОЩЬЮ НЕЙРОННОЙ СЕТИ КОХОНЕНА

Цель работы

Изучить архитектуру самоорганизующихся карт Кохонена, алгоритмы конкурентного обучения без учителя и получить навыки кластеризации двумерных данных с визуальным анализом топологии сети.

Теоретические сведения

Раздел 4 теоретической части: конкурентное обучение и правило WTA (4.1), правило Кохонена (4.2), топологии gridtop и hextop, функция окрестности, фазы обучения двухфазное правило обучения, показатели качества QE , TE и U-матрица, реализация на Python (подраздел 4.4).

Постановка задачи

Построить нейронную сеть Кохонена, разделяющую множество точек декартовой плоскости на заданное число кластеров. Выборка генерируется программно: вокруг каждого центра из таблицы Л4.1 случайно располагаются 30–50 точек с разбросом (среднеквадратичным отклонением) 0,10–0,15.

Таблица Л4.1 – Варианты заданий к лабораторной работе №4

№	Центры кластеров ($x; y$)	№	Центры кластеров ($x; y$)
1	(-0,5; 0,5), (0,5; 0,5), (0; -0,6)	6	(-0,6; 0,6), (0,6; 0,6), (0; 0), (0; -0,7)
2	(-0,6; -0,4), (0,6; -0,4), (0; 0,7)	7	(-0,5; -0,5), (0,5; 0,5), (-0,7; 0,6)
3	(±0,5; 0,5), (±0,5; -0,5)	8	(±0,4; 0,4), (0; -0,2), (0,6; -0,6)
4	(±0,7; 0), (0; ±0,7)	9	(-0,6; 0), (0; 0,6), (0,6; -0,6)
5	(-0,3; 0,3), (0,4; -0,4), (-0,6; -0,6)	10	(±0,5; 0), (-0,5; 0,6), (0,5; -0,6)

Требования к программе

Через интерфейс задаются топология решётки (hextop или gridtop), её размер (например, 1×4 , 2×2 , 3×3 , 5×5) и число эпох обучения. На плоскости $[-1; 1]^2$ одновременно отображаются цветные маркеры точек выборки и текущие веса нейронов, соединённые линиями решётки. Положение нейронов и связей перерисовывается каждые 10–20 эпох, так что видно, как первоначально хаотичная сетка растягивается и покрывает облака точек. Скорость анимации регулируется ползунком «Задержка (мс)», процесс управляется кнопками «Старт», «Пауза», «Сброс» (в Jupyter — `ipywidgets.DropDown`, `IntSlider`, `Button` и `FigureWidget` Plotly).

Порядок выполнения работы

1. Программно сформировать облака точек для кластеров варианта (`rng.normal(center, sigma, (n, 2))`) и отобразить их на плоскости.

2. Реализовать карту Кохонена на NumPy с числом нейронов, равным числу кластеров, и обучить её с мультипликацией. Зафиксировать итоговые положения нейронов и сравнить их с центрами кластеров.

3. Провести серию экспериментов: сделать решётку избыточной (5×5) и сравнить топологии hextop и gridtop. Проанализировать, как веса распределяются внутри плотных облаков данных.

4. Для каждой конфигурации вычислить QE , TE и число мёртвых нейронов (среднее по 5–10 запускам), построить карту попаданий и U-матрицу для карты 5×5.

5. Дополнительно: сравнить гауссову окрестность с двухфазным правилом, в котором радиус не сужается ниже 1; сравнить результат с MiniSom и KMeans; оценить, сколько эпох нужно для упорядочивания карты.

Пример выполнения

Вариант 3: четыре кластера с центрами ($\pm 0,5$; $\pm 0,5$), сгенерировано 47, 31, 48 и 46 точек (всего 172). Карта 2×2 hextop с гауссовой окрестностью за 200 эпох поместила каждый нейрон в центр своего облака с отклонением 0,005–0,012 от среднего кластера; $QE = 0,157$ совпадает с результатом k -средних, разбиение совпадает с истинным ($ARI = 1$). Результаты серии — в таблице Л4.2.

Таблица Л4.2 – Показатели качества карт (200 эпох, среднее по 10 запускам)

Конфигурация	Нейронов	QE	TE	Мёртвых нейронов	ARI
1×4	4	0,157	0,241	0	1,000
2×2	4	0,157	0,000	0	1,000
3×3 gridtop	9	0,135	0,017	1,0	1,000
3×3 hextop	9	0,134	0,000	0	1,000
5×5 gridtop	25	0,077	0,108	3,7	1,000
5×5 hextop	25	0,076	0,022	2,9	1,000

Избыточная карта 5×5 (рисунок Л4.1) за 30–50 эпох упорядочивается в ровную решётку, затем растягивается к облакам. Число нейронов на облако пропорционально числу его точек: около 6 нейронов на облака из 46–48 точек и 3,4 на облако из 31 точки. Три–четыре нейрона остаются между облаками без попаданий и образуют на U-матрице границы кластеров (рисунок Л4.2). Гексагональная решётка сохраняет топологию в 5 раз лучше прямоугольной ($TE = 0,022$ против 0,108) при одинаковой квантизационной ошибке. Двухфазное правило с несужающейся окрестностью (радиус 1) стягивает центроиды карты 2×2 к центру плоскости ($QE = 0,38–0,46$, рисунок 4.2). Для упорядочивания карте достаточно 20–50 эпох, после 100 эпох QE почти не меняется.

Контрольные вопросы

1. Каков биологический аналог структурной организации самоорганизующихся карт Кохонена?

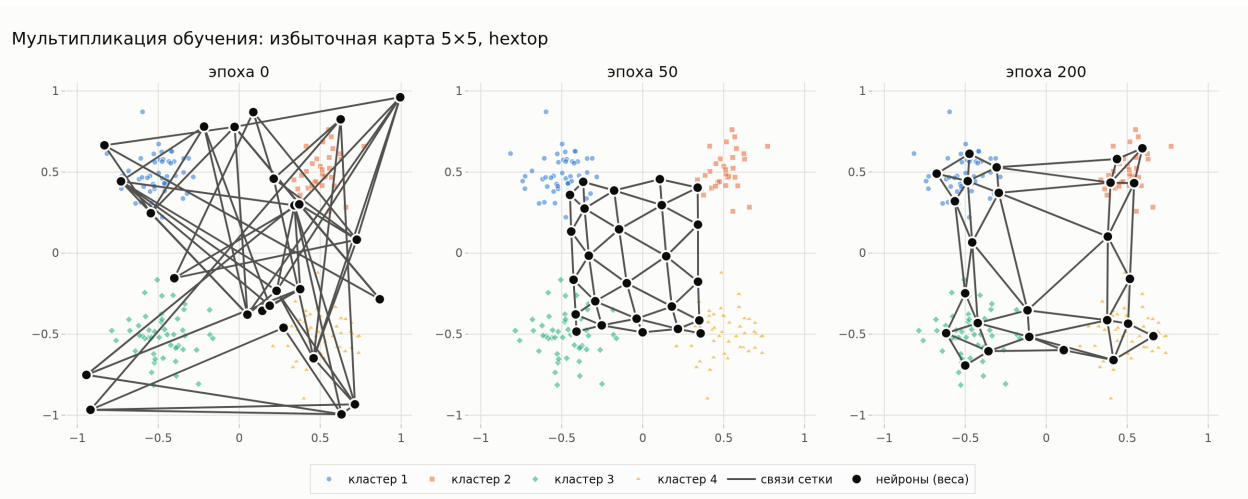


Рисунок Л4.1 – Мультипликация обучения избыточной карты 5×5 hextop: эпохи 0, 50 и 200

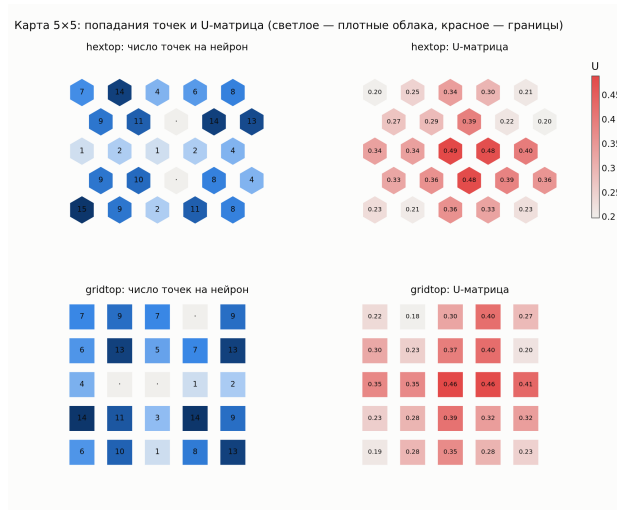


Рисунок Л4.2 – Карта 5×5: число попаданий точек на нейрон и U-матрица

2. Объясните математический принцип стратегии «победитель получает всё». Какова связь с методом k -средних?
3. Как функция окрестности h_{ci} и топология решётки (hextop/gridtop) влияют на изменение весов соседних нейронов?
4. Что происходит с картой при избыточном числе нейронов? Как в этом случае выделить кластеры?
5. Почему сеть Кохонена обучается без учителя? Откуда она извлекает информацию о структуре данных?
6. Что такое мёртвые нейроны? Что показывают квантизационная и топографическая ошибки, U-матрица?
7. Зачем радиус окрестности уменьшают в ходе обучения? Что будет, если он не сужается до нуля?

ЛАБОРАТОРНАЯ РАБОТА №5

ПОИСК ЭКСТРЕМУМА ФУНКЦИИ ОДНОЙ ПЕРЕМЕННОЙ С ПОМОЩЬЮ ГЕНЕТИЧЕСКОГО АЛГОРИТМА С БИНАРНЫМ ПРЕДСТАВЛЕНИЕМ ОСОБЕЙ

Цель работы

Изучить основные понятия теории генетических алгоритмов, реализовать классический простой ГА с представлением решений в форме бинарных строк, классическими операторами кроссинговера и мутации и исследовать влияние параметров ГА на точность и надёжность поиска.

Теоретические сведения

Раздел 5 и подразделы 6.1–6.4 теоретической части: основные понятия ГА, простой ГА и пример его работы, двоичное кодирование вещественного числа (5.2) и выбор длины хромосомы (5.3), код Грея, пропорциональная селекция, одноточечный кроссинговер и побитовая мутация, элитизм.

Постановка задачи

Дана функция одной переменной $y = f(t)$ (таблица Л5.1). Найти её экстремум на отрезке $t \in [t_0; t_1]$ генетическим алгоритмом с бинарным представлением особей и графически проиллюстрировать динамику поиска.

Таблица Л5.1 – Варианты заданий к лабораторной работе №5

№	Задача	t	№	Задача	t
1	$\max (1,5t + 0,9) \sin(\pi t + 1,1)$	$[0; 5]$	5	$\min (2,5t + 1,7) \sin(1,1\pi t + 0,7)$	$[0; 10]$
2	$\min (1,3t + 1,9) \cos(1,1\pi t - 1,5)$	$[-6; 6]$	6	$\min (0,5t - 1,4) \cos(0,5\pi t + 1,1)$	$[-9; 9]$
3	$\max (t + 1,3) \sin(0,5\pi t + 1)$	$[0; 7]$	7	$\max (1,7t + 1,5) \sin(0,7\pi t - 1,4)$	$[0; 8]$
4	$\max (1,1t - 1,7) \cos(\pi t + 1,5)$	$[-9; 9]$	8	$\max (0,7t - 1,7) \cos(0,5\pi t + 1,5)$	$[-5; 5]$

Требования к программе

Исходные данные программы: размер популяции, длина хромосомы (число битов), вероятности кроссинговера и мутации, максимальное число поколений. В процессе поиска отображаются: график функции и положение на нём особей текущей популяции; графики лучшего и среднего значений приспособленности по поколениям; вещественное значение лучшей особи текущей популяции и значение функции для неё.

Порядок выполнения работы

1. Построить график функции и найти эталонный экстремум численно: перебор по мелкой сетке `np.linspace` и уточнение `scipy.optimize.minimize_scalar(method='bounded')`.

2. Выбрать длину хромосомы по требуемой точности (5.3), реализовать кодирование и декодирование (5.2).

3. Реализовать простой ГА на NumPy (подраздел 6.8): случайную начальную популяцию, пропорциональную селекцию (для функций со знакопеременными значениями и для поиска минимума — со сдвигом приспособленности), одноточечный кроссинговер с вероятностью P_c , побитовую мутацию с вероятностью P_m , элитизм.

4. Выполнить основной запуск с визуализацией динамики популяции.

5. Оценить надёжность: выполнить не менее 30 независимых запусков и определить долю успешных (найден экстремум с заданной точностью, например $|t - t^*| < 10^{-3}$) и медиану числа поколений до успеха.

6. Исследовать влияние размера популяции, длины хромосомы, P_c , P_m и элитизма, меняя по одному параметру.

Пример выполнения

Вариант 3: максимум $f(t) = (t + 1,3) \sin(0,5\pi t + 1)$ на отрезке $[0; 7]$. Функция имеет локальный максимум $f(0,572) = 1,772$ и глобальный, найденный перебором по сетке из 700 001 точки с уточнением `scipy.optimize.minimize_scalar`: $t^* = 4,433776$, $f(t^*) = 5,698757$. Параметры ГА: $N = 60$, $L = 20$ (шаг сетки $\Delta = 6,7 \cdot 10^{-6}$), $P_c = 0,85$, $P_m = 0,01$, 200 поколений, элитизм; успех — $|t - t^*| < 10^{-3}$. Найдено $t = 4,433777$ с погрешностью $1,6 \cdot 10^{-6}$ на уровне точности 20-битного кода; окрестность 10^{-3} достигнута на 10-м поколении (660 вычислений функции). К 10-му поколению популяция покидает локальный максимум и собирается у глобального (рисунок Л5.1); средняя приспособленность отстаёт от лучшей из-за мутаций (рисунок Л5.2).

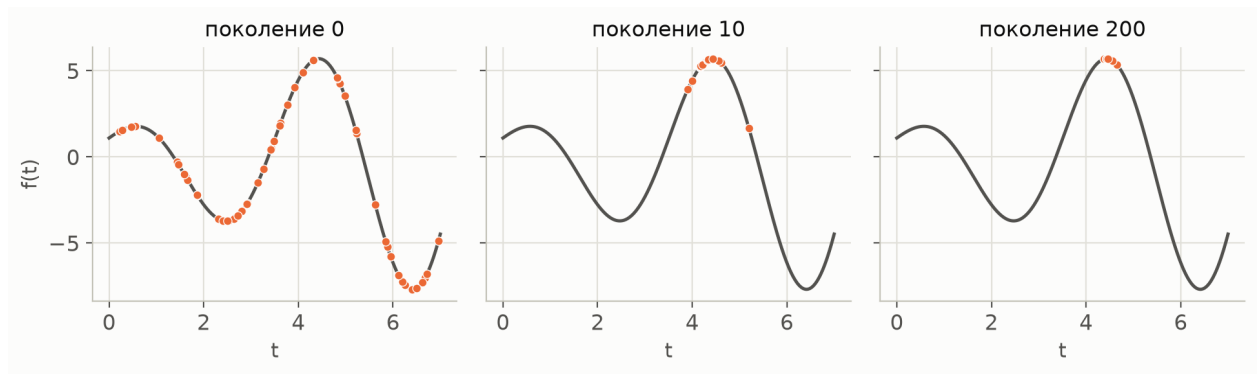


Рисунок Л5.1 – Популяция простого ГА на графике функции варианта 3: поколения 0, 10 и 200

Из 50 независимых запусков 47 успешны, медиана — 13 поколений до попадания. Все три неудачи остановились в одной точке $t = 4,375$ ($f = 5,674$) с кодом 10011111111111111111: ближайшие лучшие решения начинаются с 10100..., и переход к ним требует одновременно изменить несколько старших битов — это хэмминговский обрыв (подраздел 5.6). С кодом Грея те же запуски успешны все 50 из 50 (медиана 11 поколений), а при $N = 20$ — 30 из 30 против 12 из 30 у двоичного кода. Влияние параметров показано в таблице Л5.2.



Рисунок Л5.2 – Слева — лучшее и среднее значения функции в основном запуске (пунктир — f^*); справа — лучшая особь поколения с элитизмом и без него (среднее по 30 запускам, $P_m = 0,05$)

Таблица Л5.2 – Влияние параметров простого ГА: доля успешных запусков, худшее f и медиана поколений до успеха (30 запусков на значение, двоичный код)

	Знач.	Успех, %	Худшее f	Покол.		Знач.	Успех, %	Худшее f	Покол.
N	10	20	5,6741	49	P_m	0	47	5,6740	8,5
	20	40	5,6741	20		0,001	73	5,6741	13
	60	93	5,6741	14		0,01	93	5,6741	14
	100	100	5,6988	14,5		0,05	100	5,6988	11
L	8	0	5,6975	—	P_c	0,2	100	5,6988	17,5
	12	100	5,6988	14		0	93	5,6741	35
	20	93	5,6741	14		0,5	97	5,6741	15
	24	97	5,6741	18		0,95	100	5,6988	17,5

Размер популяции — главный фактор надёжности. При $L = 8$ шаг сетки $0,027$ больше допуска, и попадание в окрестность 10^{-3} невозможно в принципе. Без мутации популяция вырождается, и больше половины запусков неудачны; мутация $P_m \geq 0,05$ помогает выйти из ловушки у хэммингового обрыва. Кроссинговер ускоряет поиск более чем вдвое (35 и 14 поколений при $P_c = 0$ и $0,85$). Без элитизма найденный максимум теряется из-за мутаций: лучшая особь последнего поколения в среднем $5,6931$ против $5,6988$.

Контрольные вопросы

1. Поясните понятия «эволюционные вычисления» и «эволюционный поиск». Перечислите основные парадигмы эволюционных вычислений.
2. Что такое генетический алгоритм? Назовите его отличительные особенности.
3. Какие компоненты нужно задать для определения ГА? Опишите простой ГА.
4. Поясните понятия «приспособленность особи» и «приспособленность популяции». Как искать минимум функции и работать с отрицательными значениями?

5. Опишите классические операторы репродукции, кроссинговера и мутации. Приведите примеры.

6. Выполните вручную одно поколение простого ГА для поиска максимума $f(x) = -(x - 2)^2 + 5$ на целых $x \in [0; 4]$.

7. Как представить вещественное число двоичной строкой? Как выбрать длину хромосомы по требуемой точности?

8. В чём преимущество кода Грея? Опишите прямое и обратное преобразования. Почему неудачные запуски в примере остановились в точке $t = 4,375$?

9. Зачем нужны мутация и элитизм? Почему в конце поиска рулетка теряет давление отбора?

ЛАБОРАТОРНАЯ РАБОТА №6

ПОИСК ЭКСТРЕМУМА ФУНКЦИИ ДВУХ ПЕРЕМЕННЫХ С ПОМОЩЬЮ ГЕНЕТИЧЕСКИХ АЛГОРИТМОВ С БИНАРНЫМ И ВЕЩЕСТВЕННЫМ ПРЕДСТАВЛЕНИЕМ ОСОБЕЙ

Цель работы

Модифицировать простой генетический алгоритм для многомерного поиска экстремума; изучить бинарное кодирование нескольких переменных, операторы многоточечного и однородного кроссинговера, оператор инверсии, методы селекции ранжированием и турниром и стратегии сокращения промежуточной популяции; познакомиться с теоремой схем. Изучить представление особей вещественными числами, операторы вещественной рекомбинации и мутации и сравнить бинарный и вещественный ГА на одной задаче.

Теоретические сведения

Подразделы 5.5 и 5.7, раздел 6 теоретической части: кодирование нескольких переменных и предел точности сетки, теорема схем (5.6), селекция ранжированием (6.1) и турниром, многоточечный и однородный кроссинговер, инверсия, стратегии сокращения популяции; подраздел 6.5: вещественная рекомбинация (6.2), $BLX-\alpha$ (6.3), гауссова и неравномерная мутации.

Постановка задачи

Дана функция $y = f(x_1, x_2)$ (таблица Лб.1) — одна из стандартных тестовых функций численной оптимизации.

Часть А. Найти минимум функции при $x_i \in [a; b]$ генетическим алгоритмом с бинарным представлением особей и графически проиллюстрировать динамику поиска точки экстремума на плоскости $X_1 X_2$. В качестве оператора кроссинговера использовать многоточечный или однородный кроссинговер, в качестве оператора мутации — инверсию, селекцию выполнять ранжированием или турниром. Провести эксперименты с разными методами сокращения промежуточной популяции.

Часть Б. Найти минимум той же функции генетическим алгоритмом с представлением особей в форме вещественных чисел и сравнить результат с бинарным ГА части А при одинаковом бюджете вычислений функции.

Функции подобраны так, чтобы проверить разные трудности поиска (рисунок Лб.1): узкий изогнутый овраг (1), множество концентрических колец локальных минимумов (2, 3), резкие перепады значений (4), «иголка» глобального минимума на плоском фоне (5), регулярная решётка локальных минимумов (6–8). Глобальные минимумы: 0 в точке (1; 1) для функции 1, 3 в точке (0; -1) для функции 4, -1 в точке $(\pi; \pi)$ для функции 5 и 0 в начале координат для остальных.

Таблица Л6.1 – Варианты заданий к лабораторной работе №6

№	Функция	Формула	x_i
1	2-я де Йонга (Розенброка)	$100(x_1^2 - x_2)^2 + (1 - x_1)^2$	$[-2,048; 2,048]$
2	6-я Шаффера	$0,5 + \frac{\sin^2 \sqrt{x_1^2 + x_2^2} - 0,5}{(1 + 0,001(x_1^2 + x_2^2))^2}$	$[-100; 100]$
3	7-я Шаффера	$(x_1^2 + x_2^2)^{0,25} (\sin^2(50(x_1^2 + x_2^2)^{0,1}) + 1)$	$[-100; 100]$
4	Голдстейна — Прайса	$(1 + (x_1 + x_2 + 1)^2(19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3x_2^2)) \times$ $\times (30 + (2x_1 - 3x_2)^2(18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2))$	$[-2; 2]$
5	Изома	$-\cos x_1 \cos x_2 e^{-(x_1 - \pi)^2 - (x_2 - \pi)^2}$	$[-100; 100]$
6	1-я Бохачевского	$x_1^2 + 2x_2^2 - 0,3 \cos(3\pi x_1) - 0,4 \cos(4\pi x_2) + 0,7$	$[-50; 50]$
7	2-я Бохачевского	$x_1^2 + 2x_2^2 - 0,3 \cos(3\pi x_1) \cos(4\pi x_2) + 0,3$	$[-50; 50]$
8	Растригина	$20 + \sum_{i=1}^2 (x_i^2 - 10 \cos(2\pi x_i))$	$[-5,12; 5,12]$

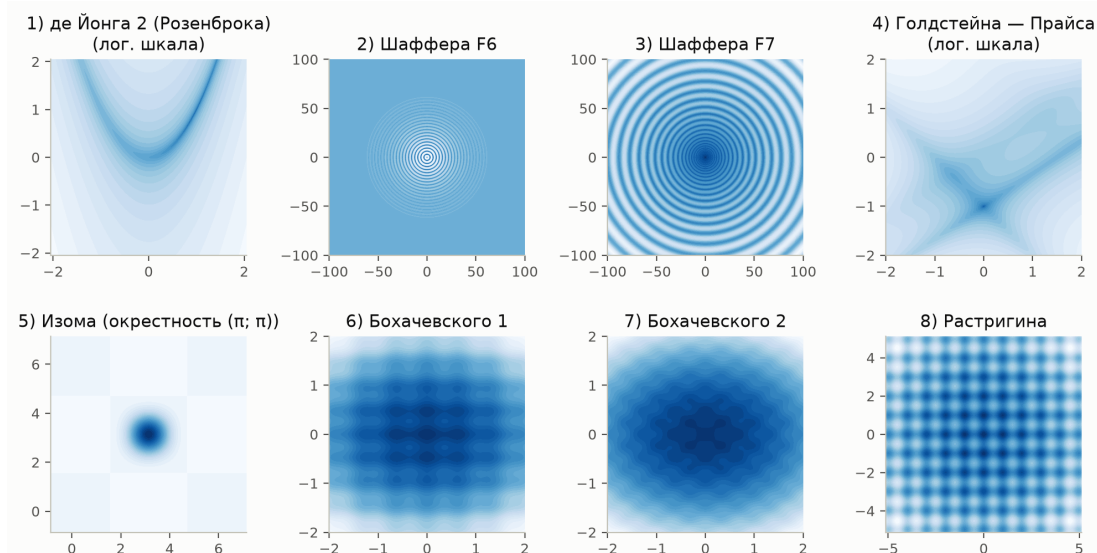


Рисунок Л6.1 – Линии уровня тестовых функций (темнее — меньше; для функций 5–7 показан фрагмент области вблизи минимума)

Требования к программе

Исходные данные: размер популяции, длина кода одной переменной (в части А), вероятности кроссинговера и мутации, параметры операторов, максимальное число поколений, метод селекции и стратегия сокращения популяции. В процессе поиска отображаются: линии уровня функции на плоскости X_1X_2 и положение особей текущей популяции; графики лучшего и среднего значений функции по поколениям; координаты лучшей особи и значение функции в ней.

Порядок выполнения работы

1. Часть А: построить линии уровня функции (`plt.contourf`), найти эталонный минимум (`scipy.optimize.differential_evolution`) и предел

точности кодирования — минимальное значение функции в узлах двоичной сетки вблизи глобального минимума.

2. Модифицировать программу ЛР №5 (операторы на NumPy — подраздел 6.8): хромосома — конкатенация кодов двух переменных; селекция — ранжирование или турнир; многоточечный или однородный кроссинговер; инверсия (совместно с побитовой мутацией); сокращение популяции — элитарная схема.

3. Выполнить основной запуск с визуализацией динамики популяции.

4. Сравнить операторы мутации: инверсия с побитовой мутацией, только инверсия, только побитовая мутация, без мутации (не менее 20 запусков каждой конфигурации).

5. Сравнить стратегии сокращения популяции: элитарную ($\mu + \lambda$), чистую замену, случайную замену и другие по выбору.

6. Исследовать влияние размера популяции, вероятностей операторов и размера турнира.

7. Часть Б: реализовать на NumPy вещественный ГА (турнирная селекция, BLX- α или промежуточная рекомбинация, гауссова или неравномерная мутация с убывающим шагом, элитизм) и выполнить основной запуск с визуализацией.

8. Исследовать влияние α , шага и вероятности мутации, размера популяции (по 20 запусков при сокращённом бюджете поколений, чтобы различия были заметны).

9. Сравнить вещественный и бинарный ГА при одинаковых N и числе поколений.

Пример выполнения

Часть А. Вариант 3: минимум 7-й функции Шаффера на $[-100; 100]^2$, глобальный минимум $f(0; 0) = 0$. Минимумы функции образуют концентрические кольца, сгущающиеся к началу координат, поэтому градиентный спуск застревает на первом же кольце. При $L = 20$ бит на переменную шаг сетки $\Delta = 1,9 \cdot 10^{-4}$, но точка 0 не представима: ближайшие узлы дают $x_i = \pm\Delta/2$, и минимальное значение функции на сетке равно 0,0200. Меньшего значения не получит никакой алгоритм с таким кодированием, поэтому успехом считается $f < 0,1$, а дополнительно фиксируется доля запусков, достигших предела 0,0200.

ГА с параметрами $N = 80$, турнир $k = 3$, однородный кроссинговер $P_c = 0,85$, инверсия $P_{inv} = 0,2$ совместно с побитовой мутацией $1/2L$, элитарное сокращение нашёл точку $(-9,5 \cdot 10^{-5}; 9,5 \cdot 10^{-5})$ с $f = 0,0200$ — точно на пределе сетки (24 080 вычислений функции). За 10 поколений популяция стягивается с области ± 100 к кольцам радиусом несколько единиц (рисунок Л6.2).

Сравнение операторов мутации и стратегий сокращения (20 запусков) приведено в таблице Л6.2. Инверсия без побитовой мутации слаба: она лишь

Популяция сжимается к началу координат (обратите внимание на масштаб осей)

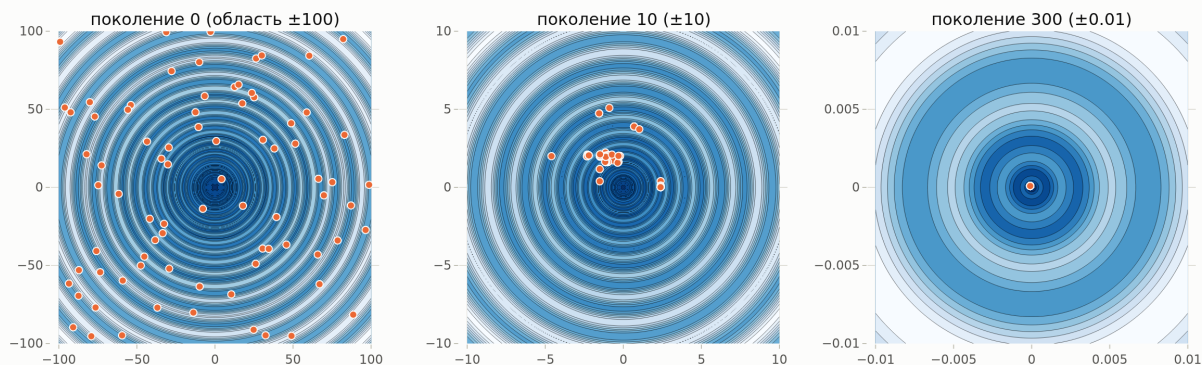


Рисунок Л6.2 – Популяция ГА на линиях уровня функции Шаффера F7: поколения 0, 10 и 300 (масштаб осей уменьшается)

переставляет биты, не меняя их состава, и не создаёт недостающих значений. Без мутации популяция вырождается. Стратегия сокращения определяет скорость поиска: без элитизма лучшие особи теряются, и кривая лучшей особи поколения колеблется. Рост популяции с 20 до 160 ускоряет поиск с 58 до 22 поколений, увеличение турнира с 2 до 8 особей — с 31 до 20 поколений.

Таблица Л6.2 – Операторы мутации и стратегии сокращения популяции (20 запусков; поколений — медиана до $f < 0,1$)

Конфигурация	Медиана f	Успех, %	До предела, %	Поколений
инверсия + побитовая, элитарное	0,0200	100	90	26
только инверсия	0,0638	70	10	47
только побитовая	0,0200	100	80	29
без мутации	1,0128	5	0	11
чистая замена	0,0200	100	65	44
случайная замена	0,0208	100	35	84

Часть Б. Вариант 3, та же функция F7. Вещественный ГА: $N = 60$, турнир $k = 3$, BLX-0,5 с $P_c = 0,9$, гауссова мутация ($P_m = 0,2$ на ген, $\sigma_0 = 0,1$, шаг убывает линейно), две элиты. За 150 поколений (9060 вычислений функции) найдено $f = 2,2 \cdot 10^{-9}$, порог $f < 10^{-3}$ пройден на 54-м поколении (рисунок Л6.3).

Сравнение с бинарным ГА части А при равном бюджете 60×150 (20 запусков): бинарный ГА упирается в предел своей сетки (медиана и лучшее $f = 0,020$, худшее 0,18), вещественный доходит до медианы $3,5 \cdot 10^{-9}$ (лучшее $4,4 \cdot 10^{-12}$, худшее 0,015). Влияние параметров при сокращённом бюджете в 50 поколений приведено в таблице Л6.3.

Главный оператор на этой функции — BLX- α : при $\alpha = 0$ популяция сжимается раньше, чем доходит до центра колец, при $\alpha = 1$ разброс потомков слишком велик для доводки. F7 радиально симметрична, BLX-0,5 сам обеспечивает исследование, поэтому при малом бюджете крупный шаг



Рисунок Л6.3 – Популяция вещественного ГА на линиях уровня функции Шаффера F7 (масштаб осей уменьшается)

Таблица Л6.3 – Вещественный ГА на функции F7, 50 поколений: доля запусков с $f < 10^{-2}$ и медиана f (20 запусков на значение)

	Знач.	Успех, %	Медиана f		Знач.	Успех, %	Медиана f
α (BLX)	0	20	$4,3 \cdot 10^{-2}$	P_m	0	95	$1,3 \cdot 10^{-5}$
	0,25	70	$5,4 \cdot 10^{-3}$		0,05	95	$5,1 \cdot 10^{-5}$
	0,5	95	$3,3 \cdot 10^{-3}$		0,2	95	$3,3 \cdot 10^{-3}$
	1	15	$1,6 \cdot 10^{-2}$		0,5	0	$1,9 \cdot 10^{-1}$
σ_0	0	100	$1,5 \cdot 10^{-4}$	N	20	5	$1,8 \cdot 10^{-1}$
	0,02	95	$3,0 \cdot 10^{-3}$		40	55	$6,6 \cdot 10^{-3}$
	0,1	95	$3,3 \cdot 10^{-3}$		60	95	$3,3 \cdot 10^{-3}$
	0,3	85	$4,7 \cdot 10^{-3}$		120	100	$1,8 \cdot 10^{-3}$

мутации лишь мешает доводке, а при $P_m = 0,5$ не успешен ни один запуск. Популяции из 20 особей недостаточно.

Контрольные вопросы

1. Поясните понятие схемы, её порядка и определяющей длины. Сформулируйте фундаментальную теорему ГА и объясните вклад каждого оператора.
2. Что такое строительные блоки и неявный параллелизм?
3. Как кодируется точка с несколькими координатами? Почему ГА с двоичным кодом не может найти точный минимум функции Шаффера на симметричной области?
4. Опишите селекцию ранжированием, турниром, усечением и локальный отбор. Как регулируется давление отбора?
5. Опишите способы выбора пар: панмиксию, селективный выбор, инбридинг и аутбридинг.
6. Опишите многоточечный и однородный кроссинговер. Чем однородный кроссинговер отличается по действию на схемы?
7. Почему инверсия без побитовой мутации не обеспечивает сходимости к минимуму?

8. Опишите стратегии сокращения промежуточной популяции: чистую замену, элитарную схему, случайную замену, пропорциональную редукцию, отбор вытеснением.

9. Что такое преждевременная сходимость и как её предотвратить?

10. Каковы преимущества вещественного кодирования? Опишите дискретную, промежуточную и линейную рекомбинацию, BLX- α .

11. Опишите равномерную, гауссову и неравномерную мутации вещественных особей. Зачем уменьшать шаг мутации? Почему вещественный ГА находит минимум F7 точнее бинарного?

ЛАБОРАТОРНАЯ РАБОТА №7

ГЕНЕТИЧЕСКИЙ АЛГОРИТМ С ПЕРЕСТАНОВОЧНЫМ ПРЕДСТАВЛЕНИЕМ ОСОБЕЙ. ЗАДАЧА КОММИВОЯЖЁРА

Цель работы

Изучить представление особей в форме перестановок, специализированные операторы скрещивания и мутации для него, особенности решения задач комбинаторной оптимизации генетическими алгоритмами и способы учёта ограничений; реализовать ГА, решающий задачу коммивояжёра, и меметический алгоритм с локальным поиском.

Теоретические сведения

Раздел 7 теоретической части: учёт ограничений в ГА на примере задач о рюкзаке (7.1) и о минимальном покрытии (7.2), задача коммивояжёра (7.3), представления тура, операторы PMX, OX, CX, мутации перестановок, локальный поиск 2-opt (7.4), меметические алгоритмы.

Постановка задачи

Дано множество городов, заданных координатами $(x_i; y_i)$ на плоскости; стоимость перехода между городами равна евклидову расстоянию. Построить генетическим алгоритмом тур обхода всех городов минимальной длины, используя представление путей, оператор кроссинговера и тестовый пример из таблицы Л7.1. Сравнить найденный тур с оптимальным из файла *.opt.tour и графически проиллюстрировать поиск.

Таблица Л7.1 – Варианты заданий к лабораторной работе №7

№	Оператор кроссинговера	Файл TSPLIB	Городов	Оптимальный тур
1	частично соответствующий (PMX)	berlin52.tsp	52	7542
2	упорядоченный (OX)	eil51.tsp	51	426
3	циклический (CX)	eil76.tsp	76	538
4	частично соответствующий (PMX)	pr76.tsp	76	108 159
5	упорядоченный (OX)	st70.tsp	70	675
6	циклический (CX)	lin105.tsp	105	14 379
7	частично соответствующий (PMX)	kroA100.tsp	100	21 282

Файлы TSPLIB [28] текстовые: заголовок (NAME, TYPE : TSP, DIMENSION — число городов, EDGE_WEIGHT_TYPE : EUC_2D — евклидово расстояние, округлённое до целого), раздел NODE_COORD_SECTION со строками «номер x y » и строка EOF. В файле *.opt.tour в разделе TOUR_SECTION перечислены города оптимального тура, список завершается числом -1; пересчёт его длины программой проверяет чтение данных и функцию расстояния.

Требования к программе

Исходные данные: размер популяции, вероятности кроссинговера и мутации, максимальное число поколений, файл *.tsp со списком городов. Отображаются лучший тур текущей популяции в виде графа на плоскости и в виде списка городов, графики лучшей и средней длины тура по поколениям и длина лучшего тура. Отладку проводить на файлах TSPLIB, сравнивая решения с оптимальными турами.

Порядок выполнения работы

1. Реализовать чтение файлов *.tsp и *.opt.tour, матрицу расстояний с округлением по правилу EUC_2D и длину тура (подраздел 7.5); проверить, что длина опубликованного оптимального тура совпадает с указанной в таблице Л7.1.

2. Реализовать ГА с представлением путей: оператор кроссинговера варианта, мутация обменом или инверсией, турнирная селекция, элитарное сокращение без дубликатов. Проверить оператор кроссинговера на примере из подраздела 7.4 (для PMX и OX — также сравнением с DEAP).

3. Выполнить серию запусков и сравнить лучшую найденную длину с оптимальной.

4. Дополнительно: добавить локальный поиск 2-opt к каждому потомку (меметический ГА) и сравнить с мультистартом 2-opt без ГА при одинаковом времени работы.

Пример выполнения

Вариант 3: набор eil76 (76 городов, оптимальный тур длиной 538, средняя длина случайного тура — 2520), циклический кроссинговер. Результаты приведены в таблице Л7.2 и на рисунке Л7.1.

Таблица Л7.2 – Задача коммивояжёра eil76, оптимум 538 (ГА — 150×500 , меметический — 60×60)

Метод	Запусков	Лучший	Медиана	Худший	Отклонение, %
ГА: CX + обмен	10	772	818	847	+52,0
ГА: CX + инверсия	10	876	965	1071	+79,4
Меметический ГА	5	538	538	538	0
Мультистарт 2-opt	5	540	550	553	+2,2

ГА с CX за 75 000 оценок даёт туры на 43–57 % длиннее оптимального: CX сохраняет позиции городов, а для маршрута важна их смежность, и рёбра хороших родителей почти не наследуются. Меметический ГА нашёл тур длиной 538 во всех пяти запусках примерно за 5 с (маршрут отличается от опубликованного: у eil76 несколько оптимальных туров), а мультистарт 2-opt за то же время — лишь 540–553.

eil76: лучшие найденные маршруты и оптимальный

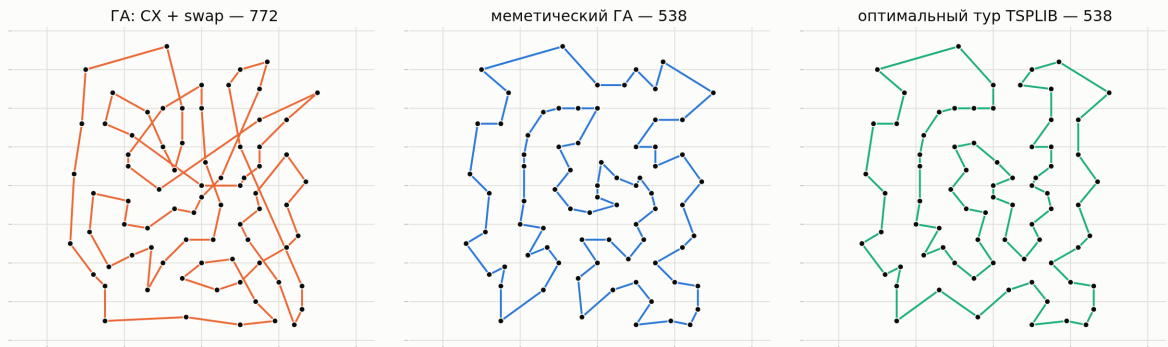


Рисунок Л7.1 – Лучшие найденные маршруты и опубликованный оптимальный тур eil76

Контрольные вопросы

1. Сформулируйте задачи о рюкзаке, о минимальном покрытии и о коммивояжёре. Какое представление решения используется в каждой из них?
2. Какие способы учёта ограничений применяются в ГА? Сравните штрафы, восстановление и декодеры на примере задачи о рюкзаке.
3. Приведите способы представления тура и их особенности. Почему для задачи коммивояжёра не подходит обычный двоичный кроссинговер?
4. Опишите представление соседства и операторы обмена рёбер, обмена подтуров и эвристического скрещивания.
5. Опишите порядковое представление тура. Какой оператор кроссинговера для него применим?
6. Опишите операторы PMX, OX и CX для представления путей; выполните их для родителей (1 2 3 4 5 6 7 8) и (3 7 5 1 6 8 2 4).
7. Какие мутации применяются к перестановкам? Опишите алгоритм 2-opt. Что такое меметический алгоритм и почему он эффективнее чистого ГА?

ЛАБОРАТОРНАЯ РАБОТА №8

СОВРЕМЕННАЯ БЕЗГРАДИЕНТНАЯ ОПТИМИЗАЦИЯ: СМА-ES, БАЙЕСОВСКАЯ ОПТИМИЗАЦИЯ И NSGA-II

Цель работы

Изучить эволюционную стратегию с адаптацией ковариационной матрицы, байесовскую оптимизацию и многокритериальный генетический алгоритм NSGA-II; сравнить их с генетическими алгоритмами ЛР №5–6 и применить к двум практическим задачам машинного обучения — подбору гиперпараметров и отбору признаков.

Теоретические сведения

Подраздел 6.5 и раздел 8 теоретической части: СМА-ES (8.1)–(8.3) и перезапуски IPOP, байесовская оптимизация и TPE, практика подбора гиперпараметров, доминирование по Парето, NSGA-II, операторы SBX и полиномиальной мутации, показатели HV и IGD.

Постановка задачи

Часть А. Реализовать СМА-ES и IPOP-СМА-ES на NumPy и найти минимум функции варианта (таблица Л6.1); сравнить со случайным поиском и вещественным ГА ЛР №6 при одинаковом бюджете вычислений функции; проверить реализацию по библиотеке `сма`.

Часть Б. Подобрать два гиперпараметра модели варианта (таблица Л8.1) при бюджете 30 обучений: перебором по сетке, случайным поиском, своим вещественным ГА, Optuna TPE и Optuna СМА-ES; целевая функция — точность на 5-блочной кросс-валидации обучающей части (80 %), итог проверяется на отложенных 20 %.

Часть В. Реализовать NSGA-II, проверить её на задаче ZDT1 (30 переменных) против `румоо` по показателям HV и IGD. Выполнить многокритериальный отбор признаков для датасета варианта: минимизировать ошибку кросс-валидации логистической регрессии и число признаков (двоичная хромосома).

Таблица Л8.1 – Варианты заданий к лабораторной работе №8 (функция — по таблице Л6.1, датасеты из `sklearn.datasets`, модели — по таблице Л8.2)

№	Функция	Модель	Датасет	№	Функция	Модель	Датасет
1	1	SVC	wine	5	5	HGB	breast_cancer
2	2	KNN	digits	6	6	SVC	digits
3	3	SVC	breast_cancer	7	7	KNN	breast_cancer
4	4	RF	wine	8	8	HGB	wine

Требования к программе

СМА-ES и NSGA-II реализуются самостоятельно; библиотеки `сма`, Optuna и `румоо` используются для сравнения. Отображаются: эллипсы

Таблица Л8.2 – Модели и гиперпараметры части Б: классы из `sklearn.svm`, `sklearn.neighbors` и `sklearn.ensemble`; целочисленные параметры при поиске округляются

Код	Класс	Гиперпараметры и диапазоны
SVC	SVC (RBF-ядро)	$\lg C \in [-2; 4]$, $\lg \gamma \in [-6; 0]$
KNN	KNeighborsClassifier	$n_neighbors \in [1; 50]$, $p \in [1; 3]$
RF	RandomForestClassifier	$max_depth \in [1; 20]$, $min_samples_leaf \in [1; 20]$
HGB	HistGradientBoostingClassifier	$\lg learning_rate \in [-3; 0]$, $max_leaf_nodes \in [2; 64]$

распределения СМА-ES на линиях уровня функции; медианные кривые «лучшее значение от числа вычислений» для всех методов; карта точности модели на плоскости двух гиперпараметров с точками, выбранными методами поиска; найденный и истинный фронты Парето.

Порядок выполнения работы

1. Реализовать СМА-ES (приложение Б), проверить на функции сферы в 10-мерном пространстве (ожидаемое $f < 10^{-20}$) и на функции Розенброка; построить эллипсы распределения.
2. Сравнить на функции варианта случайный поиск, вещественный ГА, СМА-ES, IPOP-СМА-ES и *сма* (20 запусков, одинаковый бюджет), найти долю успешных запусков и число вычислений до успеха.
3. Для модели варианта построить карту точности на сетке 31×31 и выполнить поиск всеми методами (10 запусков с разными *seed*); сравнить скорость роста лучшей точности и точность на тесте.
4. Реализовать NSGA-II и операторы SBX и полиномиальной мутации, сравнить с *рутоо* на ZDT1.
5. Выполнить отбор признаков с помощью NSGA-II (кэшируя оценки уже встречавшихся наборов) и выбрать компромиссный набор признаков по фронту Парето с учётом точности на тесте.

Пример выполнения

Часть А. Вариант 3: функция Шаффера F7, бюджет 9060 вычислений (как у вещественного ГА ЛР №6), 20 запусков. Своя СМА-ES совпадает по поведению с библиотечной (таблица Л8.3), а на функции Розенброка эллипс распределения вытягивается вдоль изогнутого оврага и сжимается у минимума (рисунок Л8.1).

Одиночная СМА-ES в успешных запусках достигает 10^{-3} за 714 вычислений — в четыре раза быстрее ГА, но в 17 из 20 запусков её распределение сжимается на одном из колец F7. Перезапуски с удвоением популяции поднимают долю успехов до 80 %. На многоэкстремальной функции популяционный ГА остаётся надёжнее, а СМА-ES выигрывает на гладких и овражных функциях.

Таблица Л8.3 – Функция F7: лучшее f за 9060 вычислений (20 запусков), успех — $f < 10^{-3}$

Метод	Медиана f	Худшее f	Успех, %	Вычислений до успеха
случайный поиск	1,3	1,8	0	—
вещественный ГА (ЛР №6)	$3,5 \cdot 10^{-9}$	0,015	90	3210
СМА-ES (своя)	0,010	10	15	714
СМА-ES (сма)	0,014	4,8	10	—
IPOP-CMA-ES (своя)	$5,6 \cdot 10^{-7}$	0,031	80	3456

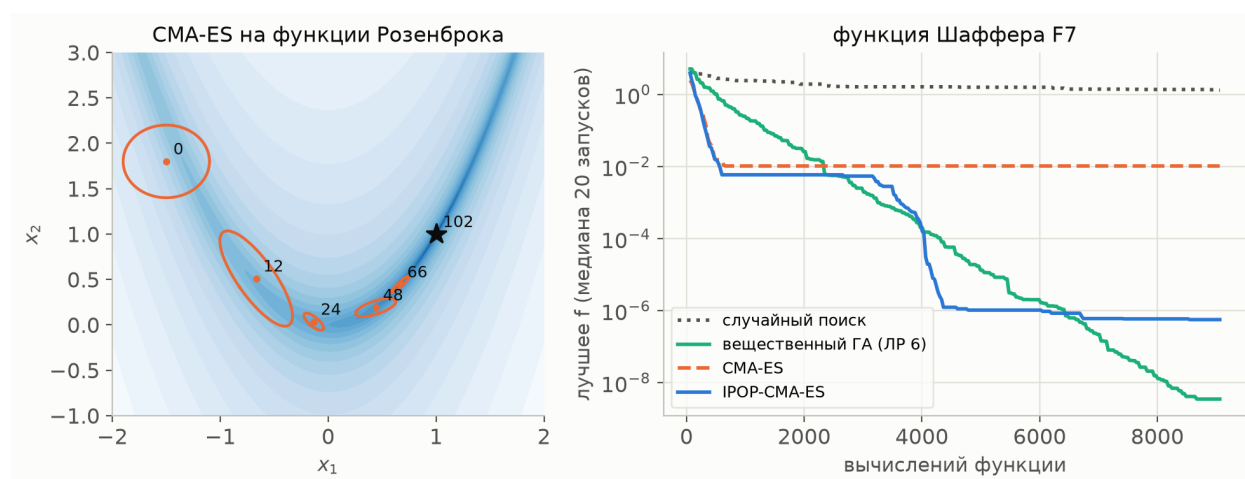


Рисунок Л8.1 – Слева — эллипсы распределения СМА-ES на функции Розенброка (числа — номер поколения); справа — медиана лучшего значения F7 по 20 запускам

Часть Б. Вариант 3: SVC с RBF-ядром на breast_cancer, $\lg C \in [-2; 4]$, $\lg \gamma \in [-6; 0]$. Максимум точности на сетке 31×31 — 0,987. Результаты приведены в таблице Л8.4 и на рисунке Л8.2.

Таблица Л8.4 – Подбор гиперпараметров SVC: 30 обучений, 10 запусков (сетка — один детерминированный); последний столбец — медиана числа обучений до $CV \geq 0,98$

Метод	CV, медиана	CV, худший	Тест, медиана	До $CV \geq 0,98$
сетка 6×5	0,985	0,985	0,974	14
случайный поиск	0,981	0,976	0,974	14 (9 из 10)
ГА ($N = 6$, 5 поколений)	0,982	0,980	0,974	10,5
Optuna TPE	0,984	0,980	0,974	7
Optuna CMA-ES	0,982	0,982	0,974	6

Методы с моделью поверхности (TPE, СМА-ES) выходят на точность 0,98 за 6–7 обучений, случайный поиск и сетка — за 14. TPE сгущает пробы в диагональной «долине» высокой точности. Итоговая точность на тесте у всех методов одинакова (0,974): различия в CV на уровне 0,003 меньше разброса оценки, и выбор максимума CV немного переобучается на валидацию. Практический вывод: экономия проявляется в числе дорогих обучений, а не в итоговом качестве простой модели.

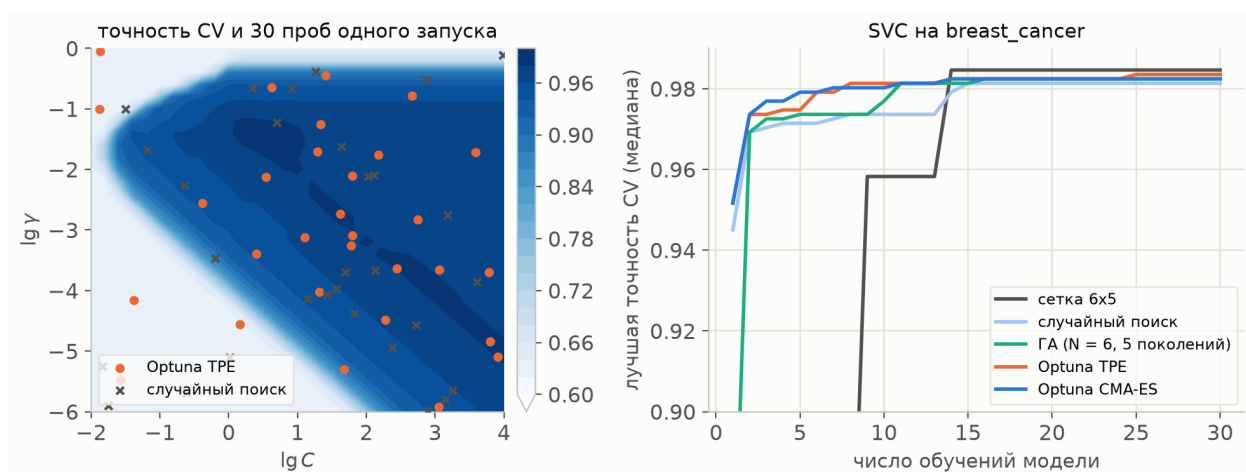


Рисунок Л8.2 – Слева — точность кросс-валидации на плоскости гиперпараметров и 30 проб TPE и случайного поиска в одном запуске; справа — медиана лучшей найденной точности

Часть В. На ZDT1 (100 особей, 200 поколений, 5 запусков) своя NSGA-II получила $IGD = 0,0044$ и $HV = 0,8707$, румоо — $0,0051$ и $0,8682$ (HV истинного фронта $0,8714$), за $0,28$ и $0,36$ с. Первая версия реализации без обмена переменными между потомками SBX давала $IGD = 0,41$: без обмена кроссинговер не комбинирует переменные разных родителей — типичная ошибка, которую ловит сравнение с библиотекой. Отбор признаков на breast_cancer (40 особей, 25 поколений, 813 различных наборов, $5,5$ с) дал фронт, показанный на рисунке Л8.3 и в таблице Л8.5.

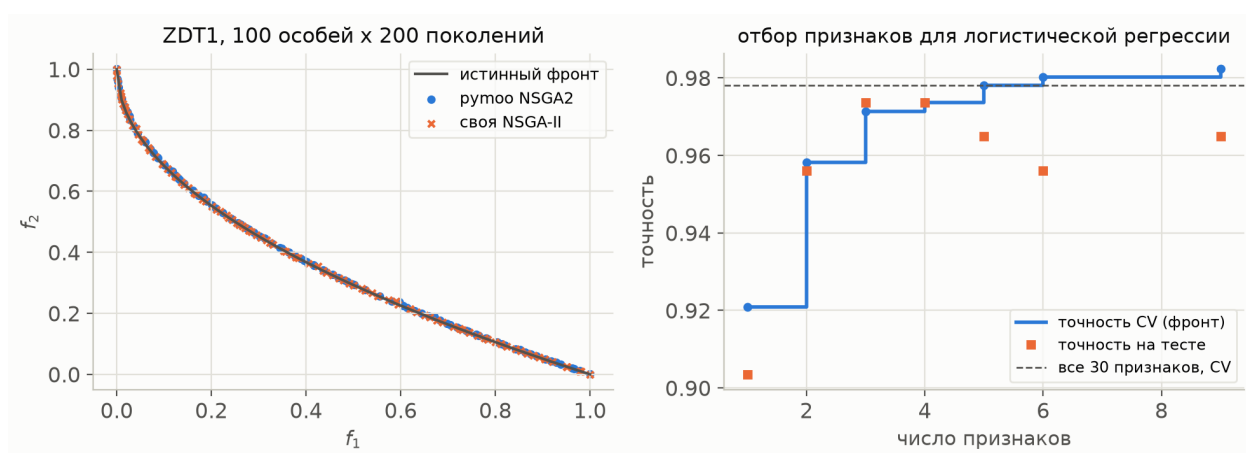


Рисунок Л8.3 – Слева — фронты задачи ZDT1; справа — фронт Парето отбора признаков и точность на тесте

Таблица Л8.5 – Фронт Парето отбора признаков: логистическая регрессия на breast_cancer

Признаков	1	2	3	4	5	6	9	все 30
Точность CV	0,921	0,958	0,971	0,974	0,978	0,980	0,982	0,978
Точность тест	0,904	0,956	0,974	0,974	0,965	0,956	0,965	0,982

Трёх–четырёх признаков достаточно для точности 0,97, пяти — для точности кросс-валидации модели на всех 30 признаках. Дальнейший рост CV не подтверждается на тесте: при отборе по той же валидации, по которой строится фронт, признаки подгоняются под шум. Компромисс выбирают в «колене» фронта — здесь 3–4 признака.

Контрольные вопросы

1. В каких задачах нужны безградиентные методы оптимизации? Когда градиентные методы предпочтительнее?
2. Опишите правило одной пятой для (1+1)-ЭС.
3. Запишите шаг СМА-ES: порождение точек, обновление среднего, ковариационной матрицы и шага. Зачем нужны пути эволюции?
4. Почему СМА-ES инвариантна к монотонным преобразованиям функции и к поворотам координат? Почему одиночная СМА-ES застревает на многоэкстремальных функциях и как это исправляет IPOR?
5. Что такое суррогатная модель и функция приобретения? Запишите ожидаемое улучшение. Как работает TPE?
6. Почему случайный поиск гиперпараметров эффективнее сетки? Почему лучшее значение кросс-валидации оптимистично смещено?
7. Дайте определения доминирования и фронта Парето. Почему взвешенная сумма критериев не находит невыпуклые части фронта?
8. Опишите недоминируемую сортировку, расстояние скученности и отбор в NSGA-II.
9. Что измеряют гипербъём и IGD?

ЛАБОРАТОРНАЯ РАБОТА №9

ЭВОЛЮЦИЯ ЭВРИСТИК С ПОМОЩЬЮ БОЛЬШОЙ ЯЗЫКОВОЙ МОДЕЛИ

Цель работы

Изучить эволюционный поиск программ, в котором оператором мутации служит большая языковая модель (подход FunSearch); реализовать безопасный оценщик сгенерированного кода и мини-FunSearch для задачи онлайн-упаковки в контейнеры; сравнить найденные эвристики с классическими правилами и с параметрической эвристикой, настроенной СМА-ES, и оценить их обобщение на новые данные.

Теоретические сведения

Подраздел 8.5 (FunSearch, требования к оценщику, онлайн-упаковка в контейнеры), подраздел 8.2 (СМА-ES для настройки параметров эвристики), подраздел 3.5 (обобщение и переобучение) и подраздел 6.7 (методика сравнения стохастических алгоритмов).

Постановка задачи

Предметы с целыми размерами, сгенерированными из распределения варианта (таблица Л9.1) и ограниченными отрезком $[1; C]$, поступают по одному. Эвристика задаётся функцией приоритета

```
def priority(item, bins, capacity):    # bins: свободное место в контейнерах, куда item помещается;  
    return -(bins - item)            # последний элемент - новый пустой контейнер (Best Fit)
```

и предмет кладётся в контейнер с наибольшим приоритетом. Качество эвристики — среднее по наборам превышение нижней границы, %:

$$E = \frac{100}{K} \sum_{k=1}^K \frac{m_k - L_1^{(k)}}{L_1^{(k)}}, \quad L_1^{(k)} = \left\lceil \sum_i s_i^{(k)} / C \right\rceil,$$

где m_k — число контейнеров, использованных на k -м наборе. Поиск ведётся на обучающих наборах ($K = 5$ наборов по 1000 предметов, seed 0–4), итог проверяется на тестовых ($K = 5$ по 5000 предметов, seed 100–104).

Часть А. Реализовать окружение упаковки и классические правила First Fit, Best Fit, Worst Fit. Сформулировать параметрическую эвристику — Best Fit со штрафом a за мелкий бесполезный остаток меньше t и бонусом b за точное заполнение:

$$p(r) = -r - a [0 < r < t] + b [r = 0], \quad r = \text{bins} - \text{item},$$

где $[\cdot]$ — индикатор условия, и подобрать параметры (a, t, b) своей СМА-ES из ЛР №8.

Часть Б. Реализовать мини-FunSearch: база программ с оценками, подсказка с двумя версиями функции, выбранными турниром и упорядоченными по возрастанию качества, генерация новой версии языковой моделью, изолированная оценка кода. Начальная программа — Best Fit, бюджет — 60 вызовов модели (15 волн по 4 параллельных вызова).

Часть В. Проверить обобщение: оценить пять лучших эвристик на тестовых наборах и на варианте переноса — том же распределении с другой ёмкостью (вариант $k + 4$ для $k \leq 4$ и $k - 4$ для $k > 4$). Выполнить абляцию лучшей эвристики: удалять из кода по одному правилу и измерять вклад каждого.

Таблица Л9.1 – Варианты заданий к лабораторной работе №9 и контрольные значения E на обучающих наборах для генератора из приложения Б

№	Распределение размеров предметов	C	First Fit, %	Best Fit, %
1	равномерное $U[20; 100]$	150	5,40	4,85
2	нормальное $N(50; 15)$	100	5,82	4,37
3	Вейбулла, масштаб 45, форма 3	100	5,25	4,76
4	смесь $0,5 N(30; 5) + 0,5 N(65; 5)$	120	10,19	9,49
5	равномерное $U[20; 100]$	120	6,73	4,94
6	нормальное $N(50; 15)$	150	4,40	4,22
7	Вейбулла, масштаб 45, форма 3	150	2,82	2,75
8	смесь $0,5 N(30; 5) + 0,5 N(65; 5)$	150	6,22	6,09

Требования к программе

- код, полученный от модели, считается недоверенным: перед запуском проверяется по синтаксическому дереву (модуль `ast`: разрешены только импорты `numpy` и `math`, запрещены `open`, `exec`, `eval`, `__import__`, модули `os`, `sys`, `subprocess` и атрибуты вида `__имя__`) и выполняется в отдельном процессе с тайм-аутом; упавшая, зависшая или вернувшая нечисловую оценку программа отбрасывается;
- языковая модель подключается функцией `llm(prompt) ->` (текст, стоимость): Claude через Anthropic SDK или Claude Code CLI (приложение Б), модели других провайдеров или локальная открытая модель — по выбору студента;
- ключ доступа к API задаётся только переменной окружения; он не записывается в код, ноутбук, отчёт и репозиторий;
- ведётся журнал вызовов: подсказка, ответ, оценка, стоимость; поиск останавливается по бюджету;
- отображаются: кривая «лучшая оценка в базе от числа вызовов» с уровнями First Fit, Best Fit и параметрической эвристики; таблица оценок на обучающих и тестовых наборах; код лучшей эвристики.

Порядок выполнения работы

1. Реализовать генератор наборов, онлайн-упаковку с функцией приоритета и оценку E ; сверить First Fit и Best Fit с контрольными значениями таблицы Л9.1, объяснить результат Worst Fit.
2. Подобрать параметры (a, t, b) СМА-ES (240 оценок, $\lambda = 8$) и объяснить найденные значения.
3. Реализовать проверку и изолированный запуск кода; убедиться, что программы с `import os, open(...)` и бесконечным циклом отвергаются.
4. Выполнить мини-FunSearch (60 вызовов) и построить кривую поиска; записать число корректных программ, время и стоимость.
5. Оценить пять лучших эвристик на тестовых наборах и на варианте переноса, выполнить абляцию лучшей эвристики и описать словами, какие правила она содержит.

Пример выполнения

Вариант 3: распределение Вейбулла, $C = 100$ — та же постановка, что в статье FunSearch [41]. Модель `claude-opus-5-5` (уровень рассуждений `medium`) вызывалась через Claude Code CLI в неинтерактивном режиме без инструментов. Все 60 ответов дали корректный код; поиск занял 5 мин и стоил 2,8 доллара США. Для сравнения тот же поиск выполнен для смеси $0,5 N(30; 5) + 0,5 N(65; 5)$ при $C = 100$: 58 корректных программ из 60, 5,5 мин, 3,1 доллара (рисунок Л9.1, таблица Л9.4).

Таблица Л9.2 – Вариант 3: превышение нижней границы L_1 на обучающих (5×1000) и тестовых (5×5000) наборах, %

Эвристика	Обучение	Тест
Worst Fit	13,23	13,11
First Fit	5,25	4,36
Best Fit	4,76	3,97
параметрическая, СМА-ES: $a = 45,2, t = 24,8, b = 8,7$	3,17	2,29
мини-FunSearch, вызов 25 (эвристика из [41])	3,42	0,67
мини-FunSearch, лучшая на обучении (вызов 51)	2,03	0,74

Часть А. Worst Fit оставляет в каждом контейнере большой остаток и втрое хуже остальных правил. СМА-ES за 240 оценок (9 с) нашла штраф $a = 45$ за остаток меньше $t = 25$: такой остаток почти никогда не заполнится, поскольку типичный предмет имеет размер около 40. Одна понятная гипотеза и три параметра улучшают Best Fit в 1,7 раза на тесте.

Часть Б. Кривая поиска ступенчатая (рисунок Л9.1): большинство предложений модели не лучше текущего рекорда, улучшения случаются редко и скачками. На 25-м вызове появилась «разностная» квадратичная оценка из эвристики, опубликованной в статье FunSearch: оценка на обучающих наборах совпала с опубликованной эвристикой до третьего знака, а её формула дословно вошла в ядро всех последующих версий. В нашем

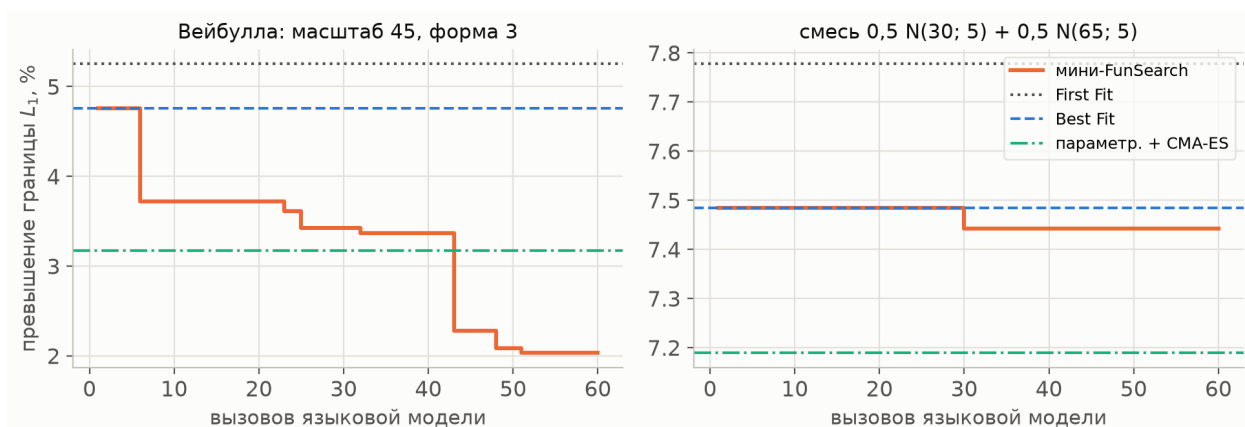


Рисунок Л9.1 – Лучшая оценка в базе программ от числа вызовов языковой модели (обучающие наборы, $C = 100$): слева — распределение Вейбулла (вариант 3), справа — смесь двух нормальных распределений

окружении эта эвристика даёт 0,67 % на тесте (в статье — 0,68 %). По-видимому, модель знает её из обучающих данных. Такая *утечка* типична для LLM: результат поиска нужно сверять с литературой, прежде чем называть его новым. Дальнейшие версии добавили к ядру правила, перечисленные в таблице Л9.3; почти точным считается остаток не больше 1 % ёмкости.

Часть В. На тесте лучшая на обучении эвристика уступает своему ядру (0,74 против 0,67 %). Абляция (таблица Л9.3) объясняет это: правило почти точного заполнения улучшает оценку на наборах по 1000 предметов почти вдвое, а на наборах по 5000 предметов немного ухудшает; штраф за «мёртвый» остаток не меняет упаковку совсем — модель добавила декоративный код. Поиск подогнал эвристику под условия обучения, как модель машинного обучения подгоняется под обучающую выборку; поэтому обучающие наборы должны соответствовать условиям применения, а итоговую версию следует выбирать по отложенным наборам.

Таблица Л9.3 – Абляция лучшей эвристики варианта 3: E , %

Эвристика	Обучение	Тест
лучшая (все правила)	2,03	0,74
без штрафа за «мёртвый» остаток	2,03	0,74
без бонуса за остаток, равный предмету	2,18	0,76
без правила почти точного заполнения	4,06	0,68
без правила точного заполнения	2,33	0,99
только ядро (эвристика из [41])	3,42	0,67

Перенос на другие данные (таблица Л9.4) показывает, что найденные эвристики специализированы. На ёмкость $C = 150$ при том же распределении выигрыш переносится (1,06 против 2,22 % у Best Fit), а на смеси эвристики, выведенные для распределения Вейбулла, в 3,3–3,6 раза хуже Best Fit. На первом тестовом наборе смеси эвристика из [41] оставляет в одиночестве 956 из 2531 крупного предмета (Best Fit — 104), а мелкие укладывает в отдельные контейнеры примерно по два.

На смеси поиск почти ничего не дал: одно улучшение за 60 вызовов, на тесте 6,27 против 6,28 % у Best Fit. Причина — в нижней границе. Предметы крупнее $C/2$ не помещаются вдвоём, поэтому контейнеров не меньше, чем таких предметов, и эта граница здесь в среднем на 5,8 % выше L_1 . Best Fit превышает её всего на 0,46 %: улучшать почти нечего. Прежде чем тратить бюджет на поиск, полезно оценить запас по более точной нижней границе. Параметры СМА-ES на смеси улучшили обучающую оценку (7,19 против 7,48 %), но не тестовую (6,29 против 6,28 %) — та же подгонка под обучающие наборы.

Таблица Л9.4 – Перенос эвристик: E на тестовых наборах (5×5000), %

Эвристика (где настроена)	Вейбулла, $C = 100$	Вейбулла, $C = 150$	смесь, $C = 100$
First Fit	4,36	2,40	6,45
Best Fit	3,97	2,22	6,28
параметрическая + СМА-ES (Вейбулла, 100)	2,29	2,04	6,36
эвристика из [41]	0,67	0,67	22,33
мини-FunSearch (Вейбулла, 100)	0,74	1,06	21,02
мини-FunSearch (смесь, 100)	3,97	2,25	6,27

Итог: поиск с языковой моделью за несколько долларов нашёл эвристику, в 5 раз лучшую Best Fit на своём распределении (но не на другом), и частично воспроизвёл известный результат. Параметрическая эвристика с СМА-ES хуже, зато бесплатна, понятна и настраивается за секунды — с неё стоит начинать.

Контрольные вопросы

1. Чем эволюция программ с языковой моделью отличается от генетического программирования? Какие операторы ГА заменяет модель?
2. Опишите цикл FunSearch. Зачем в подсказку помещают несколько версий, упорядоченных по возрастанию качества, и зачем база программ делится на острова?
3. Каким требованиям должен удовлетворять оценщик? Почему код модели выполняется в отдельном процессе с тайм-аутом и почему статической проверки кода недостаточно в промышленной системе?
4. Сравните правила First Fit, Best Fit и Worst Fit. Запишите нижнюю границу L_1 ; почему для смеси крупных и мелких предметов она недостижима?
5. Когда достаточно параметрической эвристики с настройкой СМА-ES, а когда нужен поиск в пространстве программ?
6. Почему улучшение на обучающих наборах может не переноситься на тестовые? Как выбирать итоговую эвристику?
7. Что такое утечка данных при поиске с языковой моделью и как её обнаружить? Как учитывать стоимость вызовов модели при сравнении методов?

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Рутковская, Д. Нейронные сети, генетические алгоритмы и нечёткие системы / Д. Рутковская, М. Пилиньский, Л. Рутковский ; пер. с польск. – М. : Горячая линия – Телеком, 2006. – 452 с.
- 2 Головкин, В. А. Нейросетевые технологии обработки данных : учеб. пособие / В. А. Головкин, В. В. Краснопрошин. – Минск : БГУ, 2017. – 263 с. – (Классическое университетское издание).
- 3 Медведев, В. С. Нейронные сети. MATLAB 6 / В. С. Медведев, В. Г. Потёмкин ; под общ. ред. В. Г. Потёмкина. – М. : ДИАЛОГ-МИФИ, 2002. – 496 с.
- 4 Скобцов, В. Ю. Интеллектуальный анализ данных: генетические алгоритмы : учеб.-метод. пособие / В. Ю. Скобцов, Н. В. Лапицкая, С. Н. Нестеренков. – Минск : БГУИР, 2018. – 92 с.
- 5 Holland, J. H. Adaptation in Natural and Artificial Systems / J. H. Holland. – Ann Arbor : University of Michigan Press, 1975. – 183 p.
- 6 Goldberg, D. E. Genetic Algorithms in Search, Optimization and Machine Learning / D. E. Goldberg. – Reading, MA : Addison-Wesley, 1989. – 412 p.
- 7 Michalewicz, Z. Genetic Algorithms + Data Structures = Evolution Programs / Z. Michalewicz. – 3rd ed. – Berlin : Springer, 1996. – 387 p.
- 8 Хайкин, С. Нейронные сети: полный курс / С. Хайкин ; пер. с англ. – 2-е изд. – М. : Вильямс, 2006. – 1104 с.
- 9 Абламейко, С. В. Модели и технологии распознавания образов с приложением в интеллектуальном анализе данных / С. В. Абламейко, В. В. Краснопрошин, В. А. Образцов // Вестник БГУ. Сер. 1, Физика. Математика. Информатика. – 2011. – № 3. – С. 62–72.
- 10 Harris, C. R. Array programming with NumPy / C. R. Harris, K. J. Millman, S. J. van der Walt [et al.] // Nature. – 2020. – Vol. 585. – P. 357–362.
- 11 Virtanen, P. SciPy 1.0: fundamental algorithms for scientific computing in Python / P. Virtanen, R. Gommers, T. E. Oliphant [et al.] // Nature Methods. – 2020. – Vol. 17. – P. 261–272.
- 12 Pedregosa, F. Scikit-learn: machine learning in Python / F. Pedregosa, G. Varoquaux, A. Gramfort [et al.] // Journal of Machine Learning Research. – 2011. – Vol. 12. – P. 2825–2830.
- 13 Rosenblatt, F. The perceptron: a probabilistic model for information storage and organization in the brain / F. Rosenblatt // Psychological Review. – 1958. – Vol. 65, № 6. – P. 386–408.
- 14 Минский, М. Перцептроны / М. Минский, С. Пейперт ; пер. с англ. – М. : Мир, 1971. – 261 с.
- 15 Widrow, B. Adaptive switching circuits / B. Widrow, M. E. Hoff // IRE WESCON Convention Record. – 1960. – Part 4. – P. 96–104.

- 16 Cybenko, G. Approximation by superpositions of a sigmoidal function / G. Cybenko // *Mathematics of Control, Signals and Systems*. – 1989. – Vol. 2, № 4. – P. 303–314.
- 17 Marquardt, D. W. An algorithm for least-squares estimation of nonlinear parameters / D. W. Marquardt // *Journal of the Society for Industrial and Applied Mathematics*. – 1963. – Vol. 11, № 2. – P. 431–441.
- 18 Nguyen, D. Improving the learning speed of 2-layer neural networks by choosing initial values of the adaptive weights / D. Nguyen, B. Widrow // *Proc. of the International Joint Conference on Neural Networks*. – San Diego, 1990. – Vol. 3. – P. 21–26.
- 19 Кохонен, Т. Самоорганизующиеся карты / Т. Кохонен ; пер. с англ. – 3-е изд. – М. : БИНОМ. Лаборатория знаний, 2008. – 655 с.
- 20 Vettigli, G. MiniSom: minimalistic and NumPy-based implementation of the Self Organizing Map [Электронный ресурс] / G. Vettigli. – 2018. – Режим доступа: <https://github.com/JustGlowing/minisom>. – Дата доступа: 29.09.2026.
- 21 Скобцов, Ю. А. Основы эволюционных вычислений : учеб. пособие / Ю. А. Скобцов. – Донецк : ДонНТУ, 2008. – 326 с.
- 22 Гладков, Л. А. Генетические алгоритмы / Л. А. Гладков, В. В. Курейчик, В. М. Курейчик ; под ред. В. М. Курейчика. – 2-е изд., испр. и доп. – М. : Физматлит, 2006. – 320 с.
- 23 Eshelman, L. J. Real-coded genetic algorithms and interval-schemata / L. J. Eshelman, J. D. Schaffer // *Foundations of Genetic Algorithms 2*. – San Mateo : Morgan Kaufmann, 1993. – P. 187–202.
- 24 Fortin, F.-A. DEAP: evolutionary algorithms made easy / F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner [et al.] // *Journal of Machine Learning Research*. – 2012. – Vol. 13. – P. 2171–2175.
- 25 Martello, S. Knapsack Problems: Algorithms and Computer Implementations / S. Martello, P. Toth. – Chichester : Wiley, 1990. – 296 p.
- 26 Chvátal, V. A greedy heuristic for the set-covering problem / V. Chvátal // *Mathematics of Operations Research*. – 1979. – Vol. 4, № 3. – P. 233–235.
- 27 Beasley, J. E. A genetic algorithm for the set covering problem / J. E. Beasley, P. C. Chu // *European Journal of Operational Research*. – 1996. – Vol. 94, № 2. – P. 392–404.
- 28 Reinelt, G. TSPLIB – a traveling salesman problem library / G. Reinelt // *ORSA Journal on Computing*. – 1991. – Vol. 3, № 4. – P. 376–384.
- 29 Goldberg, D. E. Alleles, loci, and the traveling salesman problem / D. E. Goldberg, R. Lingle // *Proc. of the 1st International Conference on Genetic Algorithms*. – Hillsdale : Lawrence Erlbaum, 1985. – P. 154–159.
- 30 Oliver, I. M. A study of permutation crossover operators on the traveling salesman problem / I. M. Oliver, D. J. Smith, J. R. C. Holland // *Proc. of the 2nd*

International Conference on Genetic Algorithms. – Hillsdale : Lawrence Erlbaum, 1987. – P. 224–230.

31 Whitley, D. Scheduling problems and traveling salesmen: the genetic edge recombination operator / D. Whitley, T. Starkweather, D. Fuquay // Proc. of the 3rd International Conference on Genetic Algorithms. – San Mateo : Morgan Kaufmann, 1989. – P. 133–140.

32 Croes, G.A. A method for solving traveling-salesman problems / G. A. Croes // Operations Research. – 1958. – Vol. 6, № 6. – P. 791–812.

33 Hansen, N. Completely derandomized self-adaptation in evolution strategies / N. Hansen, A. Ostermeier // Evolutionary Computation. – 2001. – Vol. 9, № 2. – P. 159–195.

34 Hansen, N. The CMA evolution strategy: a tutorial [Электронный ресурс] / N. Hansen. – 2016. – arXiv:1604.00772. – Режим доступа: <https://arxiv.org/abs/1604.00772>. – Дата доступа: 30.09.2026.

35 Snoek, J. Practical Bayesian optimization of machine learning algorithms / J. Snoek, H. Larochelle, R. P. Adams // Advances in Neural Information Processing Systems 25. – 2012. – P. 2951–2959.

36 Bergstra, J. Algorithms for hyper-parameter optimization / J. Bergstra, R. Bardenet, Y. Bengio, B. Kégl // Advances in Neural Information Processing Systems 24. – 2011. – P. 2546–2554.

37 Akiba, T. Optuna: a next-generation hyperparameter optimization framework / T. Akiba, S. Sano, T. Yanase [et al.] // Proc. of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. – Anchorage, 2019. – P. 2623–2631.

38 Bergstra, J. Random search for hyper-parameter optimization / J. Bergstra, Y. Bengio // Journal of Machine Learning Research. – 2012. – Vol. 13. – P. 281–305.

39 Deb, K. A fast and elitist multiobjective genetic algorithm: NSGA-II / K. Deb, A. Pratap, S. Agarwal, T. Meyarivan // IEEE Transactions on Evolutionary Computation. – 2002. – Vol. 6, № 2. – P. 182–197.

40 Zitzler, E. Comparison of multiobjective evolutionary algorithms: empirical results / E. Zitzler, K. Deb, L. Thiele // Evolutionary Computation. – 2000. – Vol. 8, № 2. – P. 173–195.

41 Romera-Paredes, B. Mathematical discoveries from program search with large language models / B. Romera-Paredes, M. Barekatain, A. Novikov [et al.] // Nature. – 2024. – Vol. 625. – P. 468–475.

42 Novikov, A. AlphaEvolve: a coding agent for scientific and algorithmic discovery [Электронный ресурс] / A. Novikov, N. Vñ, M. Eisenberger [et al.]. – 2025. – arXiv:2506.13131. – Режим доступа: <https://arxiv.org/abs/2506.13131>. – Дата доступа: 30.09.2026.

43 Blank, J. pymoo: multi-objective optimization in Python / J. Blank, K. Deb // IEEE Access. – 2020. – Vol. 8. – P. 89497–89509.

ПРИЛОЖЕНИЕ А

(справочное)

Справочник функций Python для лабораторных работ

Таблица А.1 – Функции NumPy, SciPy и библиотек машинного обучения, используемые в практикуме

Задача	Функции
Случайные числа (воспроизводимо)	<code>rng = np.random.default_rng(seed); rng.random,</code> <code>rng.integers, rng.normal, rng.uniform, rng.choice(p=...),</code> <code>rng.permutation</code>
Векторные операции	<code>@, np.outer, np.where, np.argsort, np.argmin, np.argmax,</code> <code>np.roll, np.cumsum, np.searchsorted,</code> <code>np.bitwise_xor.accumulate</code>
Линейная алгебра	<code>np.linalg.lstsq, np.linalg.pinv, np.linalg.solve,</code> <code>np.linalg.eigvalsh</code>
Временные ряды	<code>np.lib.stride_tricks.sliding_window_view</code>
Предобработка	<code>sklearn.preprocessing.MinMaxScaler,</code> <code>sklearn.model_selection.train_test_split</code>
Модели для сравнения	<code>Perceptron, LinearRegression, SGDRegressor, MLPRegressor</code> (<code>sklearn</code>); <code>torch.nn.Sequential, torch.optim; MiniSom;</code> <code>KMeans</code>
Метрики	<code>mean_squared_error, r2_score, adjusted_rand_score</code> (<code>sklearn.metrics</code>); <code>som.quantization_error,</code> <code>som.topographic_error</code>
Эталонные решения и проверка	<code>scipy.optimize.minimize_scalar, differential_evolution,</code> <code>least_squares(method='lm'), check_grad</code>
Эволюционные библиотеки	<code>deap.tools (selTournament, cxUniform, cxBlend,</code> <code>cxPartiallyMatched, cxOrdered); pygad.GA</code>
Коммивояжёр	<code>scipy.spatial.distance.cdist, np.triu_indices, пакет</code> <code>tsplib95</code>
Безградиентная оптимизация	<code>cma.CMAEvolutionStrategy, cma.fmin2;</code> <code>optuna.create_study, TPESampler, CmaEsSampler,</code> <code>NSGAIISampler; pymoo: NSGA2, minimize,</code> <code>get_problem("zdt1"), HV, IGD</code>
Подбор гиперпараметров	<code>sklearn.model_selection.cross_val_score,</code> <code>StratifiedKFold, GridSearchCV, RandomizedSearchCV;</code> <code>sklearn.datasets.load_wine, load_digits,</code> <code>load_breast_cancer</code>
Проверка и запуск кода модели	<code>ast.parse, ast.walk, subprocess.run(..., timeout=...),</code> <code>concurrent.futures.ThreadPoolExecutor;</code> <code>anthropic.Anthropic().messages.create</code>
Графики и анимация	<code>plt.contourf, plt.scatter,</code> <code>matplotlib.animation.FuncAnimation,</code> <code>plotly.graph_objects.FigureWidget</code>
Интерфейс в Jupyter	<code>ipywidgets: IntSlider, FloatText, Dropdown, Button, Play,</code> <code>interact</code>

ПРИЛОЖЕНИЕ Б

(справочное)

Листинги программ

Операторы для задачи коммивояжёра (ЛР №7): чтение файла TSPLIB, матрица расстояний EUC_2D, циклический кроссинговер и векторный локальный поиск 2-орт; последняя строка проверяет CX на примере подраздела 7.4.

```
import numpy as np

def read_tsp(path):
    """Координаты городов из раздела NODE_COORD_SECTION файла TSPLIB."""
    rows = [l.split() for l in open(path) if l.split() and l.split()[0].isdigit()]
    return np.array([[float(r[1]), float(r[2])] for r in rows])

def dist_matrix(xy):
    """Расстояния EUC_2D: евклидовы, округлённые до целого."""
    return np.floor(np.hypot(*(xy[:, None] - xy[None])).transpose(2, 0, 1)) + 0.5)

def tour_len(tour, D):
    return D[tour, np.roll(tour, -1)].sum()

def cx(p1, p2):
    """Циклический кроссинговер: циклы позиций берутся попеременно."""
    n = len(p1)
    pos1 = np.empty(n, int); pos1[p1] = np.arange(n)    # позиция города в p1
    c1, c2 = p1.copy(), p2.copy()
    seen, cyc = np.zeros(n, bool), 0
    for start in range(n):
        if seen[start]:
            continue
        i = start
        while not seen[i]:
            seen[i] = True
            if cyc % 2:
                c1[i], c2[i] = p2[i], p1[i]            # нечётные циклы - обмен
            i = pos1[p2[i]]
        cyc += 1
    return c1, c2

def two_opt(tour, D):
    """Локальный поиск 2-орт: лучшая замена пары рёбер ищется векторно среди всех пар."""
    n, t = len(tour), tour.copy()
    I, J = np.triu_indices(n, 2)
    keep = ~(I == 0) & (J == n - 1)                # рёбра, смежные через замыкание тура
    I, J = I[keep], J[keep]
    while True:
        a, b, c, d = t[I], t[I + 1], t[J], t[(J + 1) % n]
        gain = D[a, c] + D[b, d] - D[a, b] - D[c, d]    # формула (7.4) для всех пар сразу
        m = int(np.argmin(gain))
        if gain[m] > -1e-9:
            return t
        t[I[m] + 1:J[m] + 1] = t[I[m] + 1:J[m] + 1][::-1]    # инверсия отрезка b...c

p1 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9]) - 1
p2 = np.array([4, 5, 2, 1, 8, 7, 6, 9, 3]) - 1
print((cx(p1, p2)[0] + 1).tolist())    # [1, 5, 2, 4, 8, 6, 7, 9, 3]
```

СМА-ES и IPOP-СМА-ES (ЛР №8) по учебнику Н. Хансена [34]:
 порождение точек, взвешенная рекомбинация, пути эволюции, ранг-1 и ранг- μ обновления ковариационной матрицы, кумулятивная адаптация шага.

```
import numpy as np

def cmaes(func, x0, sigma0, bounds, budget, seed=0, lam=None, trace=None):
    """(mu/mu_w, lambda)-CMA-ES по учебнику Н. Хансена (2016): минимизация func.

    func принимает матрицу (lam, n) и возвращает вектор значений.
    Возвращает лучшую точку, её значение и историю (число вычислений, лучшее f);
    в список trace, если он передан, записываются (m, sigma, C) каждого поколения.
    """
    rng = np.random.default_rng(seed)
    lo, hi = bounds[:, 0], bounds[:, 1]
    n = len(x0)
    lam = lam or 4 + int(3 * np.log(n))          # размер выборки
    mu = lam // 2                                # число родителей
    w = np.log(mu + 0.5) - np.log(np.arange(1, mu + 1))  # веса рекомбинации
    w /= w.sum()
    mueff = 1 / np.sum(w ** 2)
    cc = (4 + mueff / n) / (n + 4 + 2 * mueff / n)  # скорости обучения
    cs = (mueff + 2) / (n + mueff + 5)
    c1 = 2 / ((n + 1.3) ** 2 + mueff)
    cmu = min(1 - c1, 2 * (mueff - 2 + 1 / mueff) / ((n + 2) ** 2 + mueff))
    damp = 1 + 2 * max(0, np.sqrt((mueff - 1) / (n + 1)) - 1) + cs
    chi_n = np.sqrt(n) * (1 - 1 / (4 * n) + 1 / (21 * n ** 2))  # E||N(0, I)||

    m, sigma = np.asarray(x0, float), sigma0
    C, B, D = np.eye(n), np.eye(n), np.ones(n)      # C = B diag(D^2) B^T
    pc, ps = np.zeros(n), np.zeros(n)              # пути эволюции
    best_x, best_f, evals, hist, g = None, np.inf, 0, [], 0
    while evals < budget:
        if trace is not None:
            trace.append((m.copy(), sigma, C.copy()))
        z = rng.standard_normal((lam, n))
        y = z @ (B * D).T                          # y ~ N(0, C)
        X = np.clip(m + sigma * y, lo, hi)
        F = func(X)
        evals += lam
        order = np.argsort(F)
        if F[order[0]] < best_f:
            best_f, best_x = F[order[0]], X[order[0]].copy()
        hist.append((evals, best_f))
        y_sel = y[order[:mu]]
        yw = w @ y_sel                              # взвешенный шаг среднего
        m = m + sigma * yw
        # адаптация шага: путь ps в отбелённых координатах
        ps = (1 - cs) * ps + np.sqrt(cs * (2 - cs) * mueff) * (B @ ((B.T @ yw) / D))
        g += 1
        hsig = np.linalg.norm(ps) / np.sqrt(1 - (1 - cs) ** (2 * g)) / chi_n < 1.4 + 2 / (n + 1)
        pc = (1 - cc) * pc + hsig * np.sqrt(cc * (2 - cc) * mueff) * yw
        # ранг-1 и ранг-му обновления ковариационной матрицы
        C = ((1 - c1 - cmu) * C + c1 * (np.outer(pc, pc) + (1 - hsig) * cc * (2 - cc) * C)
            + cmu * (y_sel.T * w) @ y_sel)
        sigma *= np.exp(cs / damp * (np.linalg.norm(ps) / chi_n - 1))
        C = (C + C.T) / 2
        D2, B = np.linalg.eigh(C)
        D = np.sqrt(np.maximum(D2, 1e-30))
        if sigma * D.max() < 1e-14 * (hi - lo).max():  # распределение выродилось
            break
    return best_x, best_f, np.array(hist)

def ipop_cmaes(func, bounds, budget, seed=0, sigma_frac=0.3):
```

```

"""IPOP-CMA-ES: перезапуски из случайной точки с удвоением популяции."""
rng = np.random.default_rng(seed)
lam, used, best_x, best_f, hist, k = 4 + int(3 * np.log(len(bounds))), 0, None, np.inf, [], 0
while used < budget:
    x0 = rng.uniform(bounds[:, 0], bounds[:, 1])
    x, f, h = cmaes(func, x0, sigma_frac * np.ptp(bounds, axis=1).max(), bounds, budget - used
    ,
                    seed=seed * 1000 + k, lam=lam)
    hist += [(used + e, min(v, best_f)) for e, v in h]          # лучшее с учётом прошлых запусков
    if f < best_f:
        best_x, best_f = x, f
    used, lam, k = used + int(h[-1, 0]), 2 * lam, k + 1
return best_x, best_f, np.array(hist)

if __name__ == "__main__":
    sphere = lambda X: np.sum(X ** 2, axis=1)
    bounds = np.array([[-5.0, 5.0]] * 10)
    x, fx, _ = cmaes(sphere, np.full(10, 3.0), 2.0, bounds, budget=6000)
    print(f"сфера, n = 10: f = {fx:.1e}")                      # порядка 1e-15 и меньше

```

Мини-FunSearch (ЛР №9): генерация примеров и оценка эвристики упаковки, классические правила, извлечение и статическая проверка кода от языковой модели, оценка в отдельном процессе с тайм-аутом, вызов модели через Anthropic SDK или Claude Code CLI и эволюционный цикл.

```

"""Мини-FunSearch: эволюция эвристики онлайн-упаковки в контейнеры с помощью LLM (ЛР 9)."""
import ast
import json
import re
import subprocess
import sys
import tempfile

import numpy as np

# ----- задача и оценка
def make_instances(dist, n_items, seeds, capacity=100):
    """Наборы размеров предметов: список целочисленных массивов."""
    out = []
    for s in seeds:
        rng = np.random.default_rng(s)
        if dist == "weibull":
            # Weibull(k = 3, lambda = 45), как в
            FunSearch
            x = 45 * rng.weibull(3, n_items)
        elif dist == "uniform":
            x = rng.uniform(20, 100, n_items)
        elif dist == "normal":
            x = rng.normal(50, 15, n_items)
        elif dist == "bimodal":
            x = np.where(rng.random(n_items) < 0.5, rng.normal(30, 5, n_items), rng.normal(65, 5,
n_items))
        else:
            raise ValueError(dist)
        out.append(np.clip(np.round(x), 1, capacity).astype(int))
    return out

def pack(items, priority, capacity=100):
    """Онлайн-упаковка: предмет идёт в допустимый контейнер с наибольшим приоритетом.

    Кандидаты - открытые контейнеры, где предмет помещается, и один новый (пустой) контейнер.
    Возвращает число использованных контейнеров.
    """
    free = []
    # свободное место в открытых контейнерах

```

```

for item in items:
    fits = [i for i, r in enumerate(free) if r >= item]
    cand = np.array([free[i] for i in fits] + [capacity], dtype=float)
    k = int(np.argmax(priority(float(item), cand, float(capacity))))
    if k < len(fits):
        free[fits[k]] -= item
    else:
        free.append(capacity - item)
return len(free)

def excess(instances, priority, capacity=100):
    """Среднее превышение нижней границы L1 = ceil(sum / C), %: меньше - лучше."""
    res = []
    for items in instances:
        lb = int(np.ceil(items.sum() / capacity))
        res.append(100 * (pack(items, priority, capacity) - lb) / lb)
    return float(np.mean(res))

def first_fit(item, bins, capacity):
    return -np.arange(len(bins), dtype=float) # самый ранний открытый контейнер

def best_fit(item, bins, capacity):
    return -(bins - item) # наименьший остаток после укладки

def worst_fit(item, bins, capacity):
    p = bins - item # наибольший остаток среди открытых
    p[-1] = -np.inf # новый контейнер - только если иначе нельзя
    return p

# ----- проверка кода от LLM
ALLOWED_IMPORTS = {"numpy", "math"}
FORBIDDEN_NAMES = {"open", "exec", "eval", "compile", "__import__", "globals", "locals", "input",
                    "os", "sys", "subprocess", "socket", "shutil", "pathlib", "builtins", "getattr"}
FORBIDDEN_ATTRS = {"load", "save", "savez", "savetxt", "loadtxt", "fromfile", "tofile", "mmap",
                    "genfromtxt", "system", "popen"}

def extract_function(text):
    """Достаёт из ответа модели определение функции priority* и переименовывает его в priority."""
    blocks = re.findall(r'``(?:python)?\s*\n(.*)``', text, flags=re.S) or [text]
    for code in blocks:
        try:
            tree = ast.parse(code)
        except SyntaxError:
            continue
        funcs = [n for n in tree.body if isinstance(n, ast.FunctionDef) and n.name.startswith("priority")]
        if funcs:
            funcs[-1].name = "priority" # последняя версия - новая эвристика
            keep = [n for n in tree.body if isinstance(n, (ast.Import, ast.ImportFrom, ast.Assign))
                    or (isinstance(n, ast.FunctionDef) and not n.name.startswith("priority_v"))]
            return ast.unparse(ast.Module(body=keep, type_ignores=[]))
    return None

def is_safe(code):
    """Статическая проверка: только разрешённые импорты и никаких опасных имён."""
    for node in ast.walk(ast.parse(code)):
        if isinstance(node, ast.Import) and any(a.name.split(".")[0] not in ALLOWED_IMPORTS for a
            in node.names):

```

```

        return False
    if isinstance(node, ast.ImportFrom) and (node.module or "").split(".")[0] not in
ALLOWED_IMPORTS:
        return False
    if isinstance(node, ast.Name) and node.id in FORBIDDEN_NAMES:
        return False
    if isinstance(node, ast.Attribute) and (node.attr.startswith("__") or node.attr in
FORBIDDEN_ATTRS):
        return False
    return True

RUNNER = """
import json, math, sys
import numpy as np
sys.path.insert(0, {path!r})
from funsearch import excess, make_instances
ns = {"np": np, "numpy": np, "math": math}
exec(sys.stdin.read(), ns)
inst = make_instances({dist!r}, {n_items}, {seeds}, {capacity})
print(json.dumps(excess(inst, ns["priority"], {capacity})))
"""

def evaluate_code(code, dist, n_items, seeds, capacity=100, timeout=60):
    """Оценка кода в отдельном процессе с тайм-аутом; None - если код упал, завис или небезопасен.
    """
    if code is None or not is_safe(code):
        return None
    runner = RUNNER.format(path=str(__import__("pathlib").Path(__file__).parent), dist=dist,
                            n_items=n_items, seeds=list(seeds), capacity=capacity)
    try:
        out = subprocess.run([sys.executable, "-c", runner], input=code, capture_output=True, text
=True,
                            timeout=timeout)
        score = json.loads(out.stdout.strip().splitlines()[-1])
        return score if np.isfinite(score) else None
    except (subprocess.TimeoutExpired, json.JSONDecodeError, IndexError, ValueError):
        return None

# ----- ЯЗЫКОВАЯ МОДЕЛЬ
SYSTEM = ("Ты эксперт по алгоритмам и эвристикам комбинаторной оптимизации. "
          "Отвечай только кодом на Python в одном блоке ``python``.")

PROMPT = """Задача: онлайн-упаковка в контейнеры. Предметы с целыми размерами приходят по одному,
ёмкость контейнера {capacity}. Для очередного предмета эвристика получает item - его размер,
bins - свободное место в контейнерах, куда он помещается (последний элемент - новый пустой
контейнер), capacity - ёмкость, и возвращает массив приоритетов той же длины, что bins.
Предмет кладётся в контейнер с наибольшим приоритетом. Цель - как можно меньше контейнеров.
Оценка - превышение нижней границы числа контейнеров, в процентах (меньше - лучше).

{examples}
Напиши функцию priority_v{k}(item, bins, capacity) - улучшенную версию предыдущих.
Используй только numpy (как np) и math. Функция должна быть быстрой и детерминированной."""

def llm_anthropic(prompt, model="claude-opus-5-5", effort="medium"):
    """Вызов через Anthropic SDK (ключ ANTHROPIC_API_KEY или профиль 'ant auth login')."""
    import anthropic
    client = anthropic.Anthropic()
    try:
        resp = client.beta.messages.create(
            model=model, max_tokens=16000, system=SYSTEM,
            messages=[{"role": "user", "content": prompt}],
            output_config={"effort": effort},
            betas=["server-side-fallback-2026-07-01"], fallbacks="default")

```

```

except (anthropic.RateLimitError, anthropic.APIStatusError, anthropic.APIConnectionError) as e
:
    print("LLM error:", type(e).__name__)
    return "", 0.0
if resp.stop_reason == "refusal":
    return "", 0.0
return "".join(b.text for b in resp.content if b.type == "text"), None

def llm_claude_cli(prompt, model="claude-opus-5-5", effort="medium"):
    """Вызов через Claude Code CLI в неинтерактивном режиме, без инструментов и MCP."""
    cmd = ["claude", "-p", prompt, "--model", model, "--effort", effort, "--tools", "",
           "--system-prompt", SYSTEM, "--output-format", "json", "--no-session-persistence",
           "--strict-mcp-config", "--setting-sources", ""]
    try:
        with tempfile.TemporaryDirectory() as cwd:
            out = subprocess.run(cmd, capture_output=True, text=True, timeout=600, cwd=cwd)
            d = json.loads(out.stdout)
            return (" " if d.get("is_error") else d.get("result", "")), d.get("total_cost_usd", 0.0)
    except (subprocess.TimeoutExpired, json.JSONDecodeError):
        return "", 0.0

# ----- эволюционный цикл
def funsearch(seed_codes, llm, dist, n_items, seeds, capacity=100, n_waves=10, per_wave=4,
              k_shot=2, db_size=20, rng=None, log=print):
    """База программ + подсказка из k_shot лучших по турниру + параллельные вызовы LLM."""
    from concurrent.futures import ThreadPoolExecutor
    rng = rng or np.random.default_rng(0)
    db = [] # (оценка, код)
    for code in seed_codes:
        db.append((evaluate_code(code, dist, n_items, seeds, capacity), code))
    history, cost, calls, valid = [], 0.0, 0, 0

    def make_prompt():
        pool = sorted(db)[:db_size]
        idx = sorted({min(rng.integers(0, len(pool), 2)) for _ in range(k_shot)}) # турнир из дв
        ux
        chosen = sorted((pool[i] for i in idx), key=lambda z: -z[0]) # от худшей
        к лучшей
        ex = ""
        for v, (score, code) in enumerate(chosen):
            ex += f"# версия {v}, оценка {score:.3f} %\n" + code.replace("def priority(", f"def
priority_v{v}(") + "\n\n"
        return PROMPT.format(capacity=capacity, examples=ex, k=len(chosen))

    for wave in range(n_waves):
        prompts = [make_prompt() for _ in range(per_wave)]
        with ThreadPoolExecutor(per_wave) as ex:
            answers = list(ex.map(llm, prompts))
        for text, c in answers:
            calls += 1
            cost += c or 0.0
            code = extract_function(text)
            score = evaluate_code(code, dist, n_items, seeds, capacity)
            if score is not None:
                valid += 1
                db.append((score, code))
            history.append(min(s for s, _ in db))
        log(f"волна {wave + 1}: лучшее {history[-1]:.3f} %, корректных {valid}/{calls}, стоимость
        ${cost:.2f}")
    return sorted(db), history, dict(calls=calls, valid=valid, cost=cost)

```

Учебное издание

Лебедевич Артем Владимирович
Нестеренков Сергей Николаевич

**ГЕНЕТИЧЕСКИЕ И ЭВОЛЮЦИОННЫЕ ВЫЧИСЛЕНИЯ:
НЕЙРОННЫЕ СЕТИ И ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ**

УЧЕБНО-МЕТОДИЧЕСКОЕ ПОСОБИЕ

Редактор _____

Корректор _____

Компьютерная правка, оригинал-макет _____

Подписано в печать _____. Формат 60×84 1/16. Бумага офсетная. Гарнитура
«Таймс».

Отпечатано на ризографе. Усл. печ. л. _____. Уч.-изд. л. _____. Тираж _____ экз. Заказ _____.

Издатель и полиграфическое исполнение: учреждение образования
«Белорусский государственный университет информатики и радиоэлектроники».

Свидетельство о государственной регистрации издателя, изготовителя,
распространителя печатных изданий _____.

220013, г. Минск, ул. П. Бровки, 6